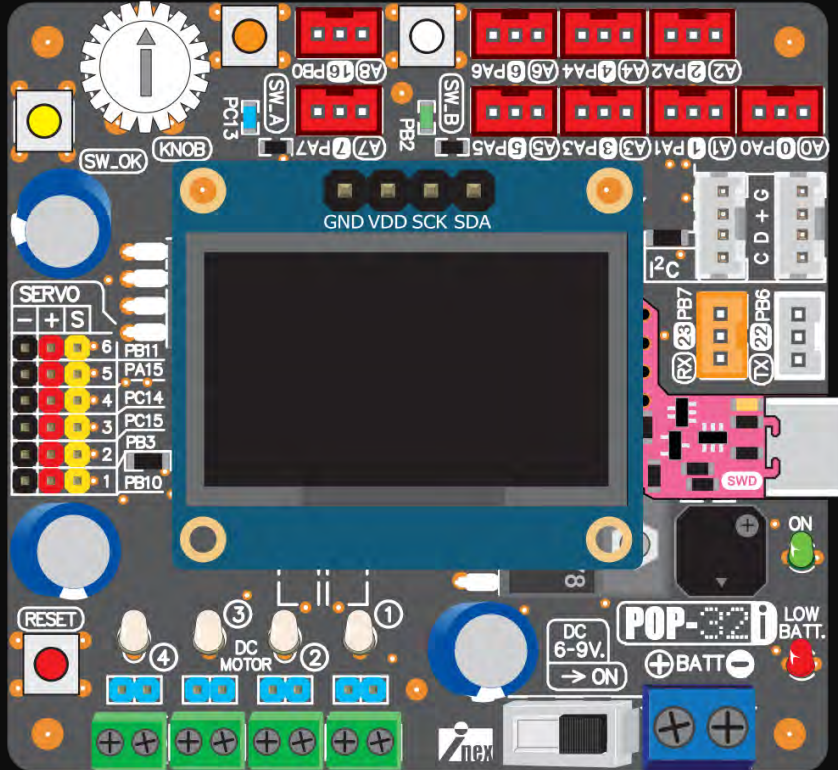


POP-32i

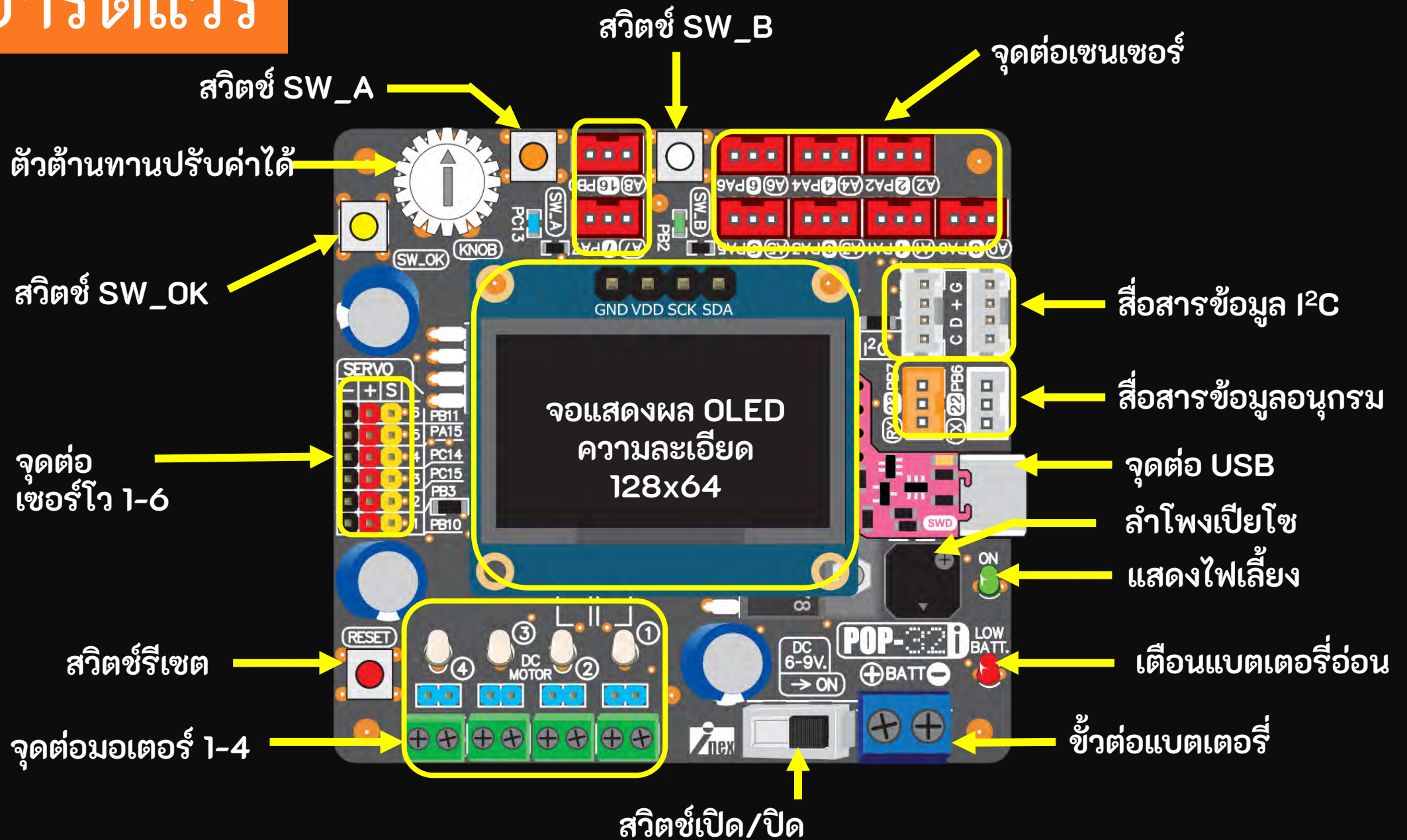


- ใช้ไมโครคอนโทรลเลอร์ 32 บิต เบอร์ STM32F103CBT6 ของ STMicroelectronics
- หน่วยความจำแฟลช 128KB
- หน่วยความจำแรม 20KB

เอกสารที่ใช้ในการอบรม

<https://tinyurl.com/RoboRescue2024>

ส่วนฮาร์ดแวร์



ติดตั้งซอฟต์แวร์

PROFESSIONAL EDUCATION STORE

Search on Arduino.cc

SIGN IN

1 SOFTWARE CLOUD DOCUMENTATION COMMUNITY BLOG ABOUT

Downloads

Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file

Linux ApplImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

2

Help

1.เข้าไปดาวน์โหลดตัวติดตั้ง Arduino จากลิง <https://www.arduino.cc>

ติดตั้งซอฟต์แวร์

PROFESSIONAL EDUCATION STORE

Search on Arduino.cc

SIGN IN

SOFTWARE CLOUD DOCUMENTATION COMMUNITY BLOG ABOUT

Downloads

Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file

Linux Applmage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)

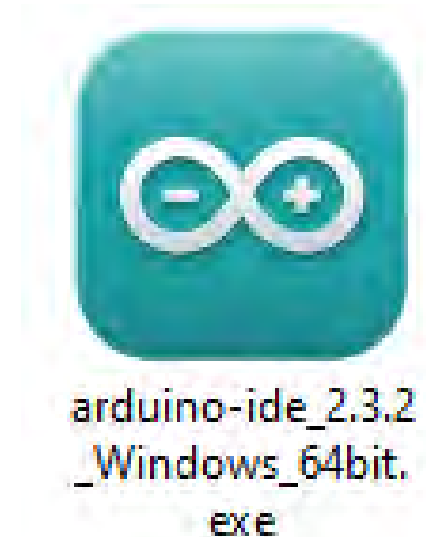
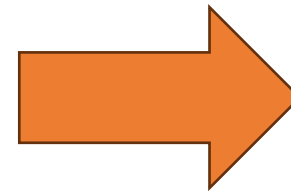
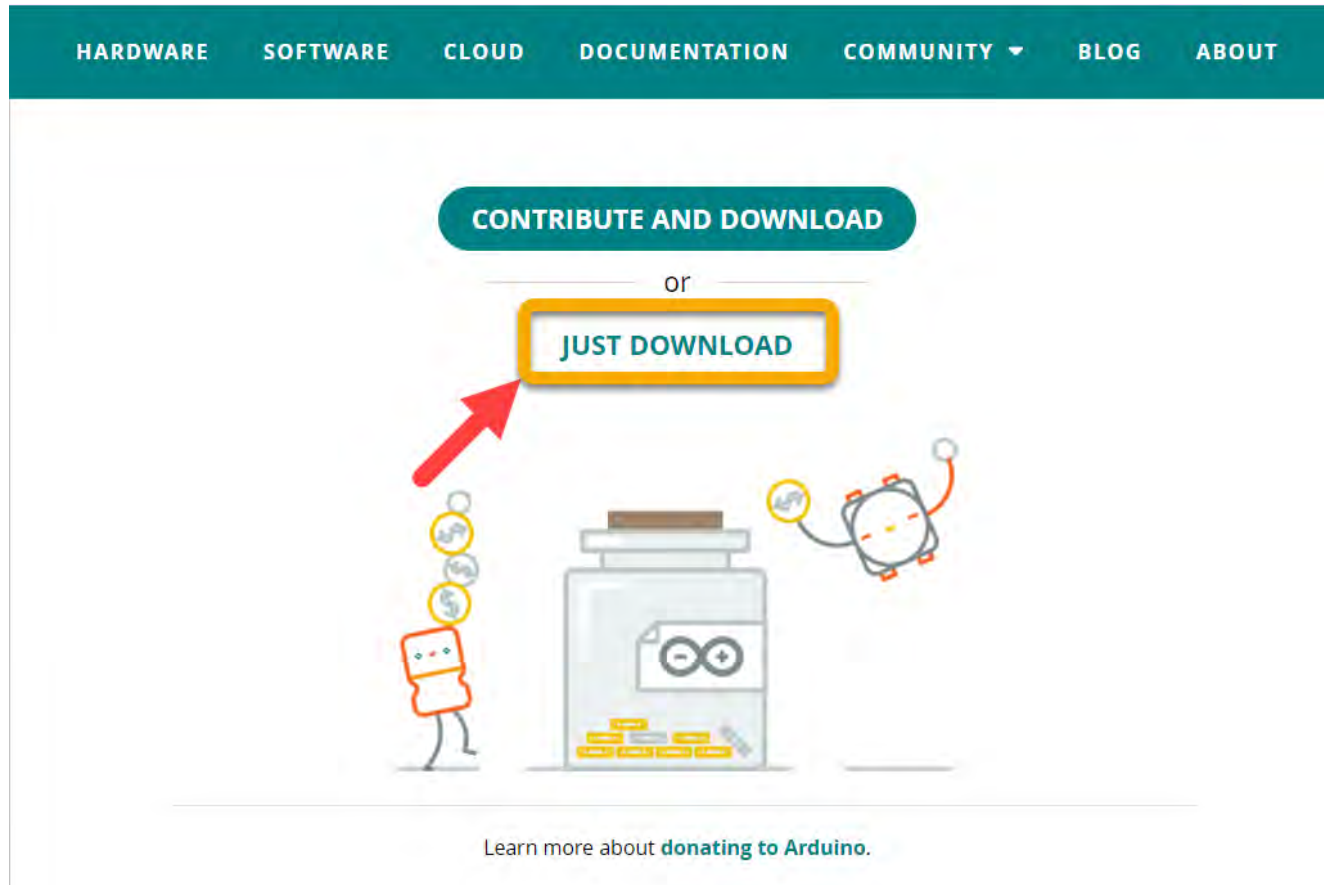
macOS Intel, 10.15: "Catalina" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Help

2. ในที่นี้เลือกเป็นเวอร์ชัน 2.3.x

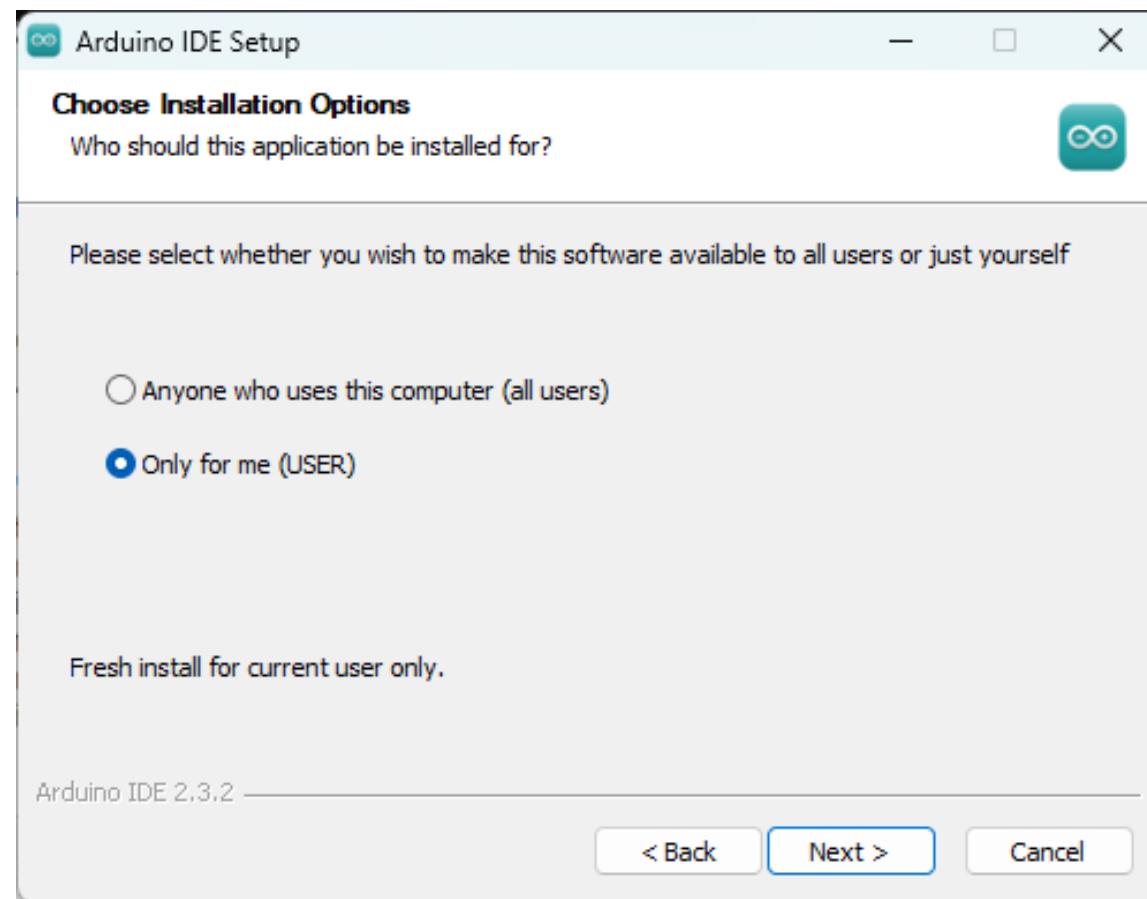
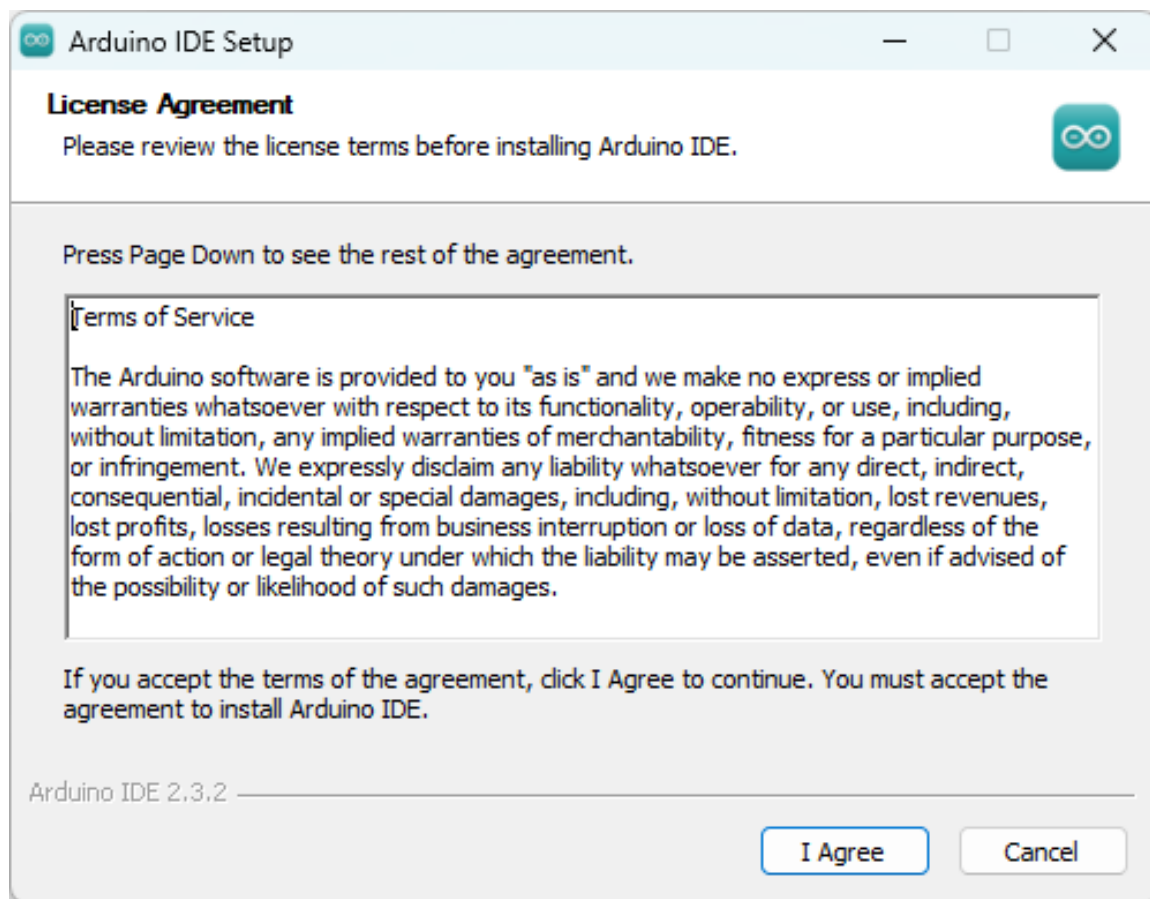
ติดตั้งซอฟต์แวร์



3. เลือกตามขั้นตอนการดาวน์โหลด

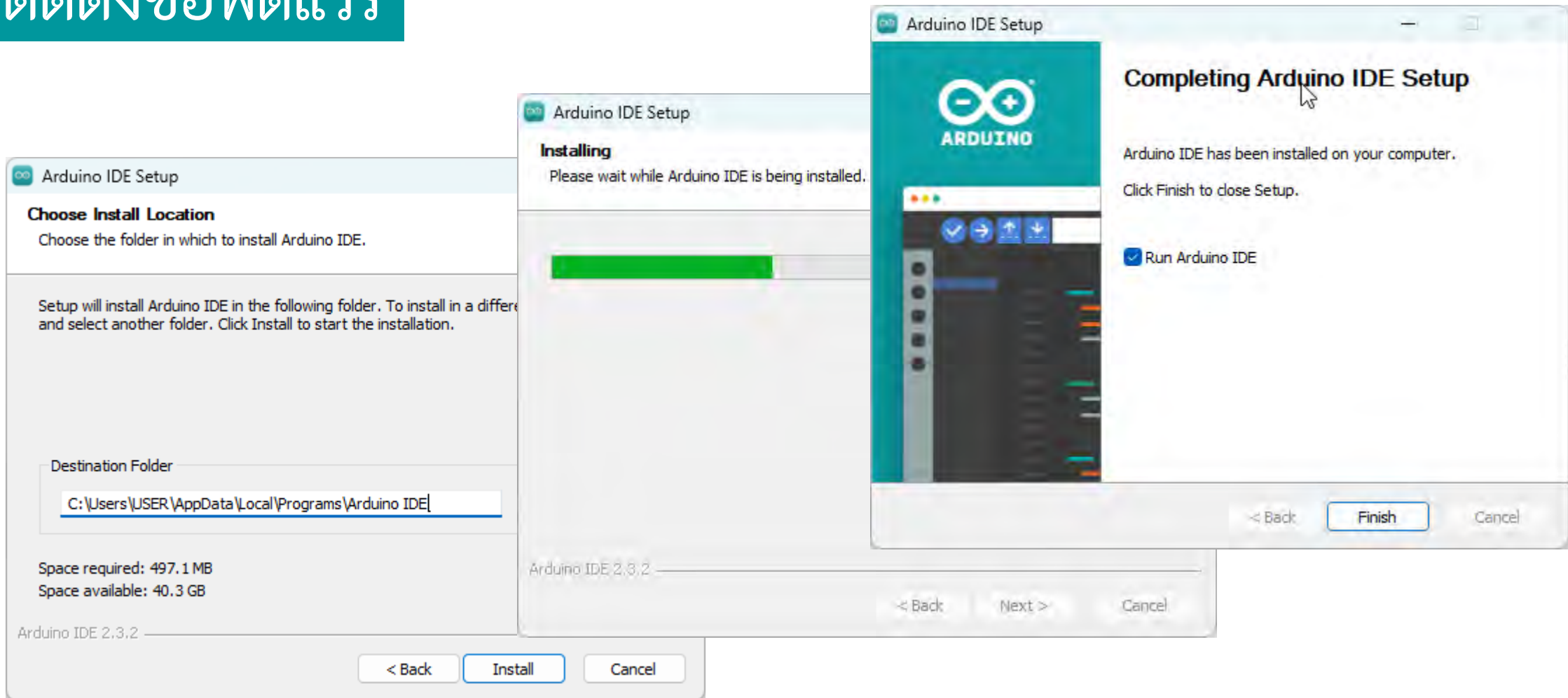
4. รอจนกระทั่งตัวติดตั้ง Arduino ดาวน์โหลดเข้ามายังคอมพิวเตอร์

ติดตั้งซอฟต์แวร์



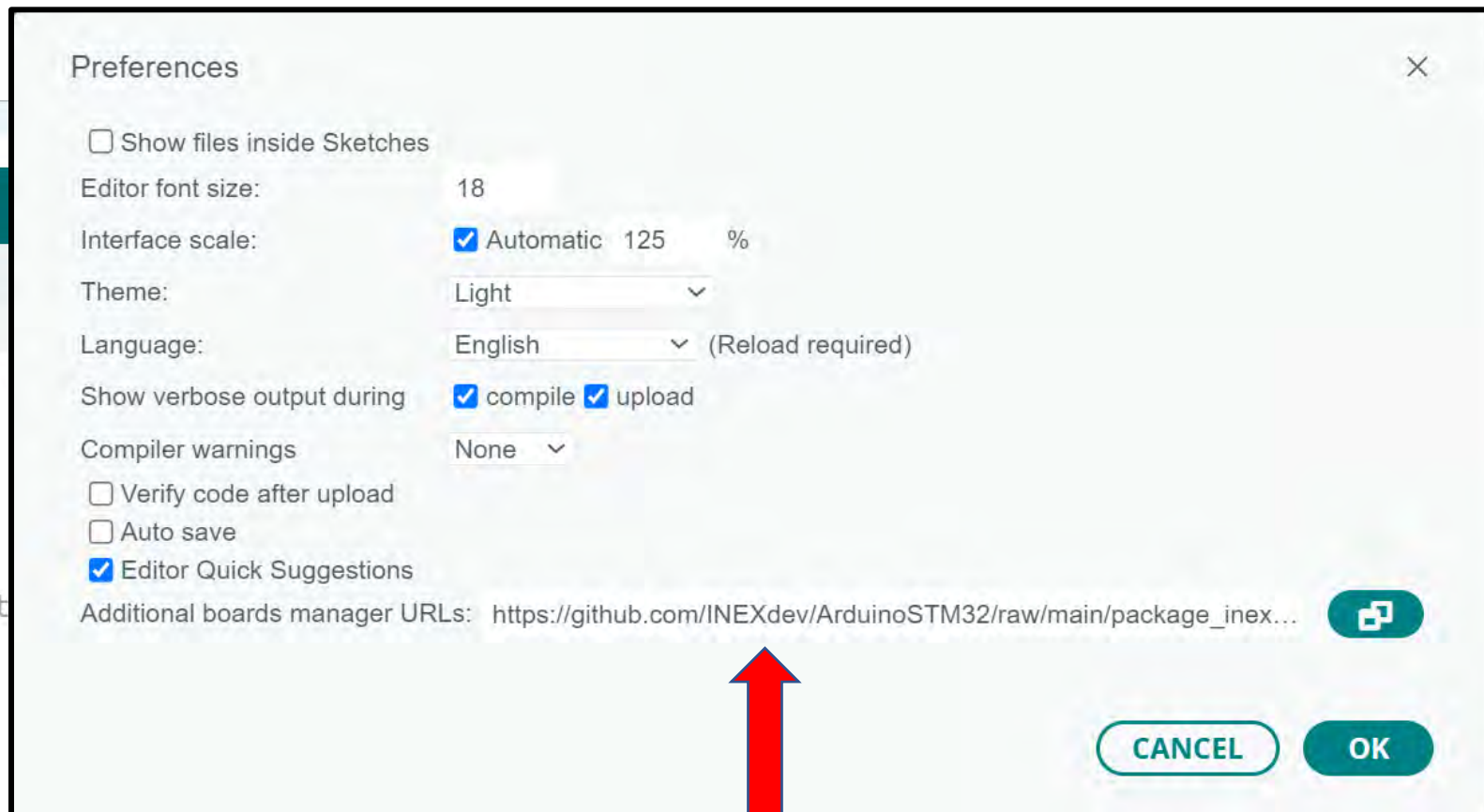
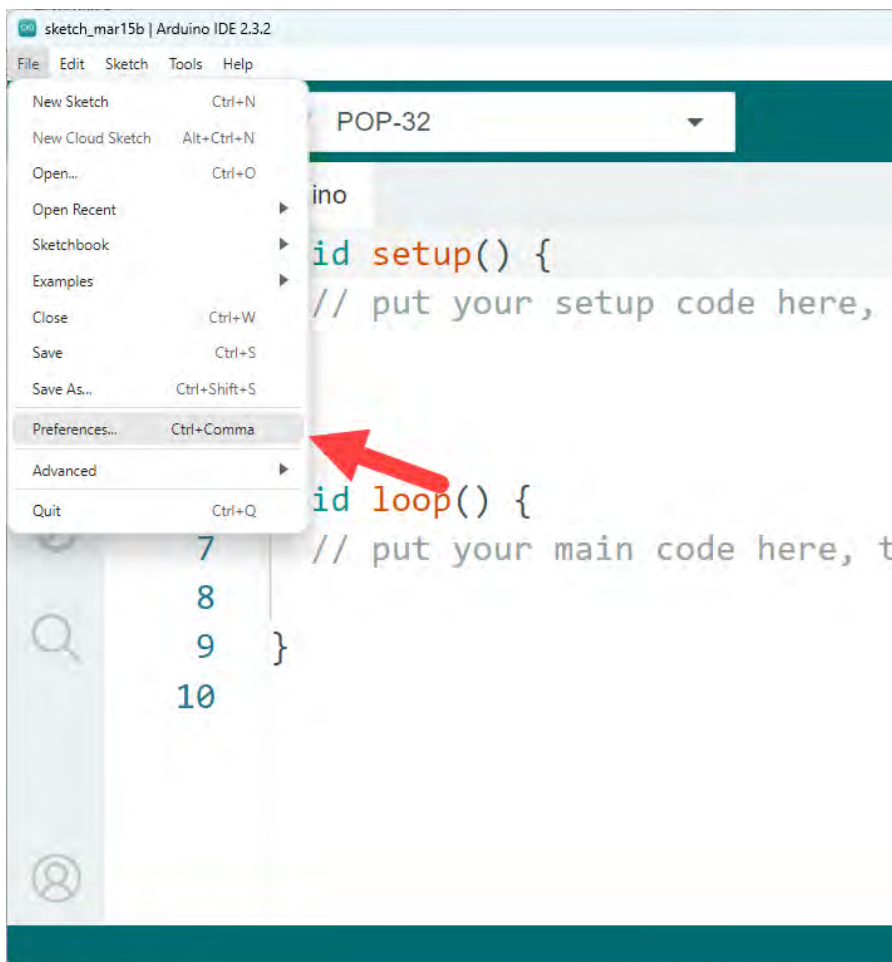
5. ทำการติดตั้งโปรแกรม Arduino ลักษณะเช่นเดียวกับโปรแกรมทั่วไป

ติดตั้งซอฟต์แวร์



6. หลังจากติดตั้งเสร็จสมบูรณ์เปิดโปรแกรม Arduino ขึ้นมา

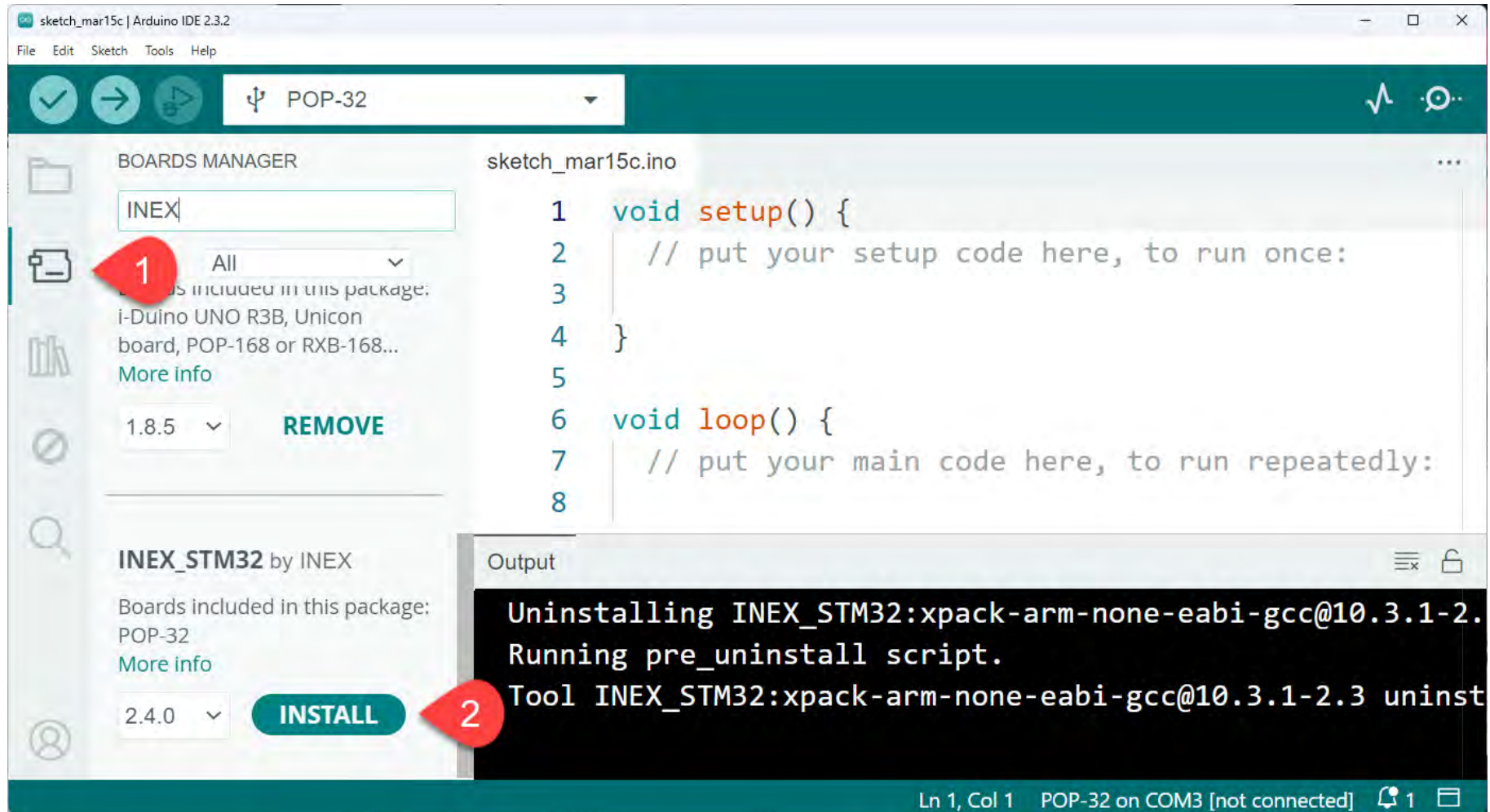
ติดตั้งซอฟต์แวร์



https://github.com/INEXdev/ArduinoSTM32/raw/main/package_inex_stm32_index.json

7. ตั้งค่าโปรไฟล์บอร์ด INEX_STM32 สำหรับบอร์ด POP-32

ติดตั้งซอฟต์แวร์



8. ตั้งค่าสำหรับในส่วน Boards Manager

ติดตั้งซอฟต์แวร์

```
Output
Installing INEX_STM32:CMSIS@5.7.0
Configuring tool.
INEX_STM32:CMSIS@5.7.0 installed
Installing platform INEX_STM32:stm32@2.4.0
Configuring platform.
Platform INEX_STM32:stm32@2.4.0 installed
```

9. รอจนกระทั่งติดตั้ง package เสร็จสมบูรณ์

การอัปโหลดโปรแกรมผ่านช่อง สื่อสารอนุกรม

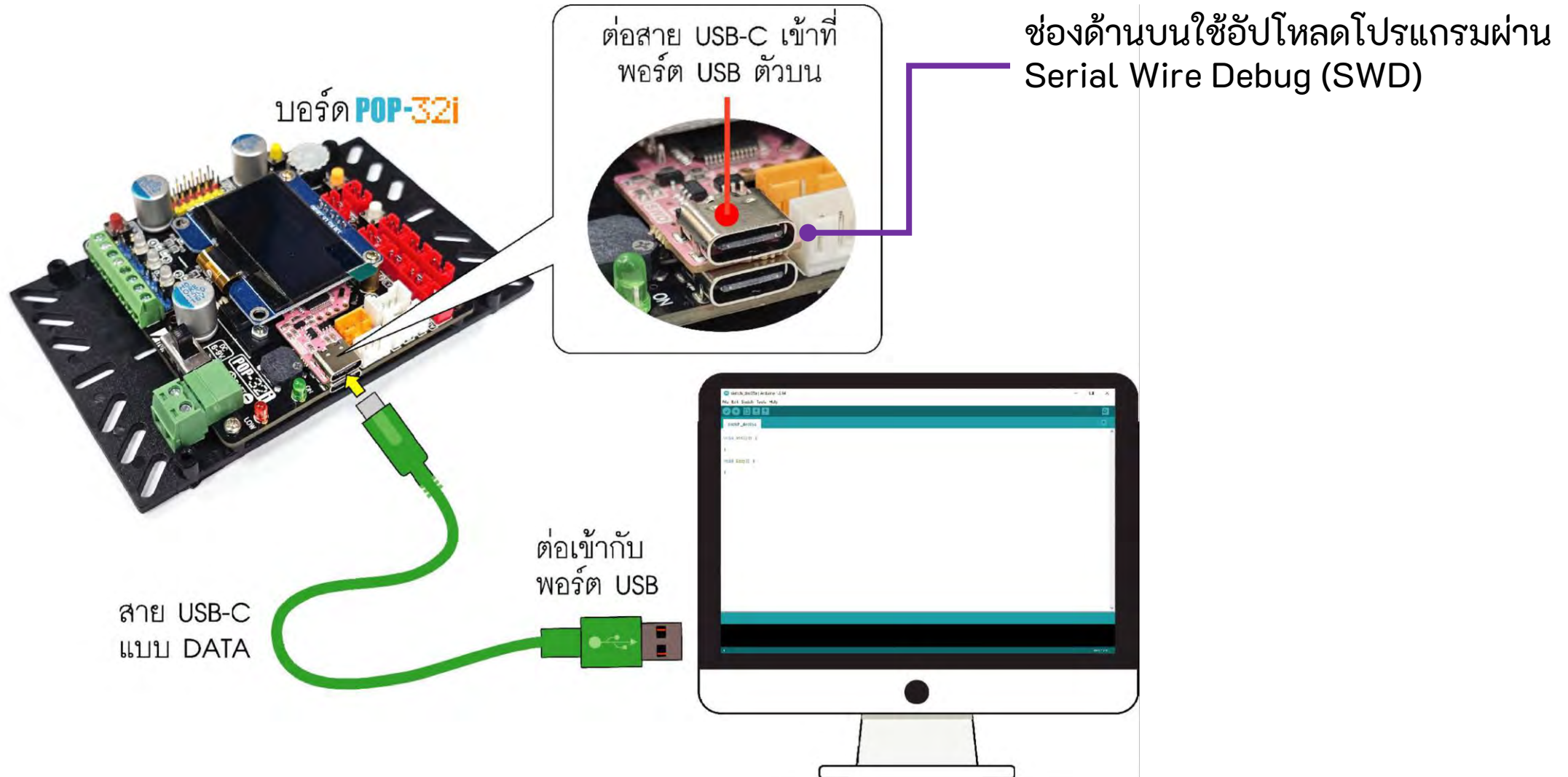
ข้อดี

สามารถใช้งาน Serial Monitor ได้ทันทีโดยไม่ต้องเปลี่ยนช่อง USB

ข้อเสีย

หากส่วน HID Bootloader บนบอร์ด POP-32 เกิดความเสียหาย ส่งผลให้ไม่สามารถอัปโหลดโปรแกรมผ่านช่องทางนี้ได้

อัปโหลดโปรแกรมผ่านช่อง SWD



ติดตั้ง STM32 Cube Programmer

ดาวน์โหลดโปรแกรม

<https://www.st.com/en/development-tools/stm32cubeprog.html>

Get Software					
	Part Number	General Description	Latest version	Download	All versions
+	STM32CubePrg-Lin	STM32CubeProgrammer software for Linux	2.13.0	Get latest	Select version
+	STM32CubePrg-Mac	STM32CubeProgrammer software for Mac	2.13.0	Get latest	Select version
+	STM32CubePrg-W32	STM32CubeProgrammer software for Win32	2.13.0	Get latest	Select version
+	STM32CubePrg-W64	STM32CubeProgrammer software for Win64	2.13.0	Get latest	Select version

ขั้นตอนดาวน์โหลดจะต้องลงทะเบียน

License Agreement

กดปุ่มยอมรับข้อตกลง



ACCEPT

Please indicate your acceptance or NON-acceptance by selecting "I ACCEPT" or "I DO NOT ACCEPT" as indicated below in the media.

BY INSTALLING COPYING, DOWNLOADING, ACCESSING OR OTHERWISE USING THIS SOFTWARE PACKAGE OR ANY PART THEREOF (AND THE RELATED DOCUMENTATION) FROM STMICROELECTRONICS INTERNATIONAL N.V, SWISS BRANCH AND/OR ITS AFFILIATED COMPANIES (STMICROELECTRONICS), THE RECIPIENT, ON BEHALF OF HIMSELF OR HERSELF, OR ON BEHALF OF ANY ENTITY BY WHICH SUCH RECIPIENT IS EMPLOYED AND/OR ENGAGED AGREES TO BE BOUND BY THIS SOFTWARE PACKAGE LICENSE AGREEMENT.

Under STMicroelectronics' intellectual property rights and subject to applicable licensing terms for any third-party software incorporated in this software package and applicable Open Source Terms (as defined here below), the redistribution, reproduction and use in source and binary forms of the software package or any part thereof, with or without modification, are permitted provided that the following conditions are met:

Get Software

If you have an account on my.st.com, login and download the software without any further validation steps.

Login/Register

If you don't want to login now, you can download the software by simply providing your name and e-mail address in the form below and validating it.

This allows us to stay in contact and inform you about updates of this software.

For subsequent downloads this step will not be required for most of our software.

กรอกชื่อและนามสกุล Email
จากนั้นกดปุ่ม Download

First Name:

Last Name:

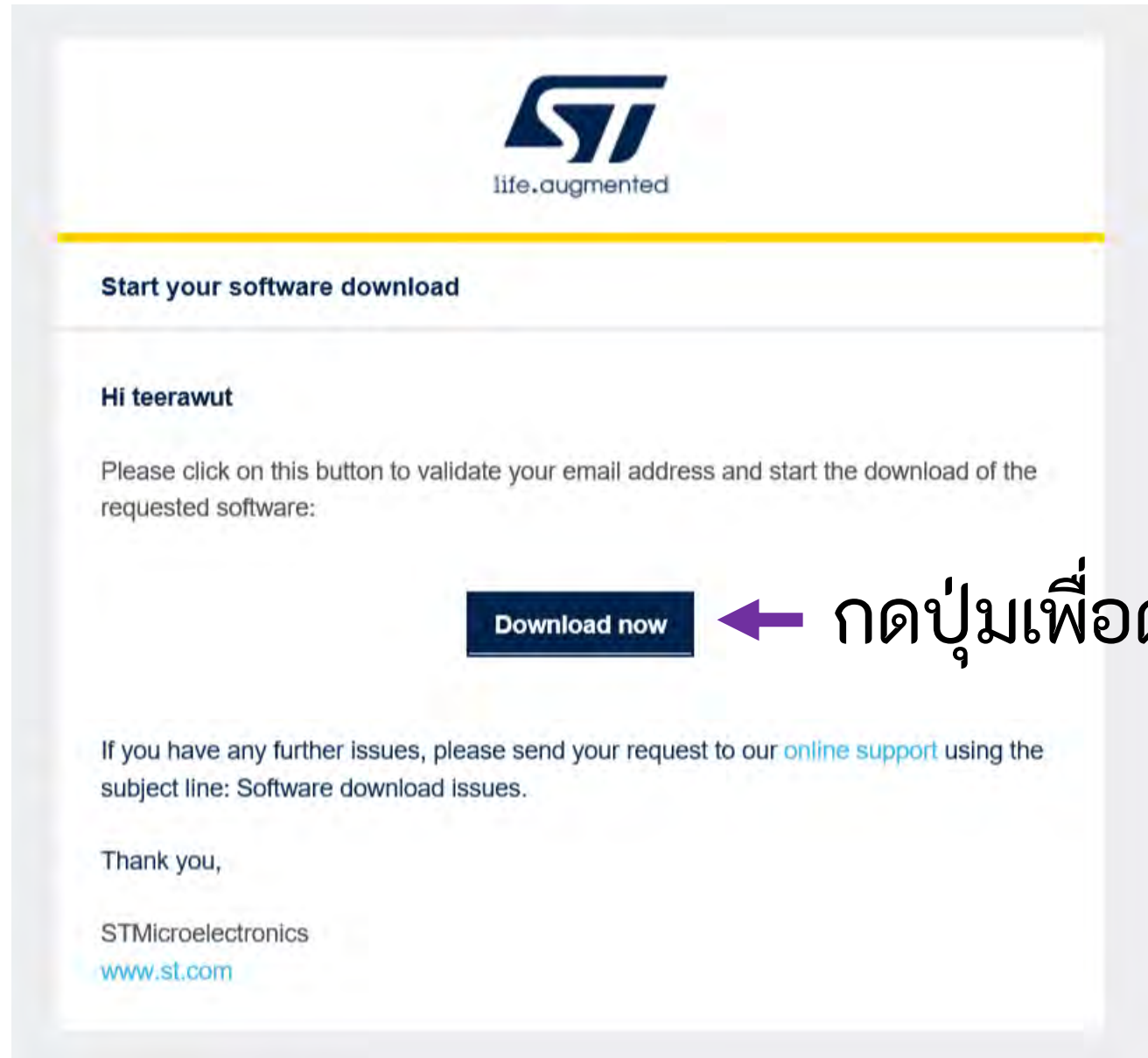
E-mail address:

Please review our [Privacy Statement](#) that describes how we process your profile information and how to assert your personal data protection rights

☐ Please keep me informed about future updates for this software or new software in the same category

Download

เปิด Email ที่ได้ลงทะเบียนไว้



← กดปุ่มเพื่อดาวน์โหลด

ได้ไฟล์ Zip จากนั้นแตกไฟล์



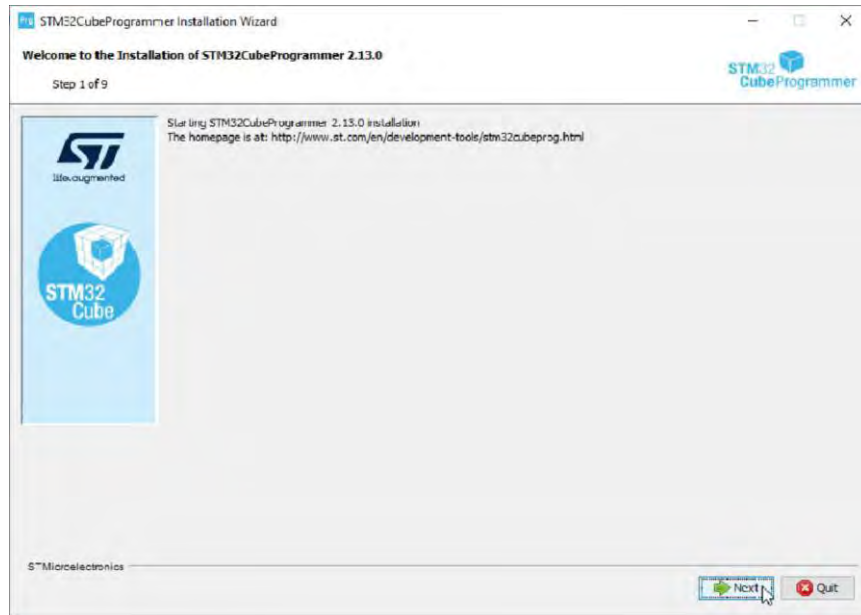
en.stm32cubeprg-win32-
v2-13-0.zip



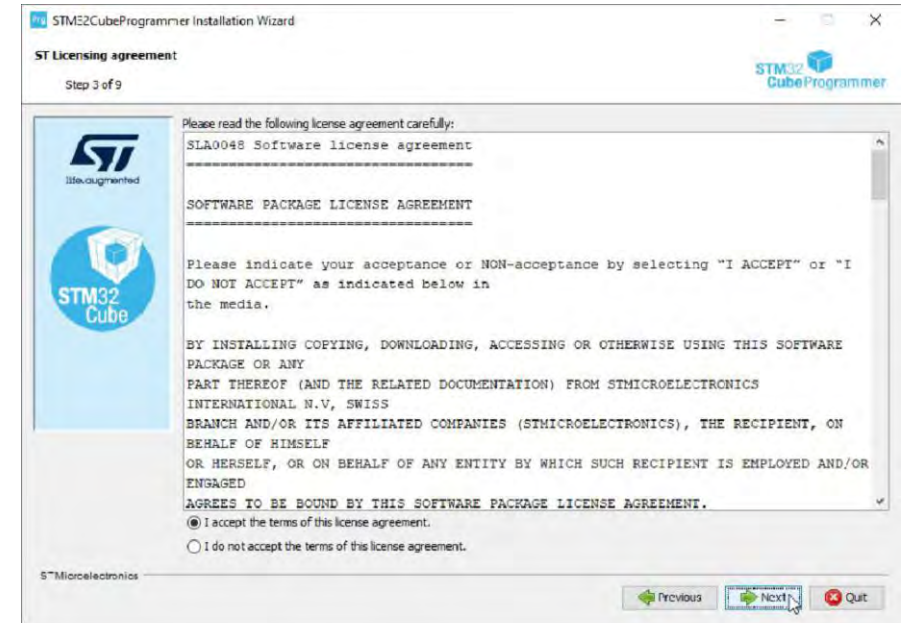
SetupSTM32CubeProgr
ammer_win32.exe

ดับเบิลคลิกเพื่อทำการติดตั้ง

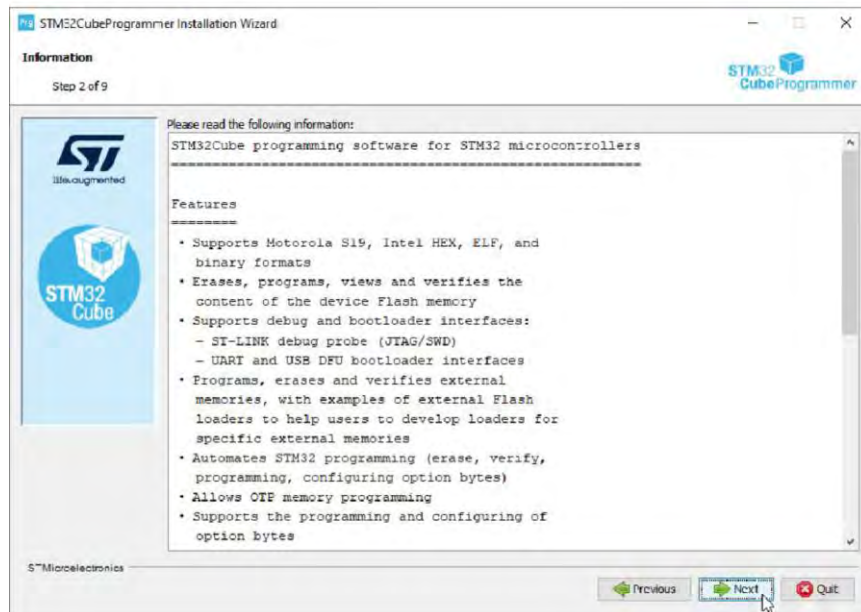
1



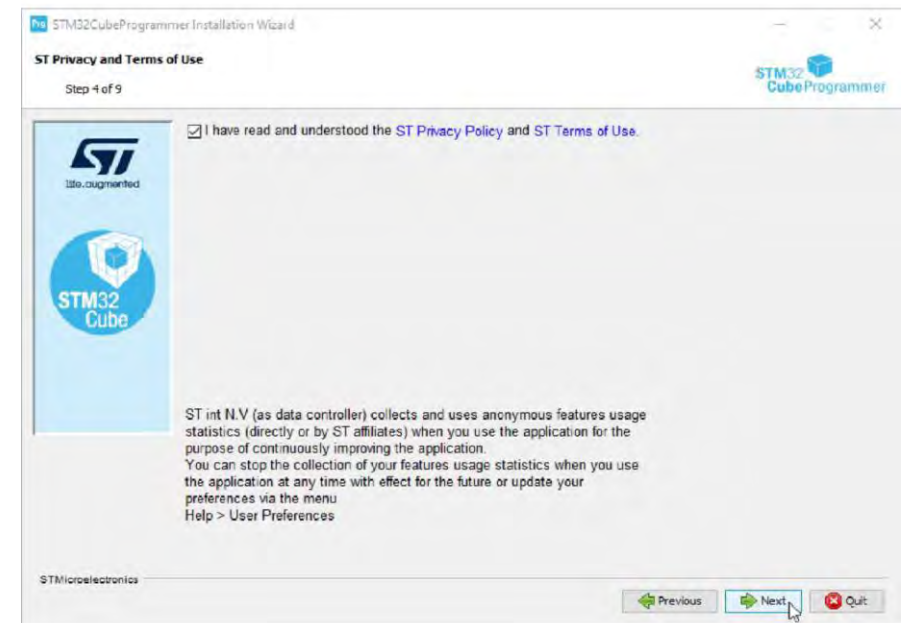
3



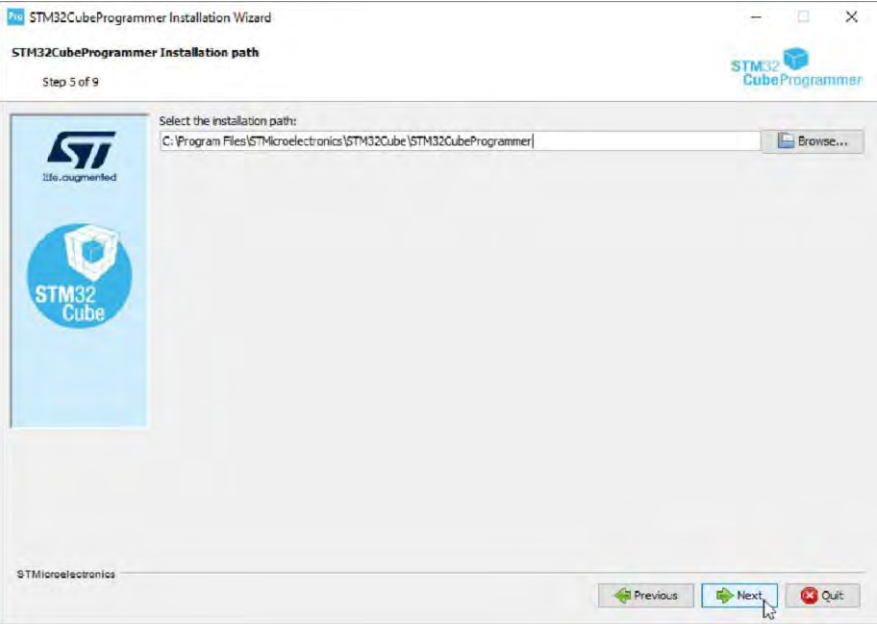
2



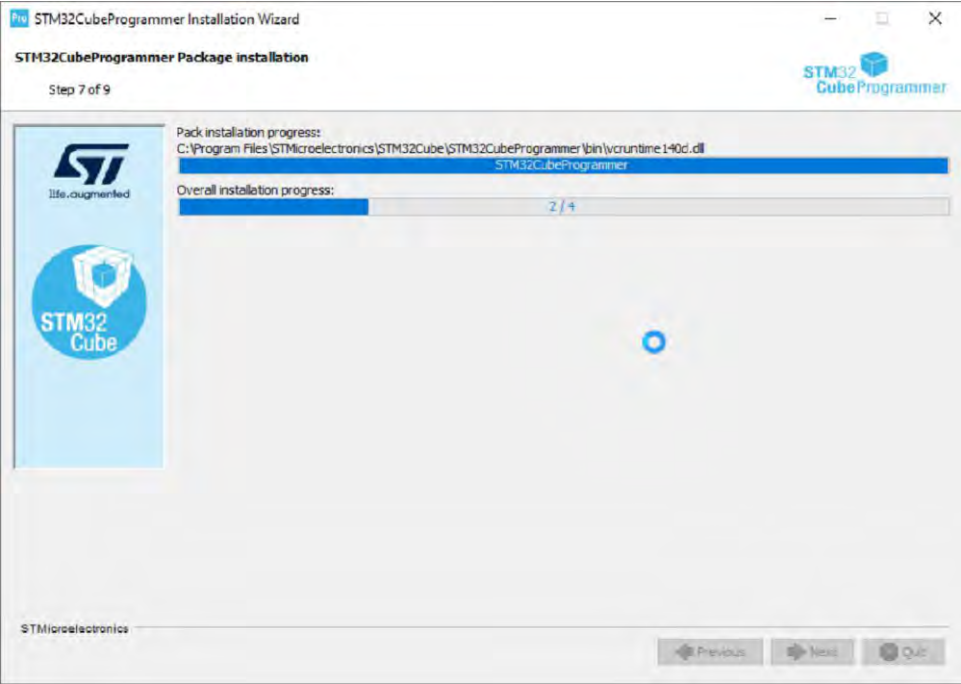
4



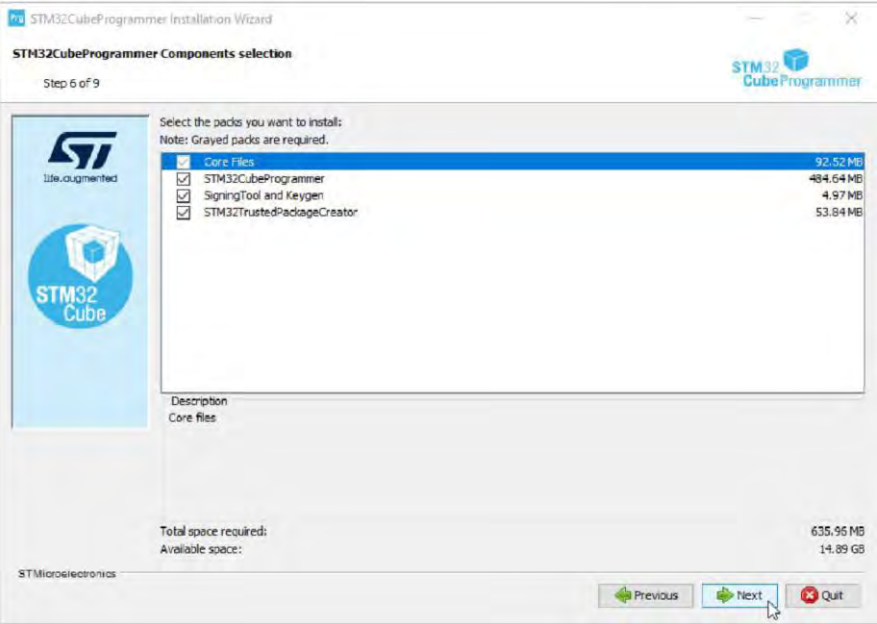
5



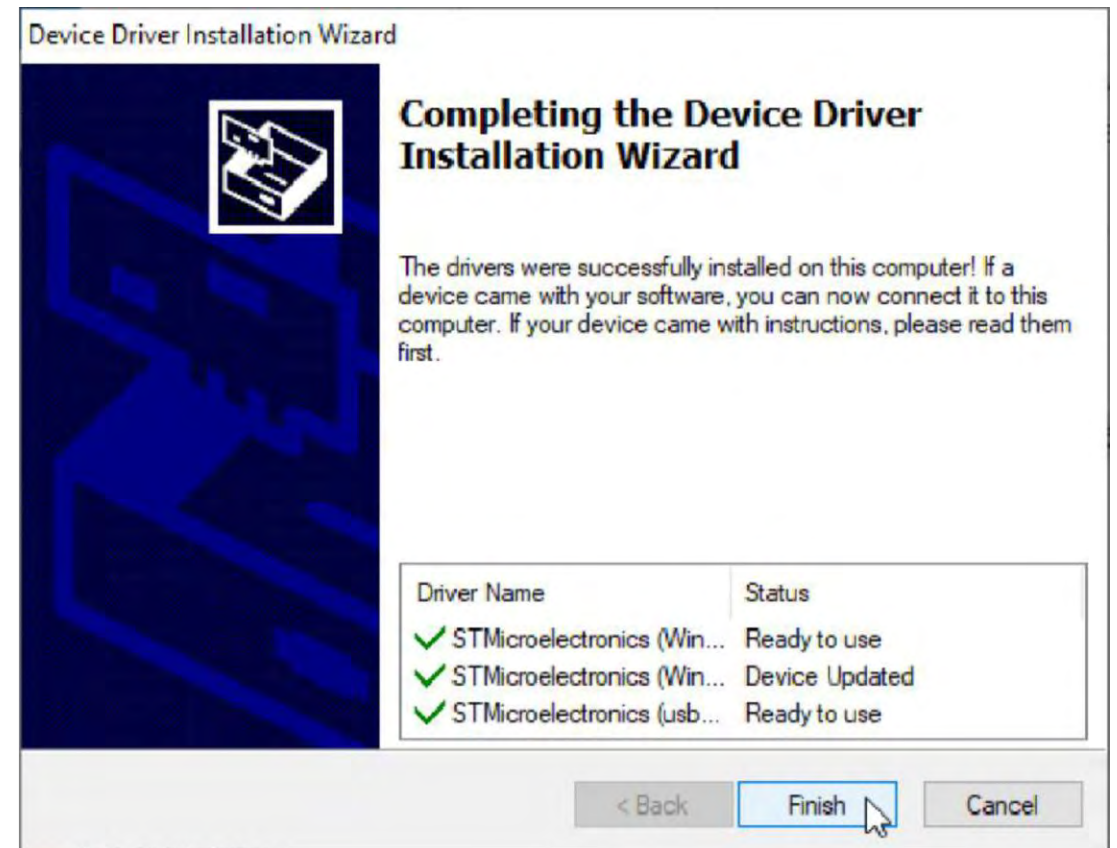
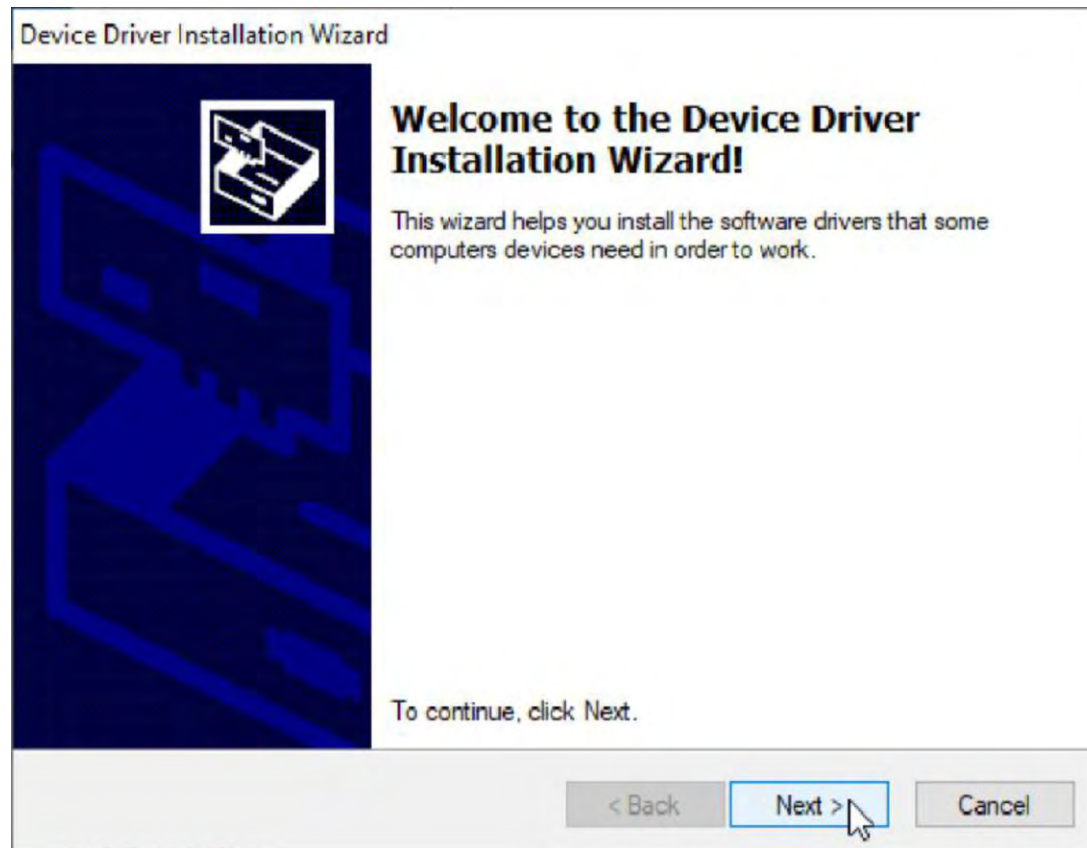
7



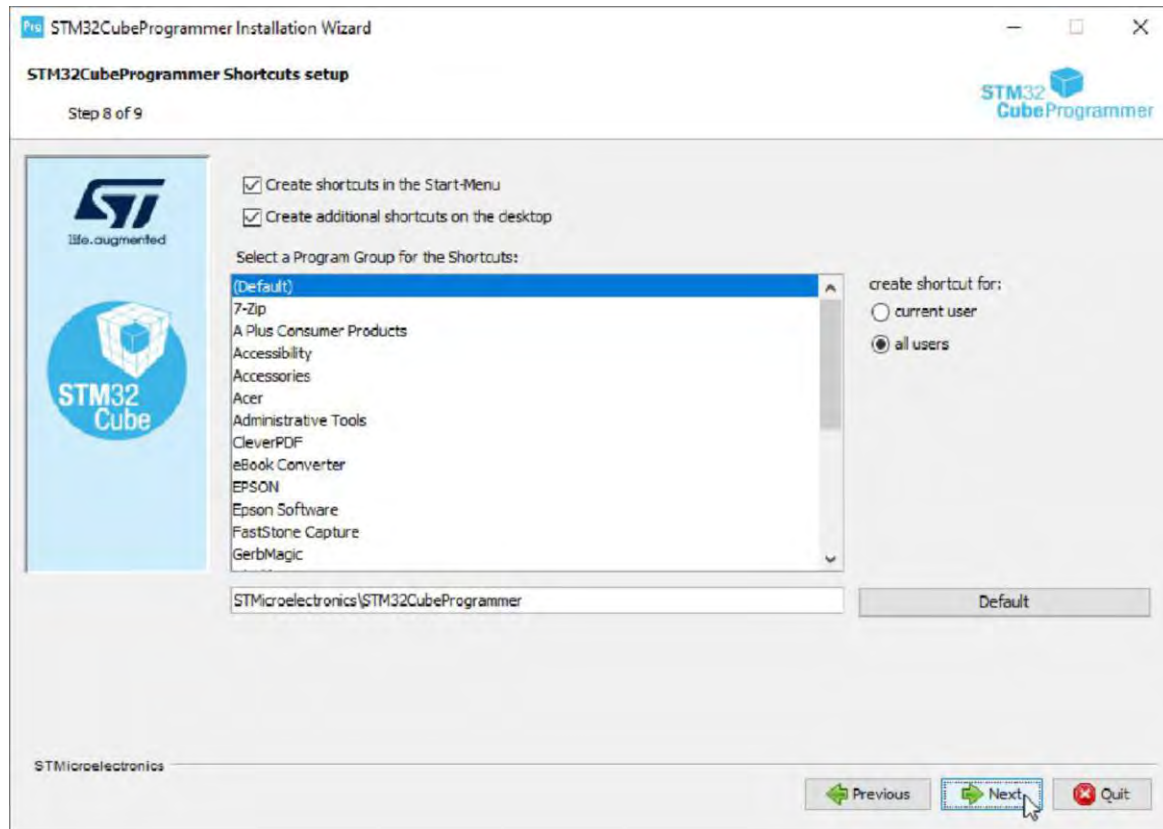
6



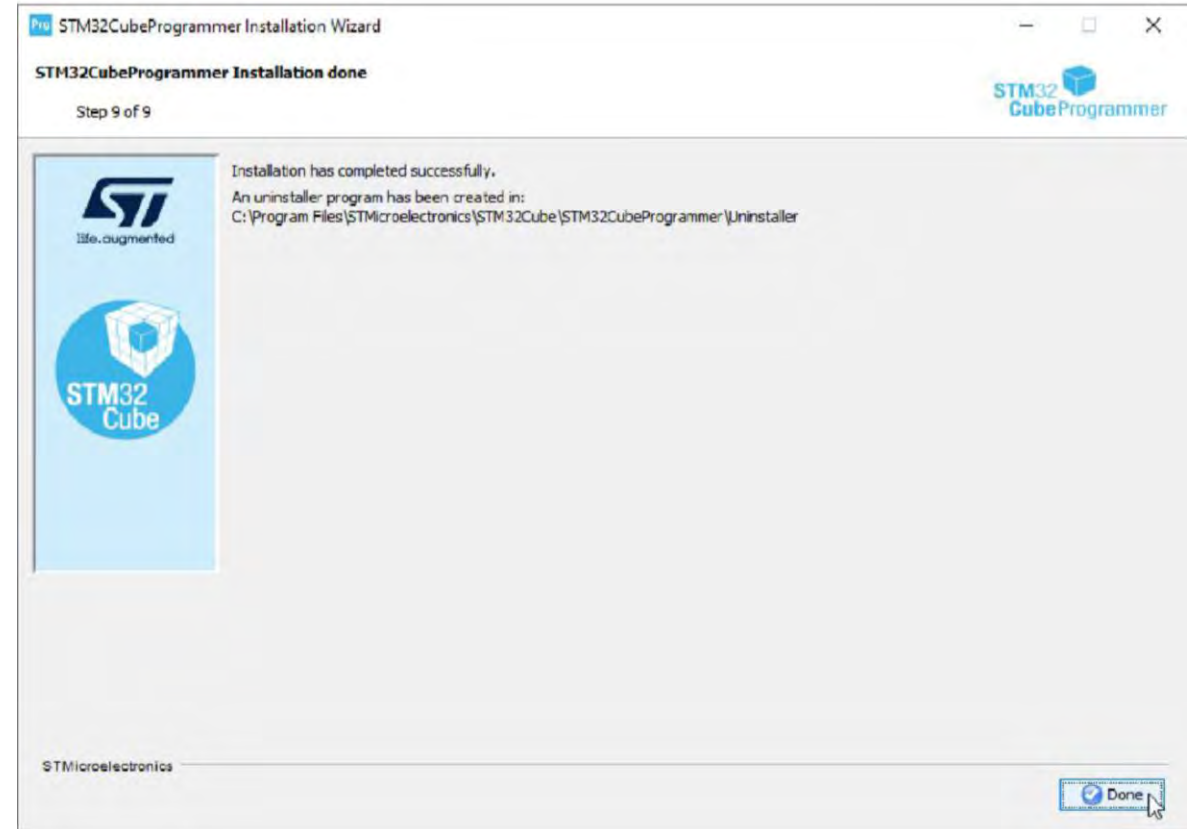
ติดตั้ง Driver



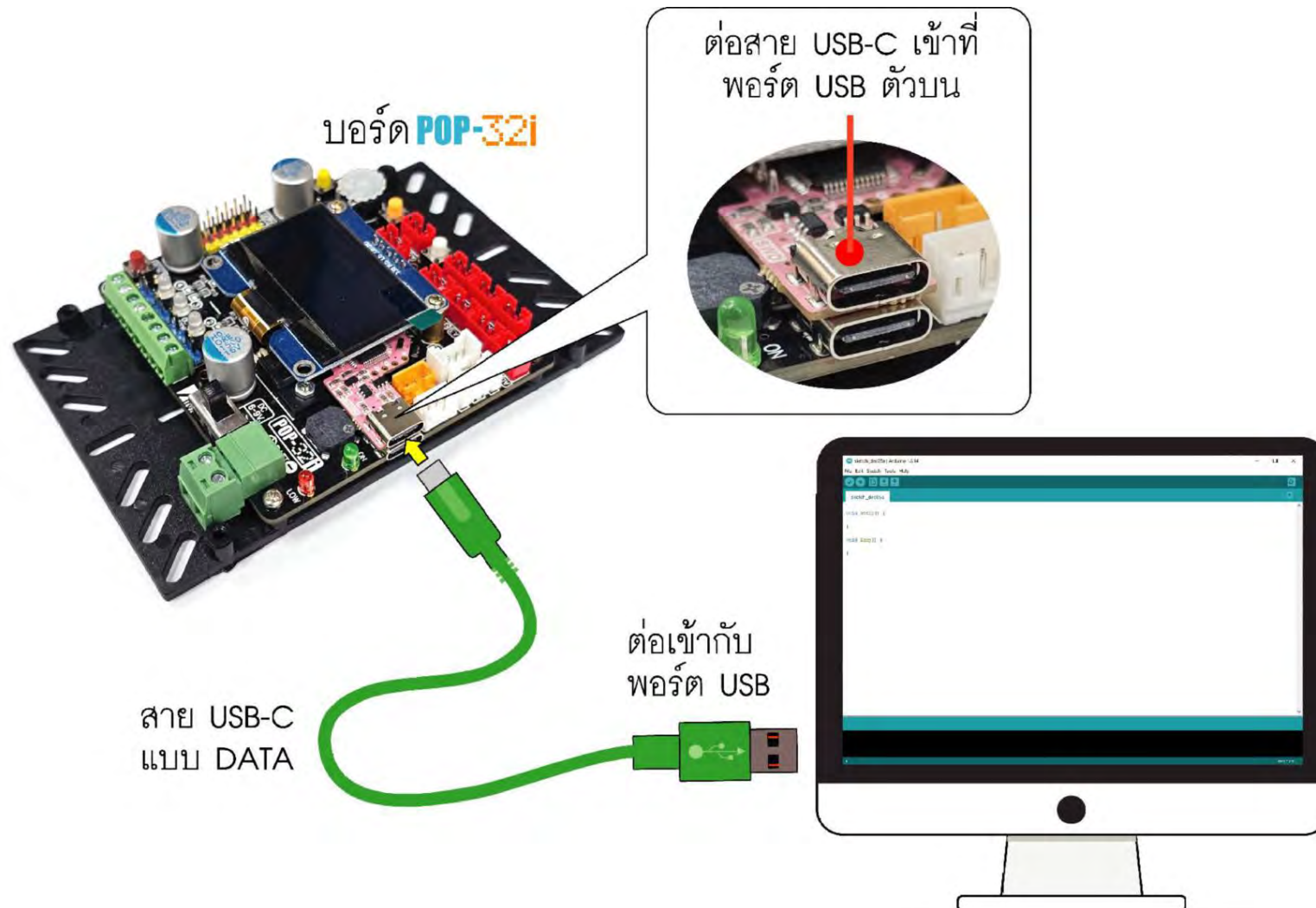
8



9



ทดสอบอัปโหลดโปรแกรมไปยัง POP-32i



แบตเตอรี่ Li-Po



แดง บวก
ดำ ลบ



แดง บวก
ดำ ลบ

- 2 เซล 7.4V
- กระแส 1100 mA
- จ่ายกระแส 30 เท่า
- ชาร์จ 5 เท่า

รายละเอียดแบตเตอรี่ ลิเทียม-โพลิเมอร์



1 เซล 3.7V อนุกรมกัน 2 เซล = 7.4V

จ่ายไฟ 1100 mA ต่อเนื่องได้ประมาณ 1 ชั่วโมง

จ่ายกระแสชั่วขณะได้ $1100 \times 30 = 33000\text{mA}$ O_o!

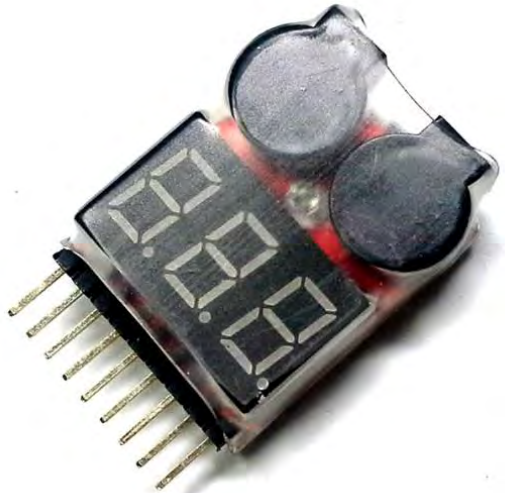
ชาร์จได้ 5 เท่า $1100 \times 5 = 5500\text{mA}$ ใช้เวลาประมาณ 20 นาที

ตรวจสอบแรงดันต่ำ

แสดงไฟและเตือน



วัด Volt อย่างเดียว



วัดโวลต์และเตือน

เตือนแรงดันต่ำ



กะพริบเมื่อไหร่เปลี่ยนแบตเตอรี่ที่

เครื่องชาร์จแบตเตอรี่



ชาร์จถ่าน Ni-MH

ชาร์จแบตเตอรี่ Li-Po

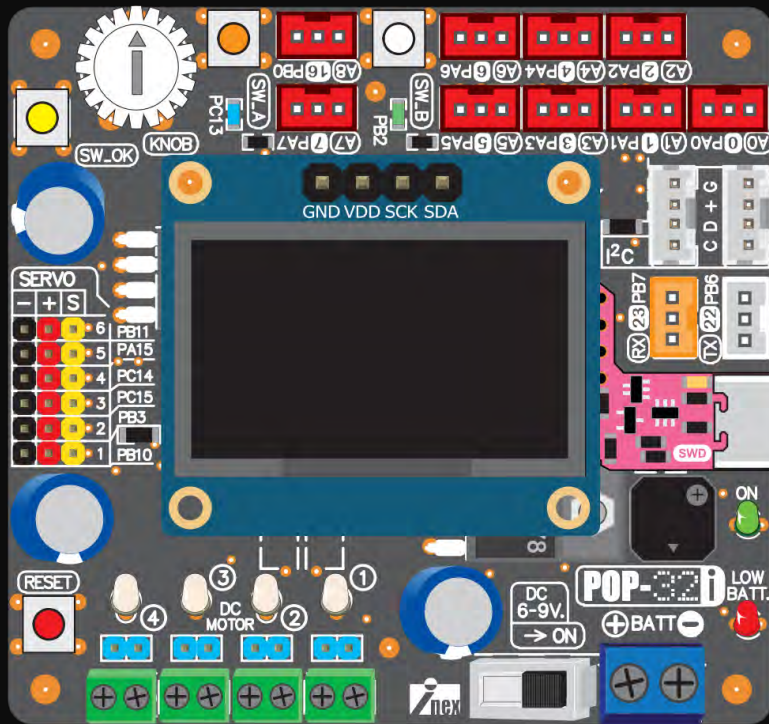
ชาร์จแบตเตอรี่รถยนต์

เครื่องชาร์จแบตเตอรี่แบบ USB

ชาร์จแบตเตอรี่ Li-Po แบบ 2 เซล



ขั้นตอนเชื่อมต่อกับคอมพิวเตอร์



1

ต่อสาย USB เข้ากับคอมพิวเตอร์

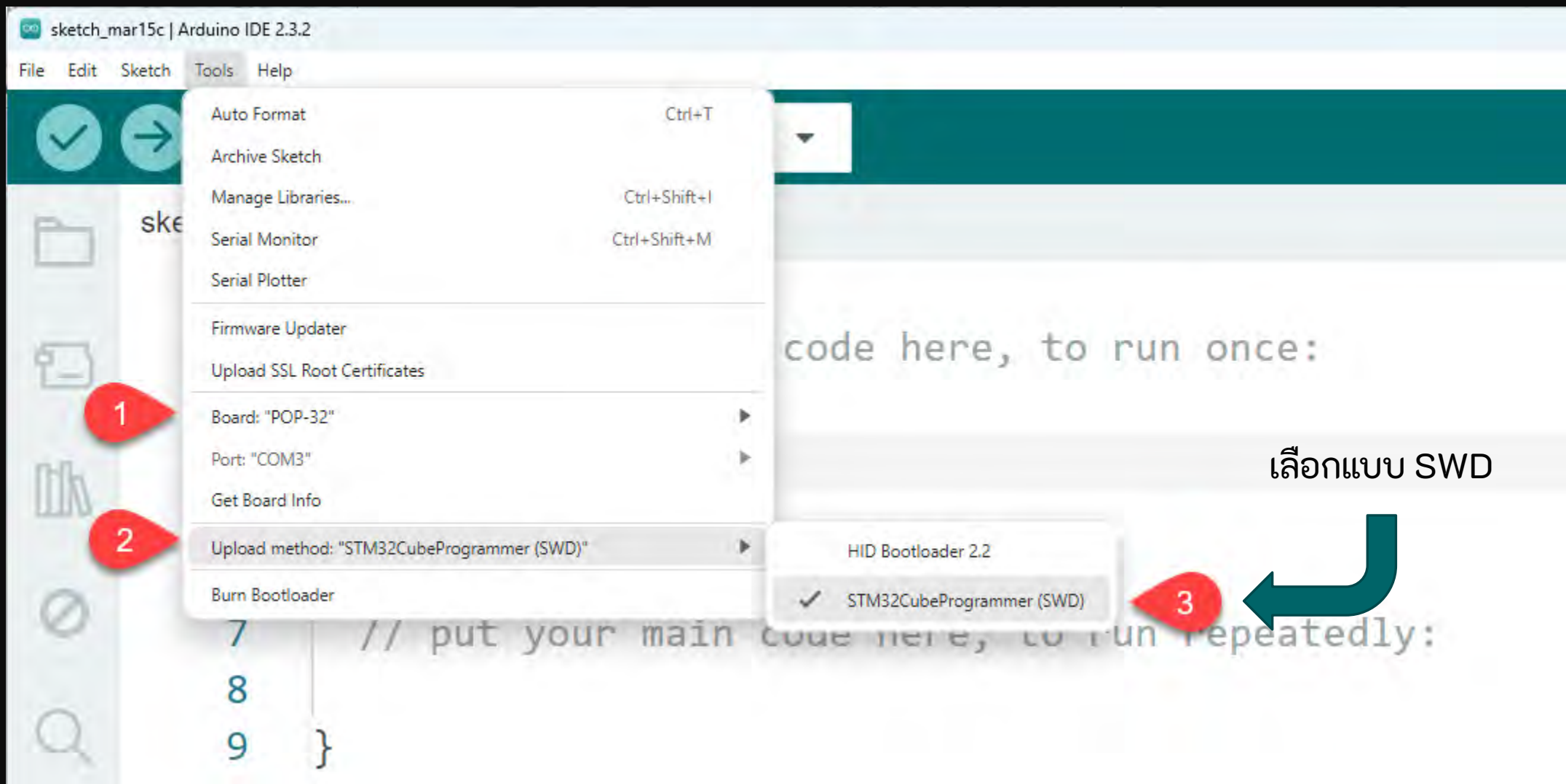
2

ต่อสาย USB เข้ากับ POP-32 ด้านบนสุด

3

เปิดสวิตช์จ่ายไฟ POP-32

เลือกบอร์ดเป็น POP-32



รูปแบบการทำงานของโปรแกรม Arduino

```
#include <POP32.h>
```

ผนวกรวมชุดคำสั่งสำหรับ POP-32

```
void setup() {
```

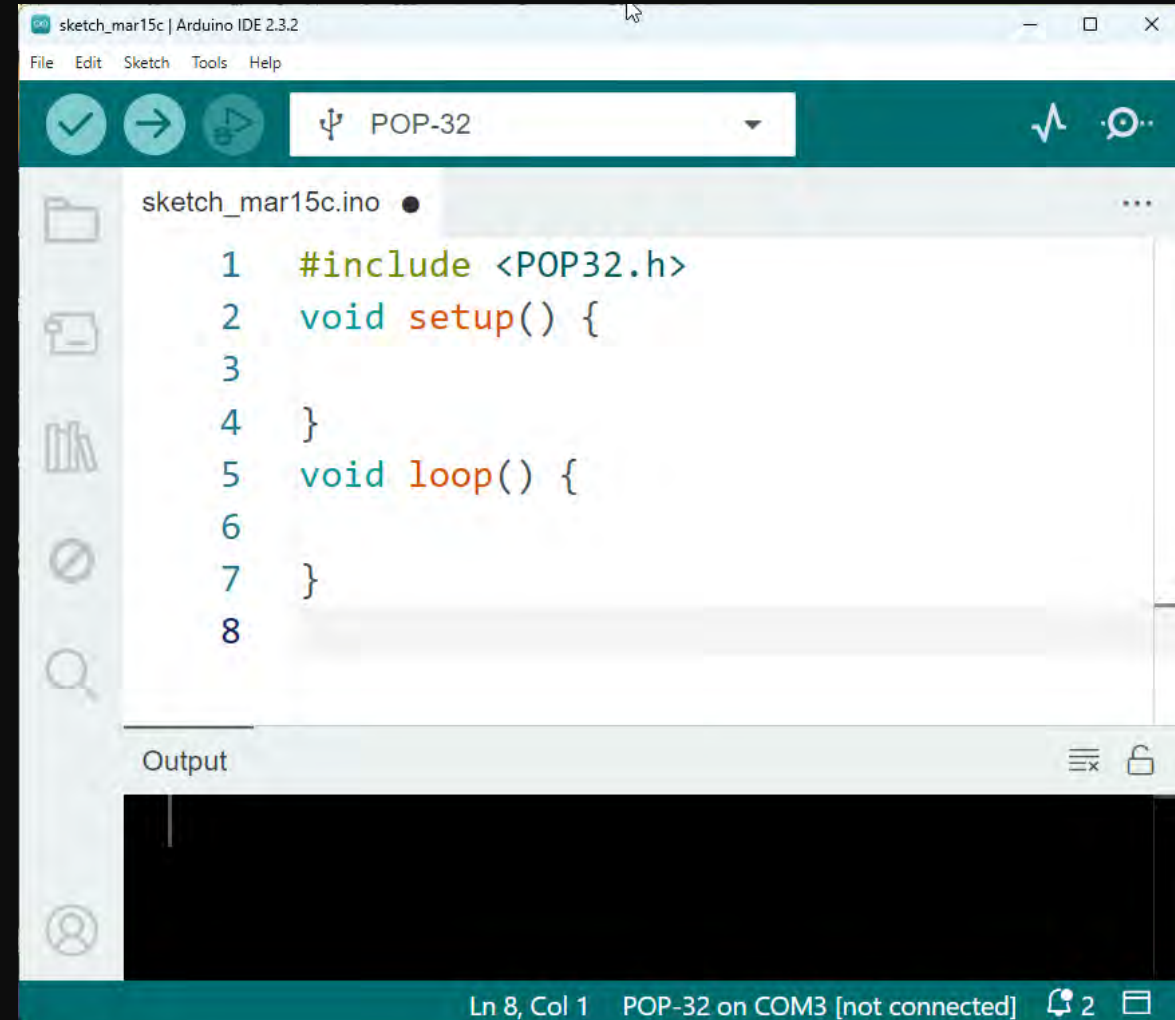
สำหรับกำหนดค่า เกิดขึ้นครั้งเดียว

```
}
```

```
void loop() {
```

โปรแกรมหลักทำงานต่อเนื่อง

```
}
```



ตัวอย่างลำดับการประมวลผลของฟังก์ชัน setup และ loop

```
#include <POP32.h>
```

```
void setup()
```

```
{
```

```
    command1;
```

```
    command2;
```

```
    command3;
```

```
}
```

```
void loop()
```

```
{
```

```
    command4;
```

```
    command5;
```

```
    command6;
```

```
}
```

การประมวลผลโปรแกรมจะทำงานดังนี้

```
command1;
```

```
command2;
```

```
command3;
```

```
command4;
```

```
command5;
```

```
command6;
```

```
command4;
```

```
command5;
```

```
command6;
```

```
command4;
```

```
command5;
```

```
command6;
```

```
...
```

```
...
```

```
...
```

รอบเดียวสำหรับ setup

รอบที่ 1 สำหรับ loop

รอบที่ 2 สำหรับ loop

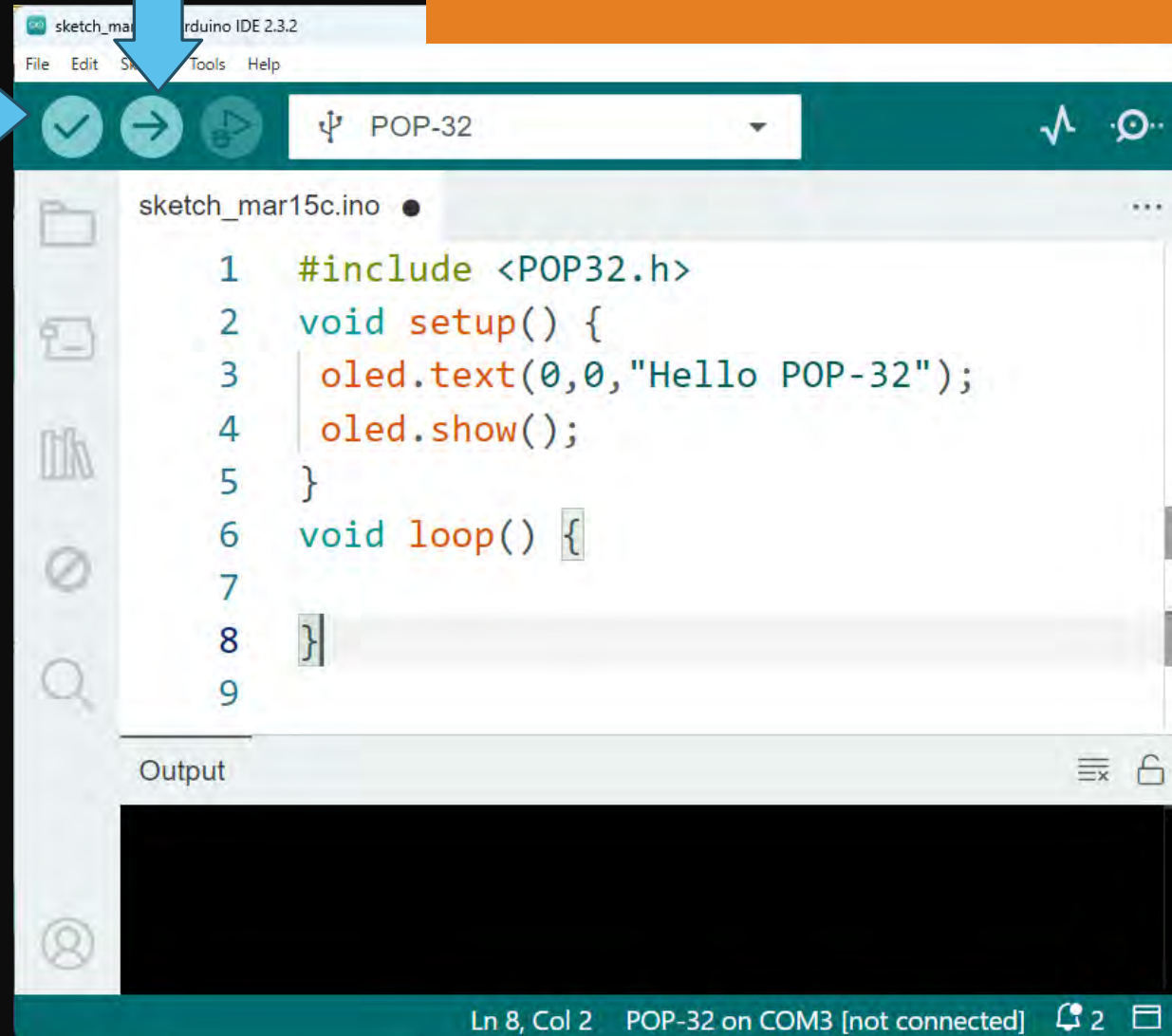
รอบที่ 3 สำหรับ loop

รอบที่ n สำหรับ loop

เขียนโปรแกรมผ่าน Arduino

คอมไพล์ คีย์ลัด Ctrl+R

ดาวน์โหลด คีย์ลัด Ctrl+U



แจ้งผลการคอมไพล์และขนาดหน่วยความจำที่ใช้

อัปโหลดโค้ดไปยัง POP-32
คีย์ลัด Ctrl+U

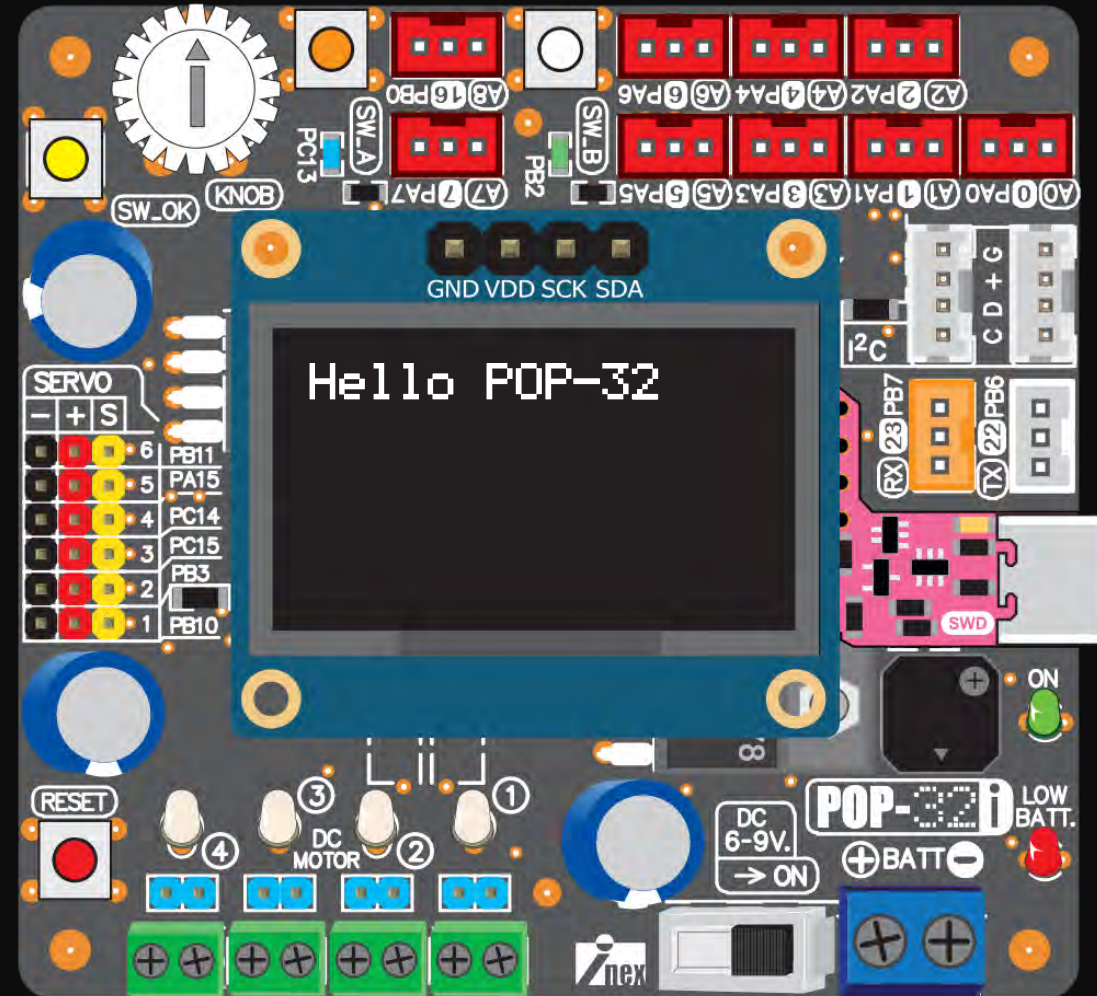


แจ้งผลว่าสมบูรณ์

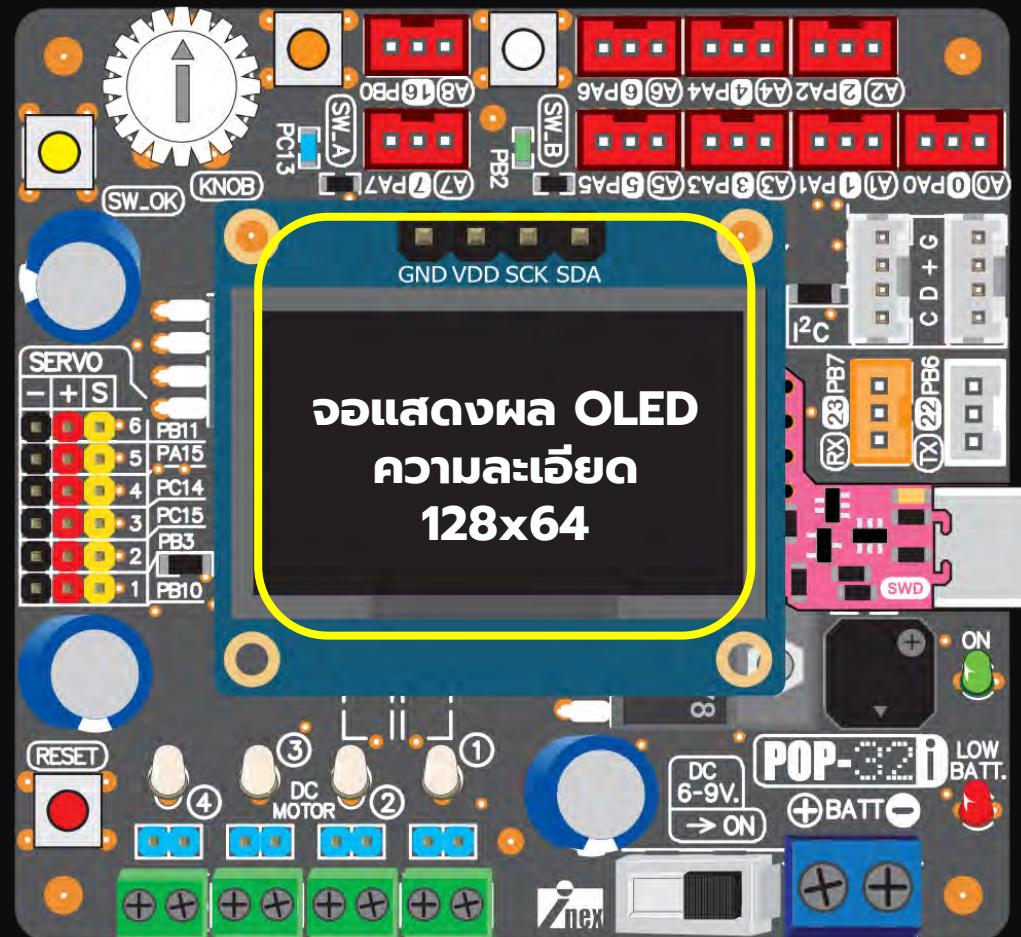
```
Output
```

```
RUNNING Program ...  
| Address:      : 0x8000000  
Application is running, Please Hold on...  
Start operation achieved successfully
```

Ln 6, Col 14 POP-32 on COM3 [not connected] 4



กลุ่มคำสั่งแสดงผลที่หน้าจอ OLED



ชุดคำสั่งสำหรับควบคุมการแสดงผล

คลาส OLED_I2C_SSD1309 เป็นคลาสที่รวบรวมเมธอดต่างๆ สำหรับควบคุมการแสดงผลกราฟิกแบบ OLED ความละเอียด 128x64 พิกเซล อย่างไรก็ดีเพื่อความสะดวกในการควบคุมการแสดงผลกราฟิกภายในคลาส OLED_I2C_SSD1309 ได้ทำการประกาศสร้างออบเจกต์ที่ทำการสืบทอดมาจากคลาส OLED_I2C_SSD1309 โดยตั้งชื่อเป็น **oled** เป็นที่เรียบร้อยแล้ว

คำสั่งสำหรับควบคุมการแสดงผลข้อความ

oled.text แสดงข้อความที่จอแสดงผล OLED ได้ 21 ตัว 8 บรรทัด (textSize=1)

รูปแบบ

```
oled.text(x,y,*p,...);
```

พารามิเตอร์

x คือตำแหน่งบรรทัดมีค่าตั้งแต่ 0-7

y คือตำแหน่งตัวอักษรมีค่าตั้งแต่ 0-20

***p** คือข้อความที่ต้องการนำมาแสดง

ค่าพิเศษ

%d แสดงตัวเลขจำนวนเต็มในช่วง -2,147,483,648 ถึง 2,147,483,647

%h แสดงตัวเลขฐานสิบหก

%b แสดงตัวเลขฐานสอง

%f แสดงผลตัวเลขจำนวนจริง (แสดงทศนิยม 3 หลัก)

คำสั่งสำหรับควบคุมการแสดงผลข้อความ

`oled.show`

ทำหน้าที่อัปเดตการแสดงผล

รูปแบบ

`oled.show()`

พารามิเตอร์ ไม่มี

การคืนค่า ไม่มี

รูปแบบ คำสั่ง delay

delay หน่วงเวลาในหน่วยมิลลิวินาที
รูปแบบ

delay(time)

พารามิเตอร์

time คือ เวลาที่ต้องการหน่วงในหน่วยมิลลิวินาที

ตัวอย่างเช่น

delay(500); // หน่วงเวลา 500 มิลลิวินาที(0.5 วินาที)

delay(2000); // หน่วงเวลา 2000 มิลลิวินาที(2 วินาที)

ตัวแปร (ที่ใช้งานบ่อย)

byte	0-255	(unsigned char)
bool	0-1	true false
int	-2147483648 ถึง 2147483647	
char	-128 ถึง 127	
float	-3.4×10^{38} ถึง 3.4×10^{38}	

หาข้อมูลเพิ่มเติมจาก [reference](#)

ชุดคำสั่งอื่นๆ สำหรับใช้งานอ็อบเจกต์ oled

แสดงตัวอักษร

text
textSize
mode
textColor
textBackgroundColor
clear
fillScreen

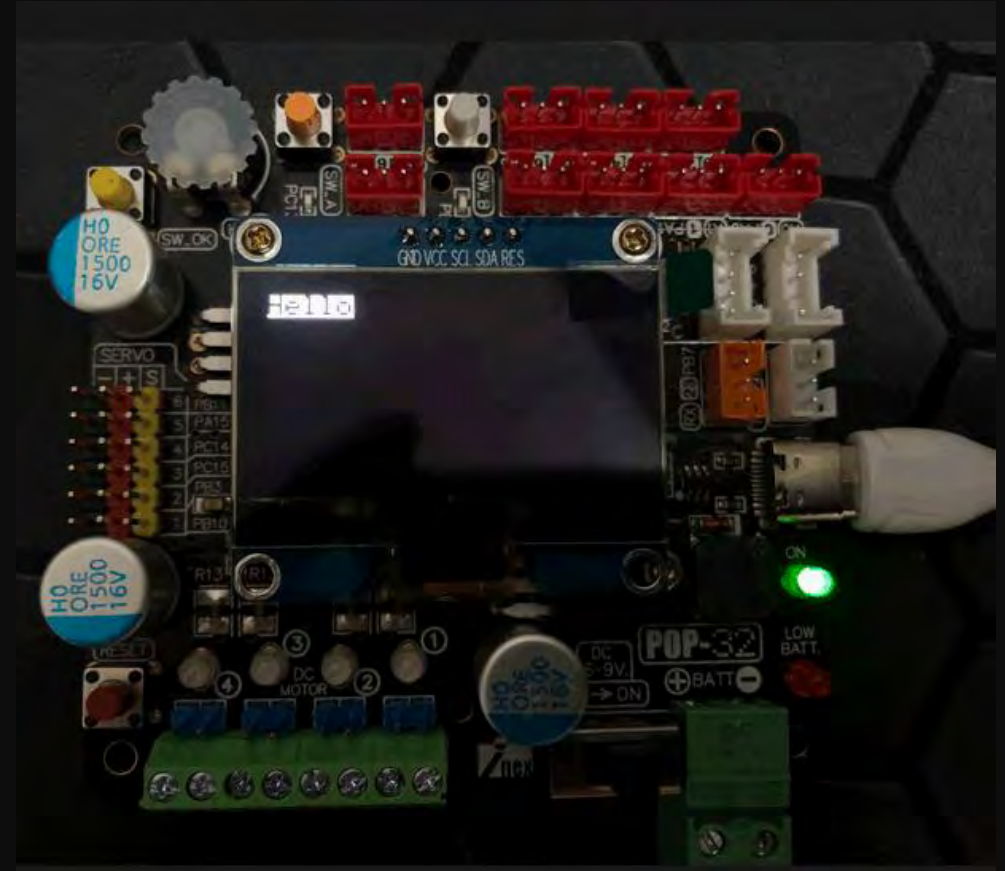
แสดงกราฟิก

drawPixel
drawRect
fillRect
drawLine
drawCircle
fillCircle

ตัวอย่างกำหนดสีตัวอักษร

```
#include <POP32.h>
void setup(){
  oled.setTextColor(BLACK,WHITE);
  oled.text(0,0,"Hello world");
  oled.show();
}
void loop(){}

```



ตัวอักษรสีดำพื้นหลังตัวอักษรเป็นสีขาวเท่านั้น

ตัวอย่างกำหนดขนาดตัวอักษร

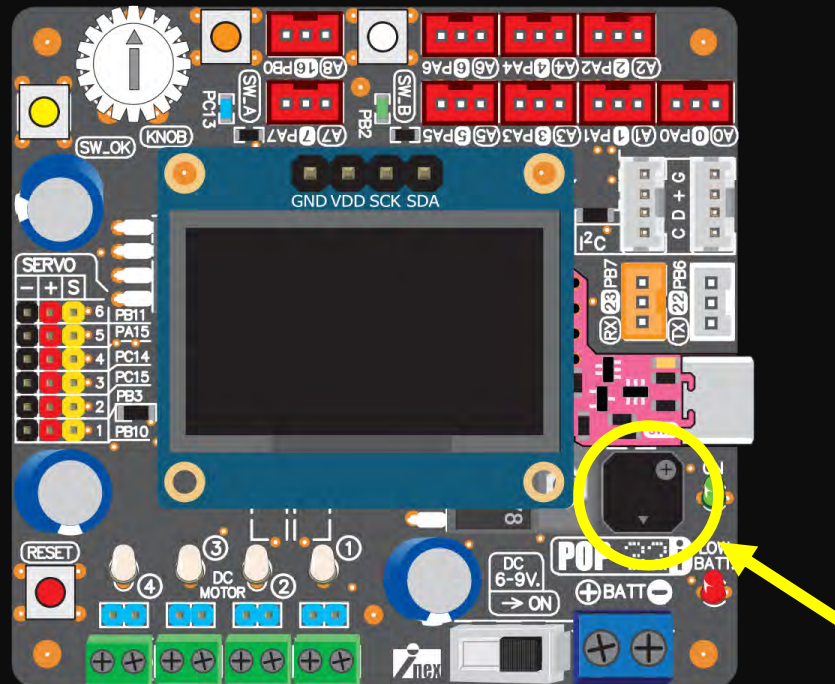
```
#include <POP32.h>
void setup(){
  oled.setTextSize(2);
  oled.setTextColor(BLACK,WHITE);
  oled.text(0,0,"Hello world");
  oled.show();
}
void loop(){}

```



ตัวอักษรสีดำพื้นหลังตัวอักษรเป็นสีขาวมีขนาดเป็น 2 เท่า

กลุ่มคำสั่งสร้างสัญญาณเสียงออกลำโพงเปียโซ



การสร้างเสียงออกลำโพง

beep()

sound(freq,time)

freq คือ ความถี่เสียง

time คือ เวลา 1/100 วินาที

มีค่าความถี่ย่าน 300 Hz ถึง 3000 Hz

ตัวอย่างโปรแกรมสร้างเสียงออกลำโพง

ตัวอย่างการใช้งานคำสั่ง beep

```
#include <POP32.h>
void setup(){}
void loop(){
    beep();
    delay(500);
}
```

สร้างเสียง 500 Hz นาน 0.1 วินาที ต่อเนื่อง

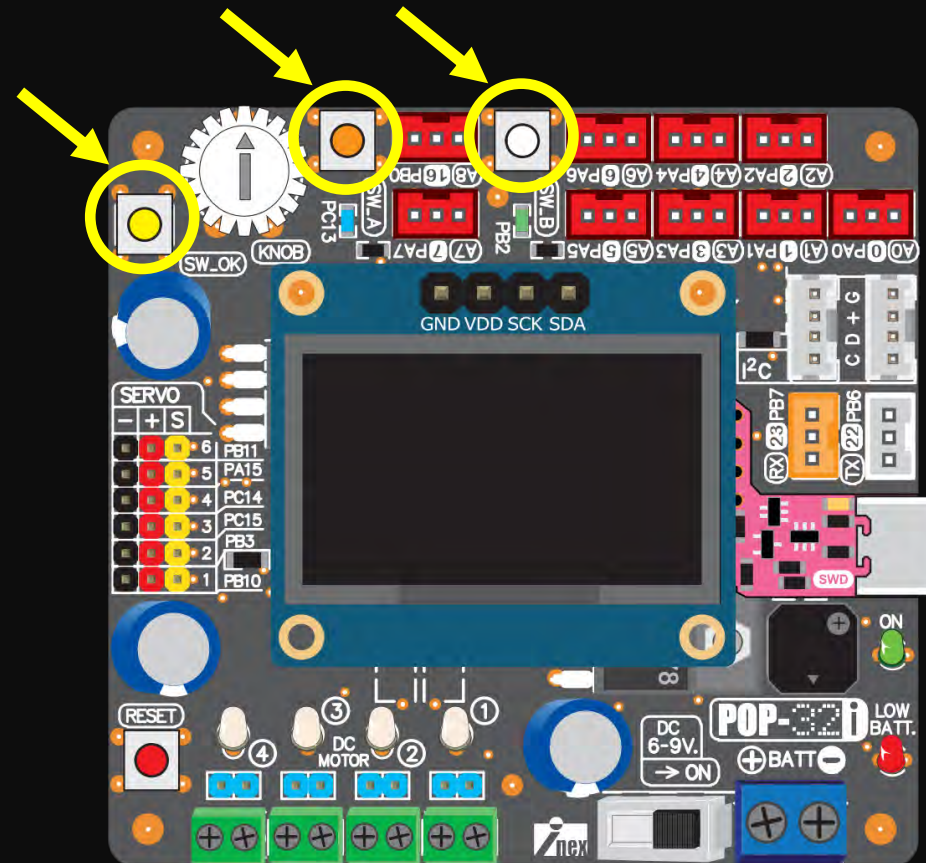
ตัวอย่างการใช้งานคำสั่ง sound

```
#include <POP32.h>
void setup(){}
void loop(){
    sound(1000,100);
    delay(500);
}
```

สร้างเสียง 1 kHz นาน 0.1 วินาที ต่อเนื่อง

กลุ่มคำสั่งอ่านค่าสถานะสวิตช์

A,B และ OK



คำสั่งอ่านค่าสถานะสวิตช์

ทำหน้าที่อ่านค่าสถานะการกดสวิตช์ A, B และ OK

รูปแบบ

SW_A()

SW_B()

SW_OK()

การคืนค่า

- คืนค่าเป็น true(1) ในกรณีที่สวิตช์ถูกกด
- คืนค่าเป็น false(0) ในกรณีที่สวิตช์ไม่ถูกกด

คำสั่งรอกการกดสวิตช์

ทำหน้าที่รอกจนกระทั่งสวิตช์ที่กำหนด

(LED ประจำตัวสวิตช์ จะกระพริบต่อเนื่องเพื่อแจ้งผู้ใช้งานในระหว่างรอกการกดสวิตช์)

รูปแบบ

waitSW_A()

waitSW_B()

waitSW_OK()

waitAnykey()

การคืนค่า

ไม่มีการคืนค่า

คำสั่งตรวจสอบเงื่อนไข if-else

```
if (เงื่อนไข) {  
    คำสั่ง1 ทำเมื่อเงื่อนไขเป็นจริง;  
    ...  
}  
else {  
    คำสั่ง2 ทำเมื่อเงื่อนไขเป็นเท็จ;  
    ...  
}
```

ตัวแปร Boolean

จริง : True : 1

เท็จ : False : 0

ตัวอย่างตรวจจบการกดสวิตช์ A

```
#include <POP32.h>
void setup(){}
void loop()
{
  if(SW_A())
  {
    beep();
    delay(200);
  }else{
    sound(2000,200);
    delay(200);
  }
}
```

หลังจากกดสวิตช์ A หรือ ไม่กดสวิตช์เสียงจะถูกกำเนิดความถี่ที่ต่างกัน

ตัวอย่างตรวจจบการกดสวิตช์ A และ B

```
#include <POP32.h>
void setup(){
}
void loop()
{
    if(SW_A())
    {
        beep();
        delay(200);
    }
    if(SW_B())
    {
        sound(2000,200);
        delay(200);
    }
}
```

หลังจากกดสวิตช์ A และ B เสียงจะถูกกำเนิดความถี่ที่ต่างกัน

ตัวอย่างการใช้ && (AND)

```
#include <POP32.h>
void setup(){
}
void loop()
{
    if(SW_A() && SW_B())
    {
        beep();
        delay(200);
    }else{
        sound(2000, 200);
        delay(200);
    }
}
```

กดสองปุ่มพร้อมกัน

ไม่กดสวิตช์

ตัวอย่างการใช้ || (OR)

```
#include <POP32.h>
void setup(){
}
void loop()
{
    if(SW_A() || SW_B())
    {
        beep();
        delay(200);
    }else{
        sound(2000, 200);
        delay(200);
    }
}
```

กดปุ่มใดปุ่มหนึ่ง

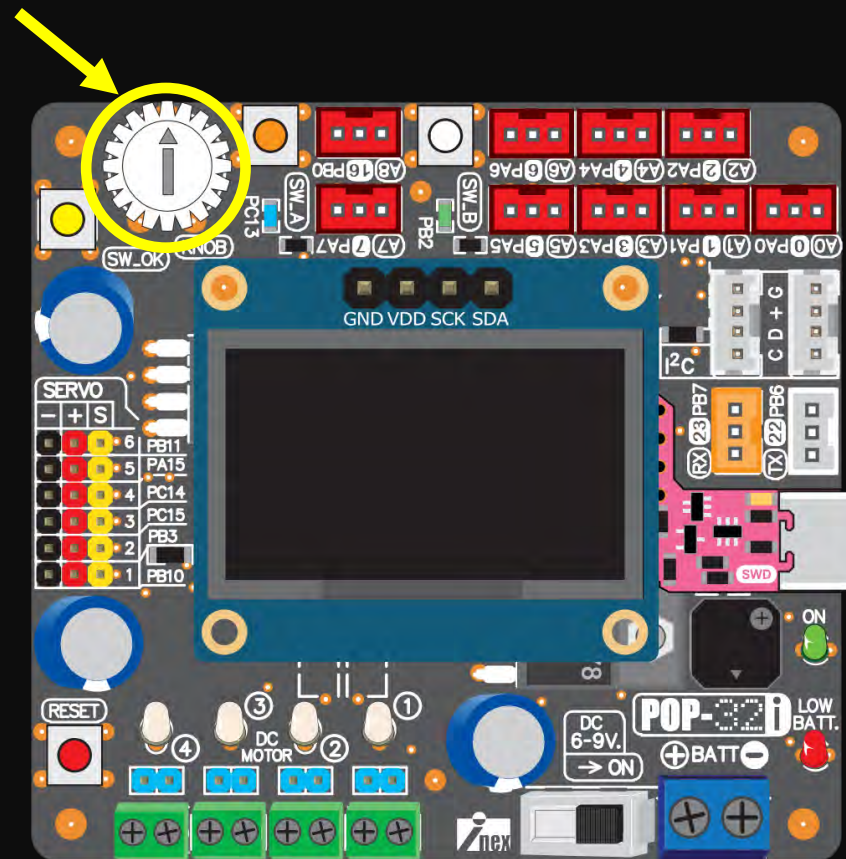
ไม่กดสวิตช์

ตัวอย่างรอกจนกระทั่งมีการกดสวิตช์ OK

```
#include <POP32.h>
void setup(){
    oled.text(0,0,"Press SW_OK");
    oled.show();
    waitSW_OK();
}
void loop(){
    beep();
    delay(1000);
}
```

หลังจากกดสวิตช์ OK เสียงจะถูกกำเนิดทุกๆ 1 วินาที

กลุ่มคำสั่งอ่านค่า KNOB



คำสั่ง knob()

ทำหน้าที่อ่านค่าอะนาลอกจาก KNOB
ในช่วง 0 ถึง 4000

รูปแบบ

knob()

การคืนค่า

คืนค่าตั้งแต่ 0 ถึง 4000 จากการปรับค่า

คำสั่ง knob(x)

ทำหน้าที่อ่านค่าอะนาลอกจาก KNOB
ในช่วง 0 ถึง x ตามที่กำหนด

รูปแบบ

knob(x)

การคืนค่า

คืนค่าตั้งแต่ 0 ถึง x จากการปรับค่า

คำสั่ง knob(x,y)

ทำหน้าที่อ่านค่าอะนาลอกจาก KNOB ในช่วง x ถึง y ตามที่กำหนด

รูปแบบ

knob(x,y)

การคืนค่า

คืนค่าตั้งแต่ x ถึง y จากการปรับค่า

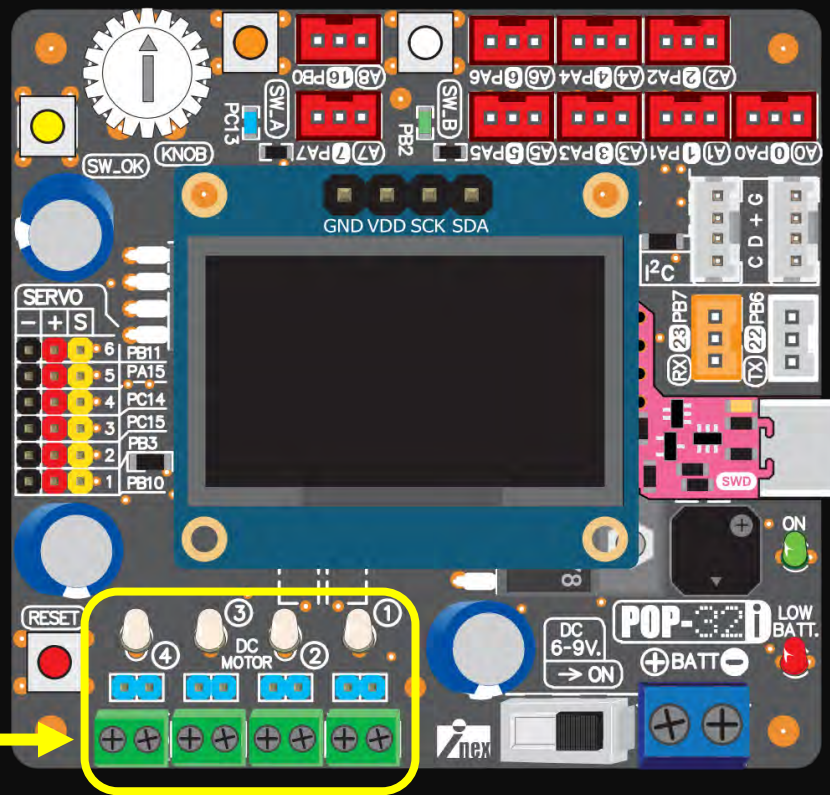
ตัวอย่างอ่านค่า knob

```
#include <POP32.h>
void setup() {
}
void loop() {
    oled.text(0,0,"knob=%d ",knob());
    oled.show();
}
```

ทดสอบปรับหมุนที่ knob แล้วลอง
เปลี่ยนการอ่านค่าทั้ง 3 รูปแบบ

คำสั่งขับเคลื่อนมอเตอร์ไฟตรง

จุดต่อมอเตอร์ 1-4



คำสั่ง motor(ch,pow)

ทำหน้าที่ขับเคลื่อนมอเตอร์ช่องที่กำหนด

รูปแบบ

motor(ch,pow)

โดยที่

ch คือ ช่องขับ 1 ถึง 4

pow คือ ค่ากำลังขับในช่วง -100 ถึง 100

-กรณีเป็นค่าบวก ขับทิศทางไปข้างหน้า

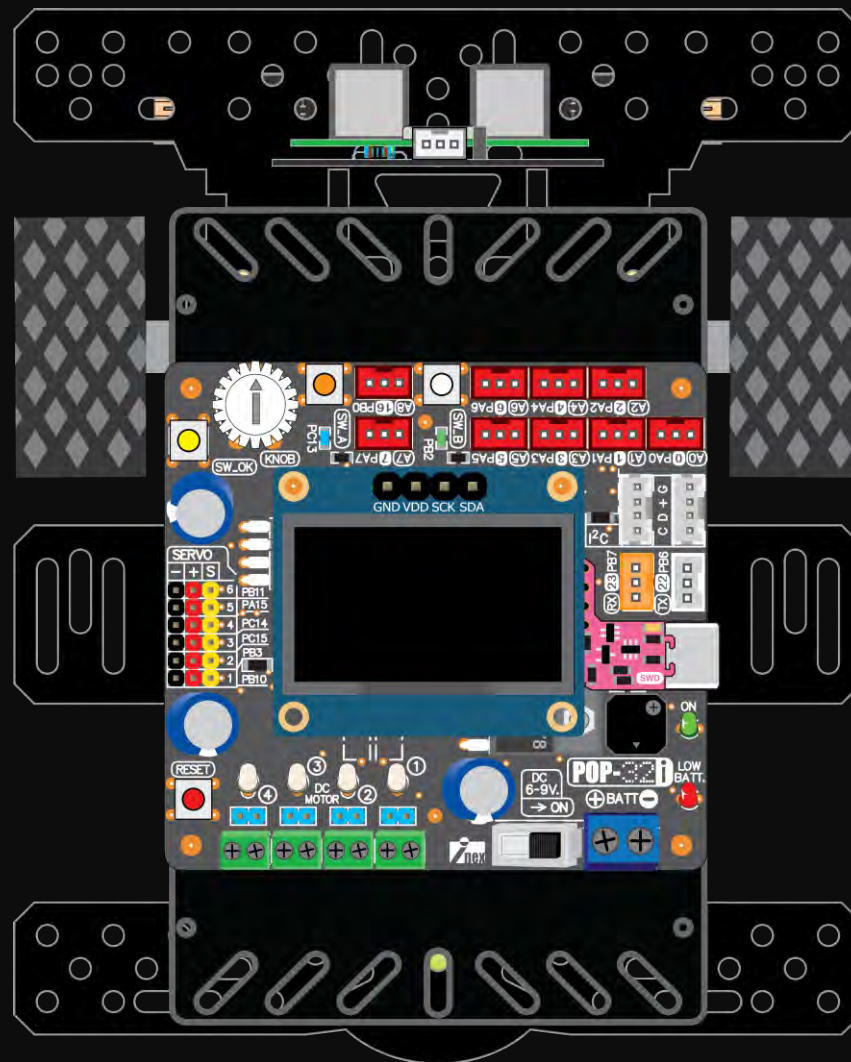
-กรณีเป็นค่าลบ ขับทิศทางถอยหลัง

การคืนค่า

ไม่มี

กลุ่มคำสั่งขับเคลื่อนหุ่นยนต์

ใช้มอเตอร์ช่อง 1 ต่อ
เข้ากับล้อด้านซ้าย



ใช้มอเตอร์ช่อง 2 ต่อ
เข้ากับล้อด้านขวา

สรุปคำสั่ง เพื่อขับเคลื่อนหุ่นยนต์

fd(Speed); เดินหน้า

bk(Speed); ถอยหลัง

sl(Speed); เลี้ยวซ้าย

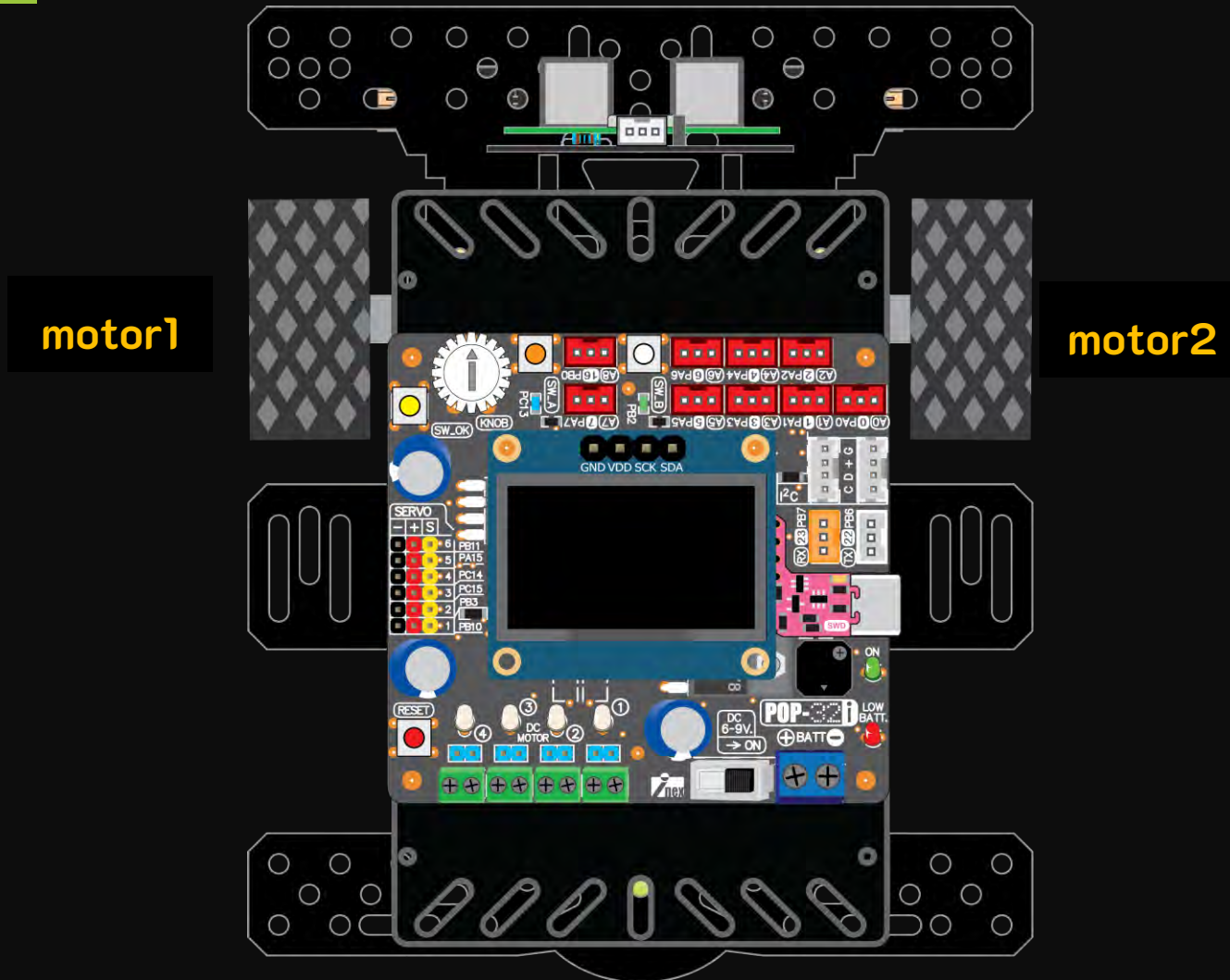
sr(Speed); เลี้ยวขวา

tl(Speed); เลี้ยวซ้ายล้อเดียว

tr(Speed); เลี้ยวขวาล้อเดียว

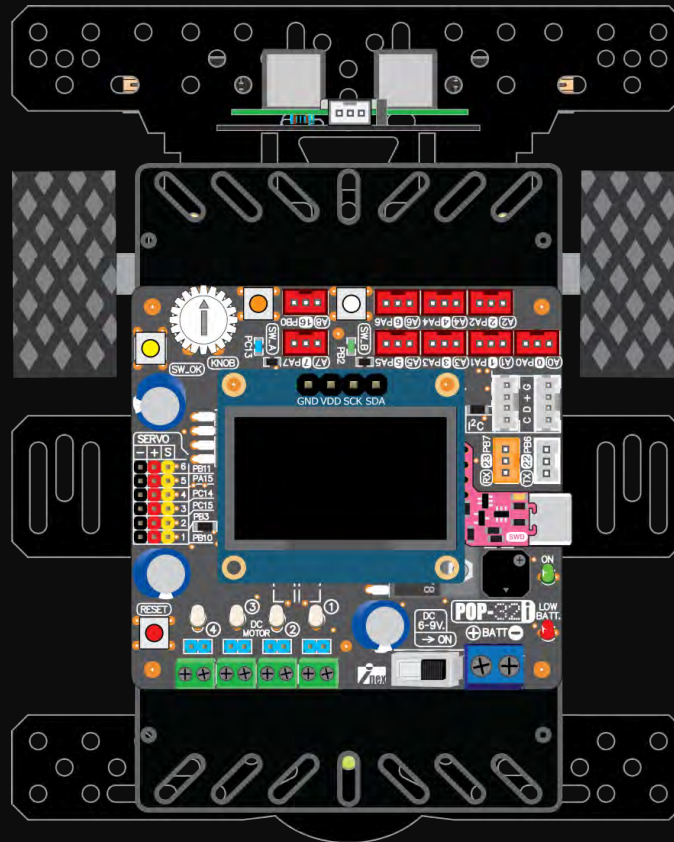
ao(); หยุด

Speed คือความเร็วหุ่นยนต์ 0-100



หุ่นยนต์เคลื่อนที่ไปข้างหน้า

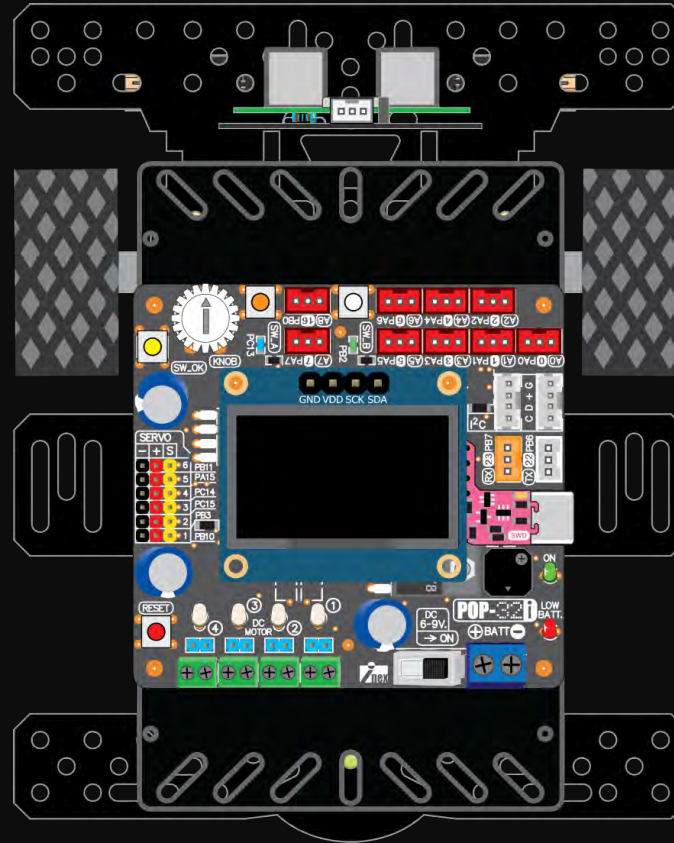
fd(50);



หุ่นยนต์เคลื่อนที่ถอยหลัง

bk(50);

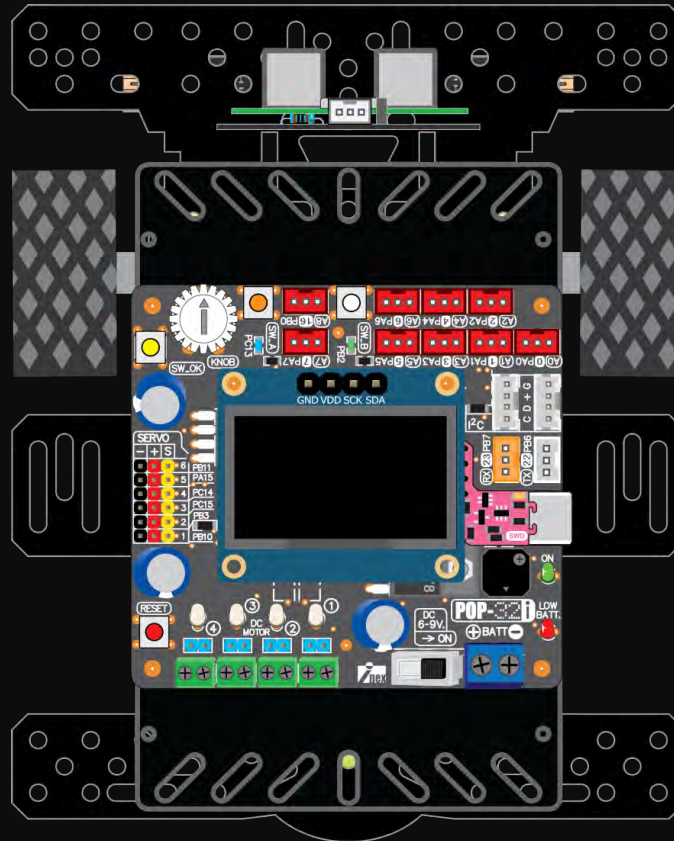
motor1



motor2

หุ่นยนต์หมุนตัวเลี้ยวซ้าย

sl(50);

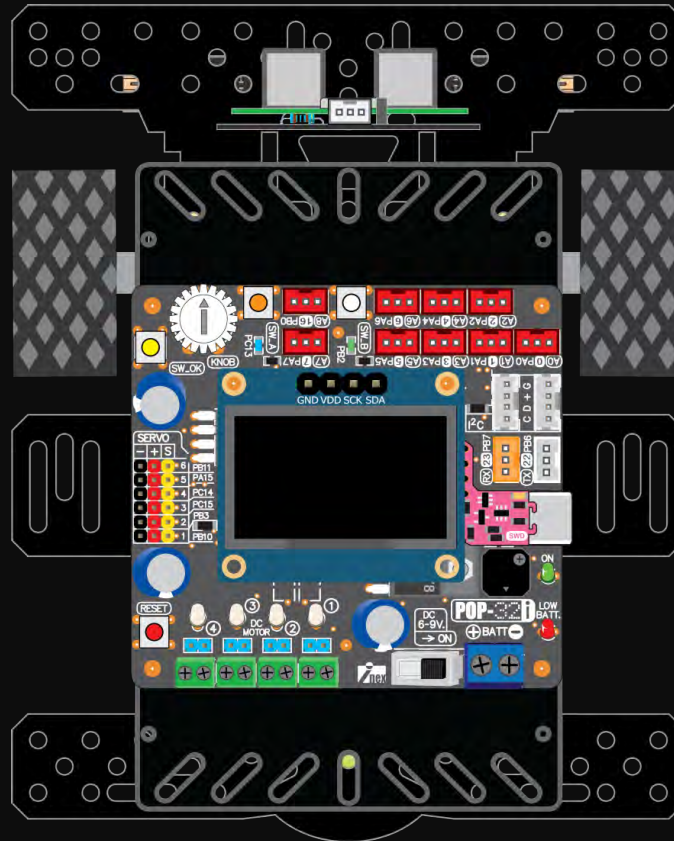


หุ่นยนต์เลี้ยวซ้ายล้อเดียว

tl(50);



motor1



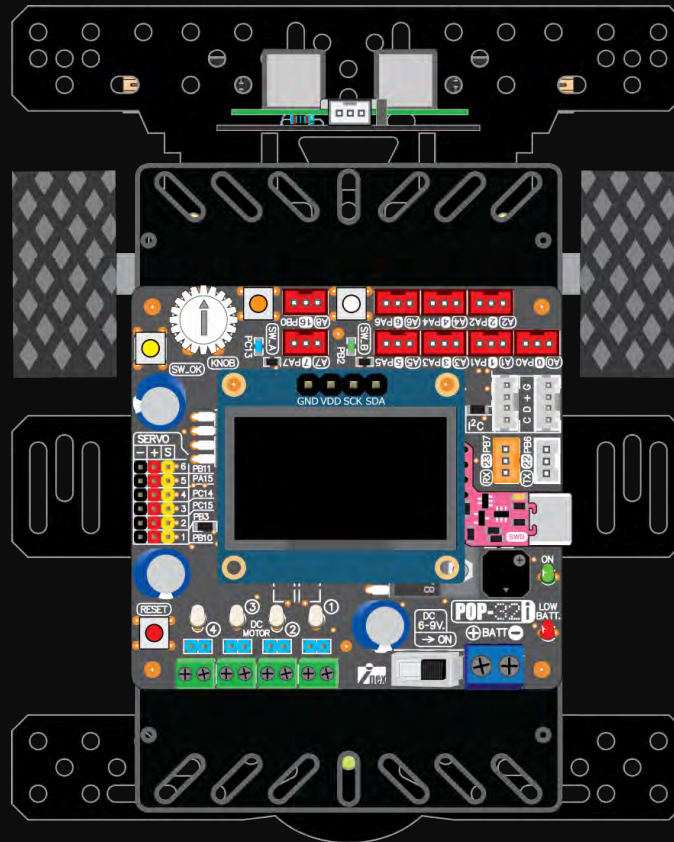
motor2

หุ่นยนต์หมุนตัวเลี้ยวขวา

sr(50);



motor1



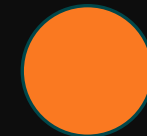
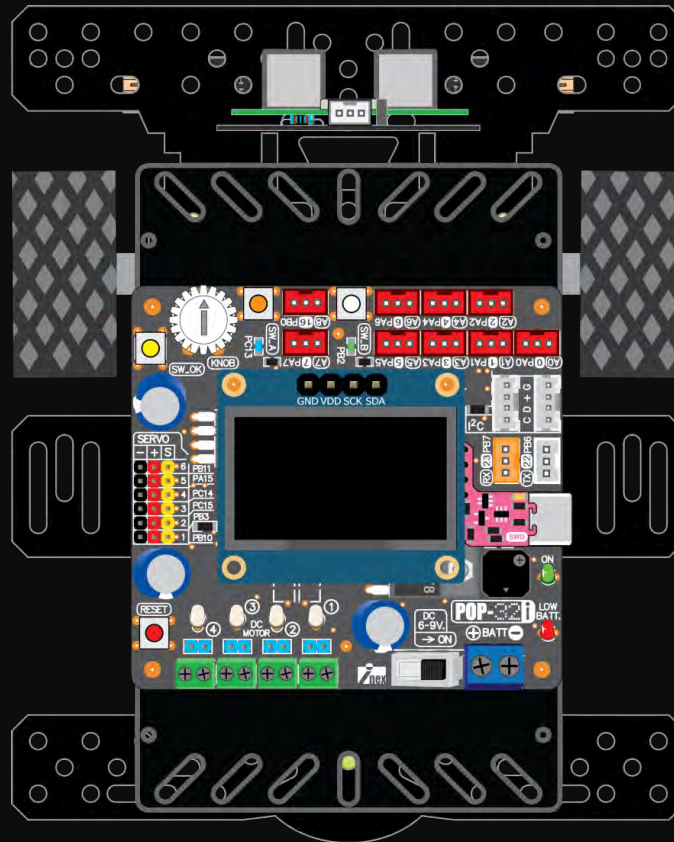
motor2

หุ่นยนต์เลี้ยวขวาอย่างเดียว

tr(50);



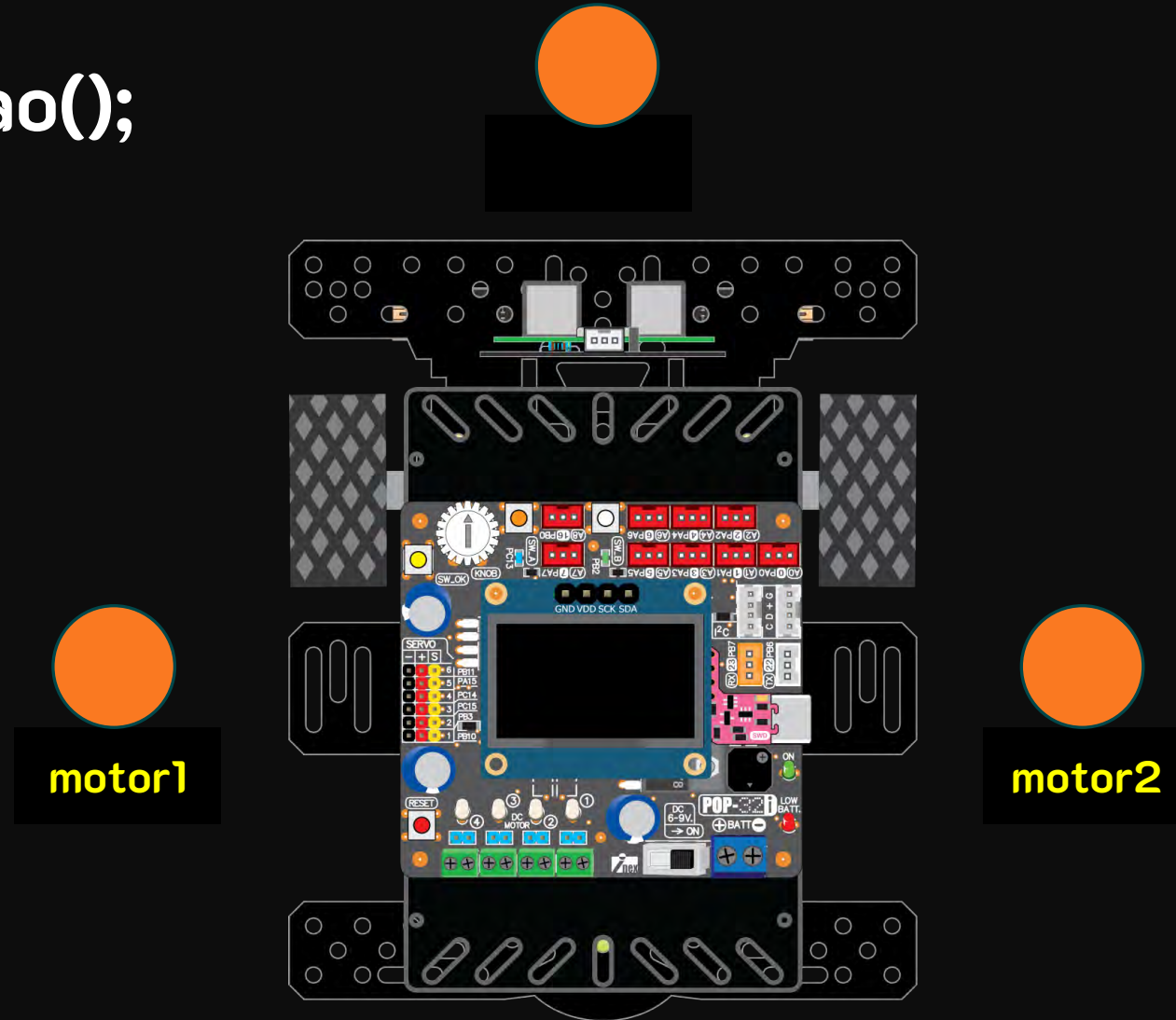
motor1



motor2

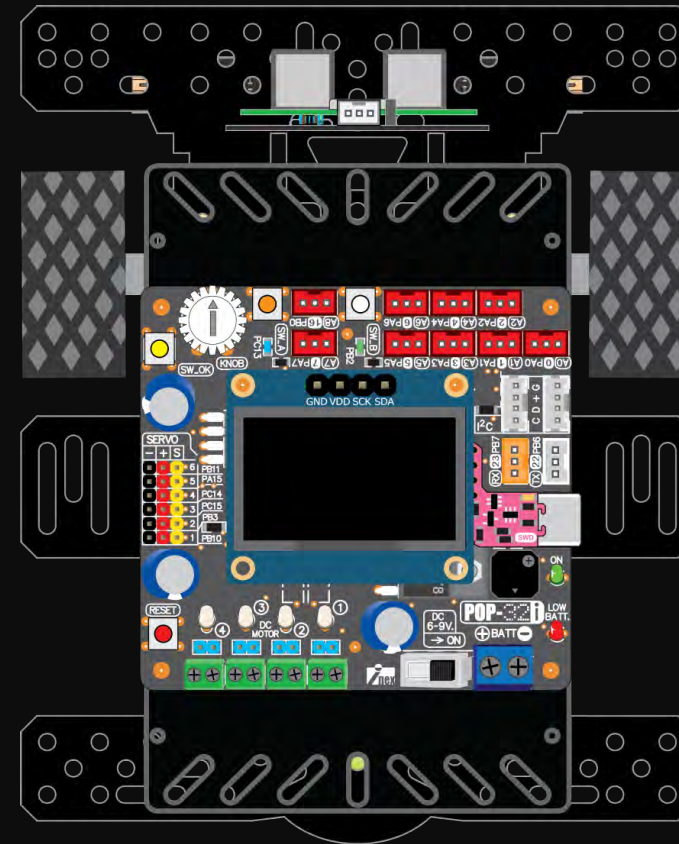
หุ่นยนต์หยุดการเคลื่อนที่

ao();



ทดสอบ เดินหน้า 1 วินาที หยุด 1 วินาที ถอยหลัง 1 วินาที

```
#include <POP32.h>
void setup()
{
    oled.text(0,0,"Press SW_OK");
    oled.show();
    waitSW_OK();
}
void loop()
{
    fd(30);
    delay(1000);
    ao();
    delay(1000);
    bk(30);
    delay(1000);
}
```



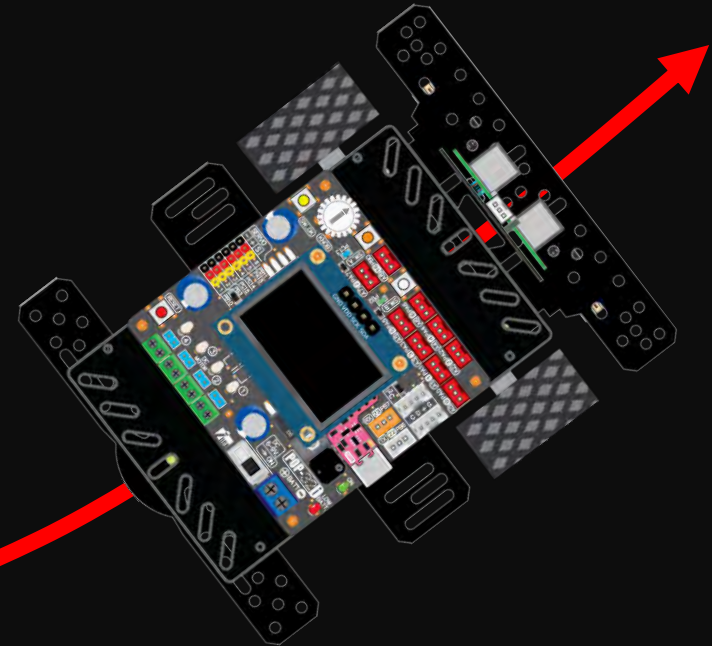
ความแม่นยำของการเคลื่อนที่ตรง

fd2 (Speed1 , speed2) ; เดินหน้า

bk2 (Speed1 , speed2) ; ถอยหลัง

Speed1 คือมอเตอร์ด้านซ้าย

Speed2 คือมอเตอร์ด้านขวา



มอเตอร์ 2 เร็วกว่ามอเตอร์ 1

ใช้งานเซนเซอร์ตรวจจับการชน (แผงวงจรสวิตช์ ZX-SWITCH-01)



- สวิตช์ถูกกดจะให้ค่าเป็น 0
- สวิตช์ไม่ถูกกดให้ค่าเป็น 1

คำสั่ง in(ch)

รูปแบบ

in(ch)

โดยที่

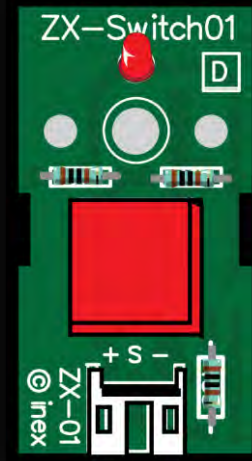
ch คือ หมายเลขพอร์ตดิจิตอลที่ต่อใช้งาน

การคืนค่า

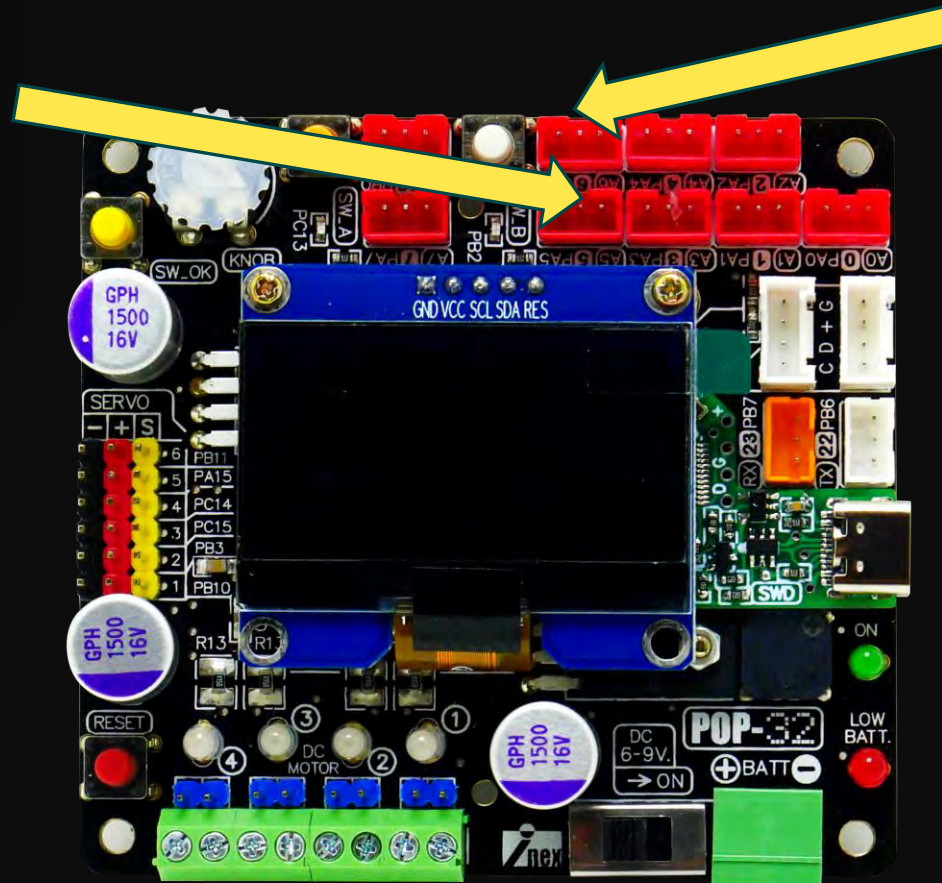
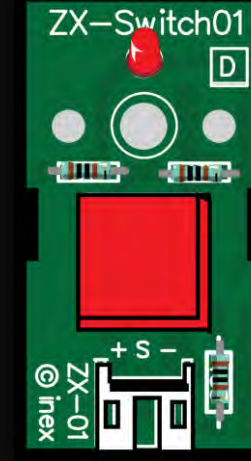
จะให้ค่าเป็น “0” (เท็จ) หรือ “1” (จริง)

ทดสอบใช้คำสั่ง `in()` กับ ZX-Switch01 โดยต่อใช้งาน 2 ตัว

Switch ตัวที่ 1 ต่อเข้ากับพอร์ต 5



Switch ตัวที่ 2 ต่อเข้ากับพอร์ต 6



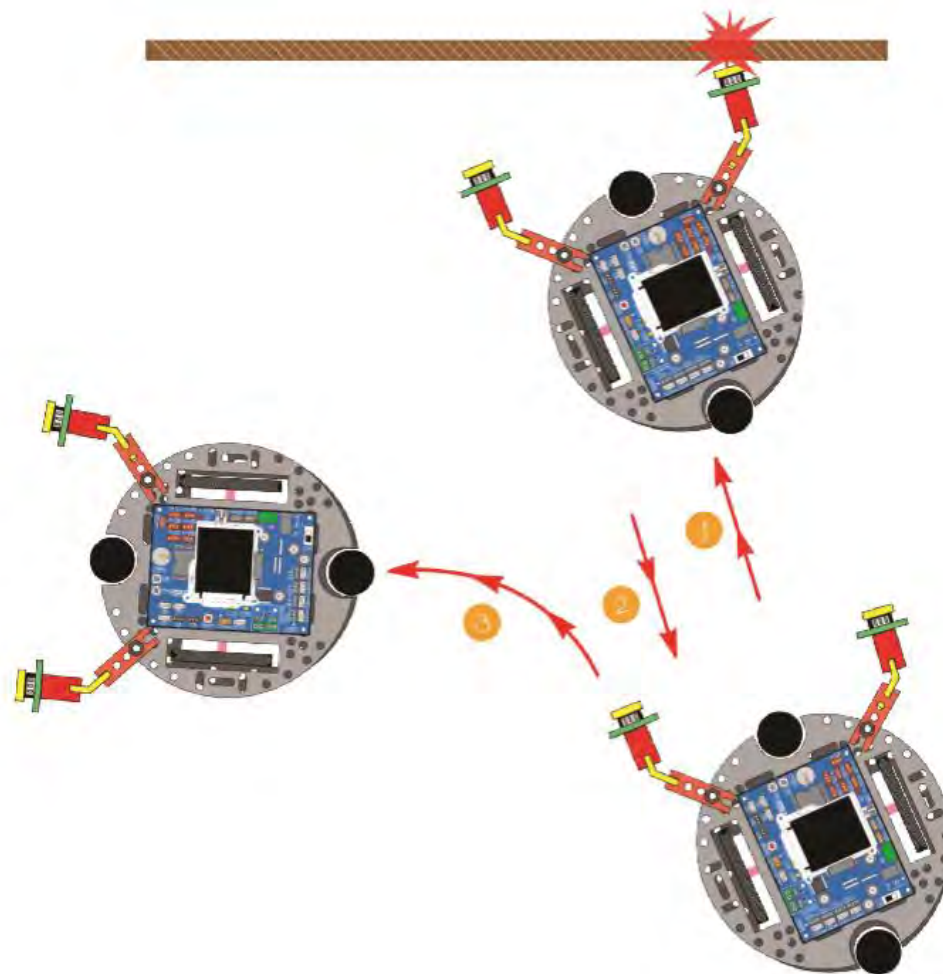
ทดสอบ เพิ่มค่าการนับจาก Switch ตัวที่ 1 และลดค่าการนับจาก Switch ตัวที่ 2

```
#include <POP32.h>
int count = 100;
void setup() {}
void loop() {
    oled.text(0,0,"count: %d    ",count);
    oled.show();
    if(in(3)==0){
        count++;
        beep();
        delay(100);
    }
    if(in(4)==0){
        count--;
        beep();
        delay(100);
    }
}
```

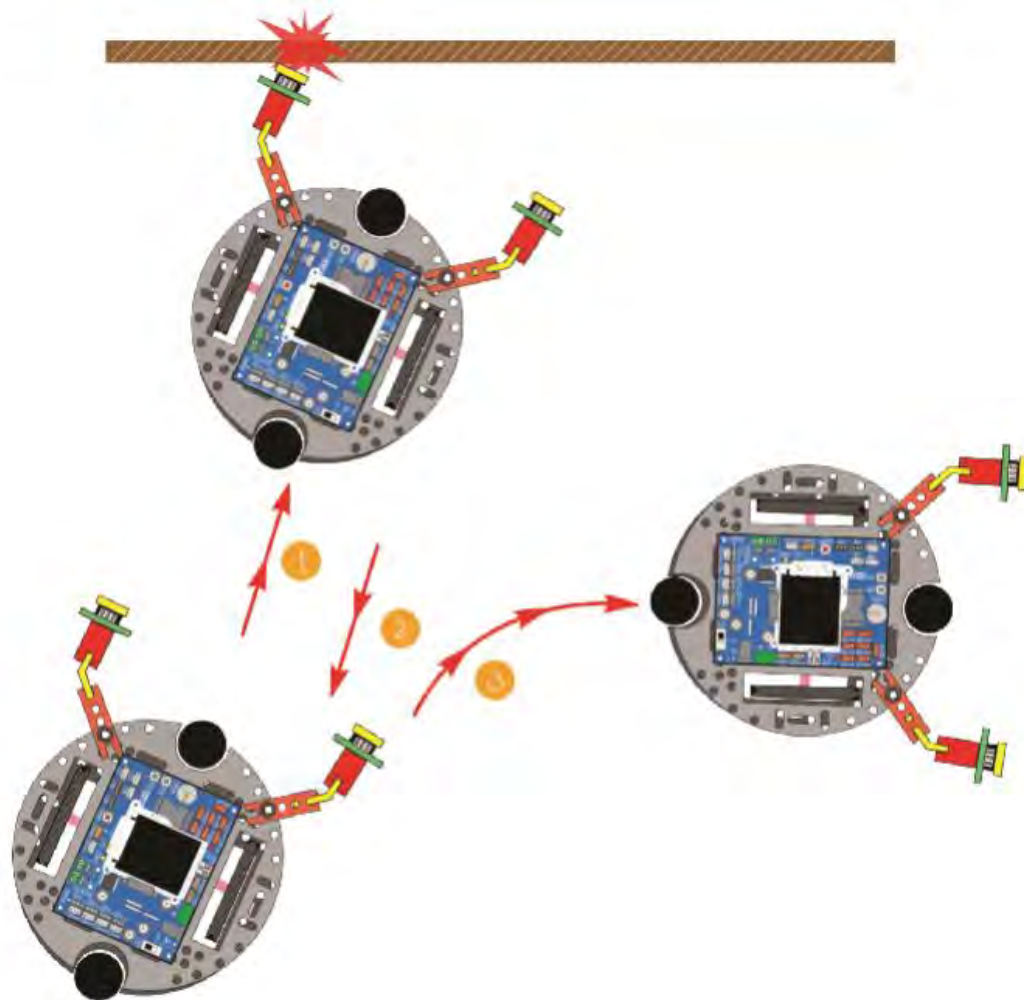
หุ่นยนต์หลบหลีกสิ่งกีดขวางจากการชน

การติดตั้งฮาร์ดแวร์

1. ติดตั้งเซนเซอร์สวิตช์เข้าที่ด้านหน้าซ้ายของหุ่นยนต์แล้วต่อเข้ากับพอร์ต 5
2. ติดตั้งเซนเซอร์สวิตช์เข้าที่ด้านหน้าขวาของหุ่นยนต์แล้วต่อเข้ากับพอร์ต 6



เมื่อหุ่นยนต์ชนจากทางด้านซ้าย หุ่นยนต์จะถอยหลังแล้วเลี้ยวไปทางด้านขวา และเคลื่อนที่ต่อไป



เมื่อหุ่นยนต์ชนจากทางด้านขวา หุ่นยนต์จะถอยหลังแล้วเลี้ยวไปทางด้านซ้าย และเคลื่อนที่ต่อไป

ทดสอบ หลบหลีกสิ่งกีดขวางจากการชนของเซนเซอร์สวิตช์

```
#include <POP32.h>
void setup(){
  oled.text(0,0,"Press SW_OK");
  oled.show();
  waitSW_OK();
}
void loop(){
  fd(50);
  if(in(5)==0) {
    bk(50); delay(500);
    sr(50); delay(400);
  }
  if(in(6)==0) {
    bk(50); delay(500);
    sl(50); delay(400);
  }
}
```

เซนเซอร์วัดระยะแบบอัลตราโซนิก (แผงวงจร ZX-SONAR1M)



- ให้ค่าผลลัพธ์เป็นแบบอะนาลอก ที่เชื่อถือได้ตั้งแต่ 2 ถึง 100 เซนติเมตร
- ระยะทางคลาดเคลื่อน +/- 2 เซนติเมตร

คำสั่ง analog(ch)

รูปแบบ

`analog(ch)`

โดยที่

ch คือ หมายเลขพอร์ตอะนาลอกที่ต่อใช้งาน

โดยกำหนดค่าเป็น 0-8(สำหรับ A0-A8 ตามลำดับ)

การคืนค่า

คืนค่าในช่วง 0 ถึง 4095

การคำนวณระยะทาง

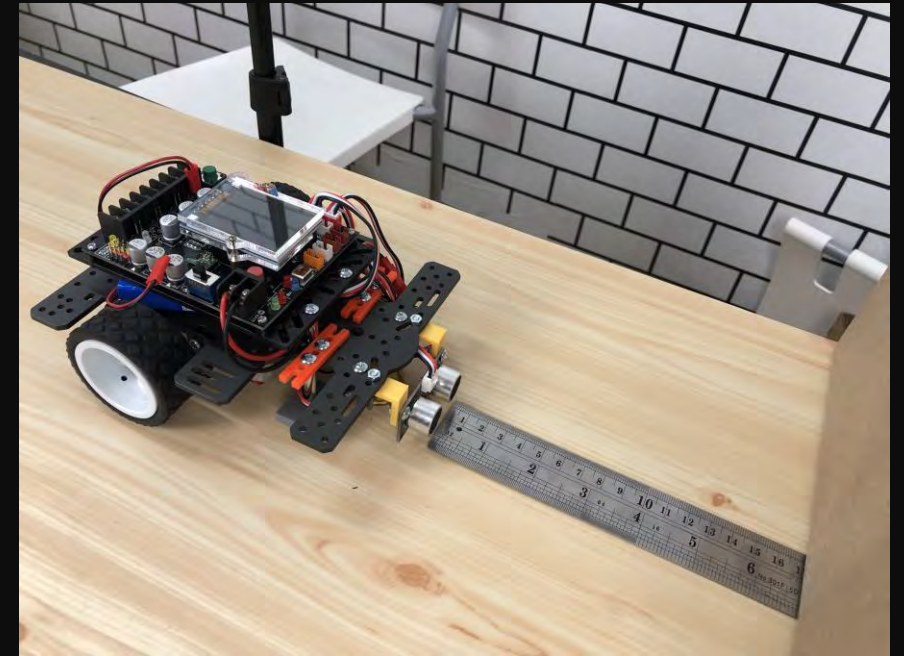
สูตรประมาณค่าระยะทาง

$$d = A/41$$

โดยที่

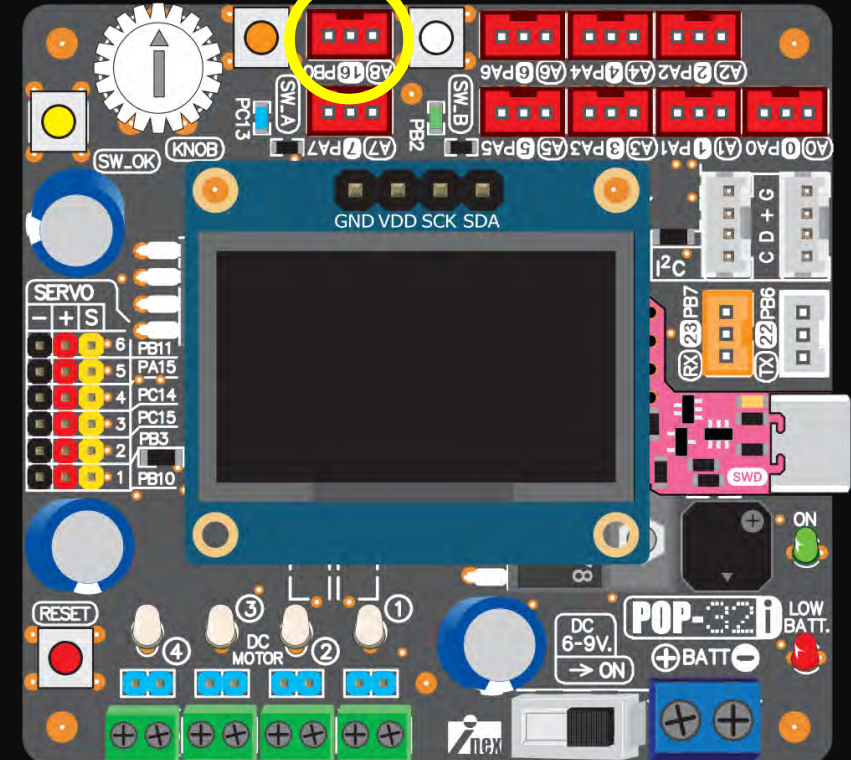
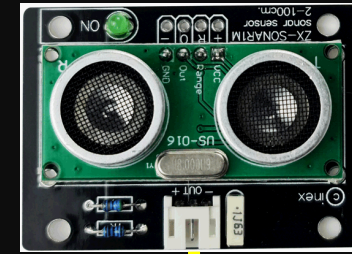
d คือ ค่าระยะทางในหน่วยเซนติเมตร(อาจมีความคลาดเคลื่อน +/- 2 เซนติเมตร)

A คือ ค่าอะนาล็อกที่อ่านในช่วง 0 ถึง 4095



ทดสอบ อ่านค่าระยะทางจาก ZX-Sonar 1M

```
#include <POP32.h>
void setup(){}
void loop()
{
    int dist = analog(A8)/41;
    oled.text(0,0,"dist: %d cm ",dist);
    oled.show();
}
```

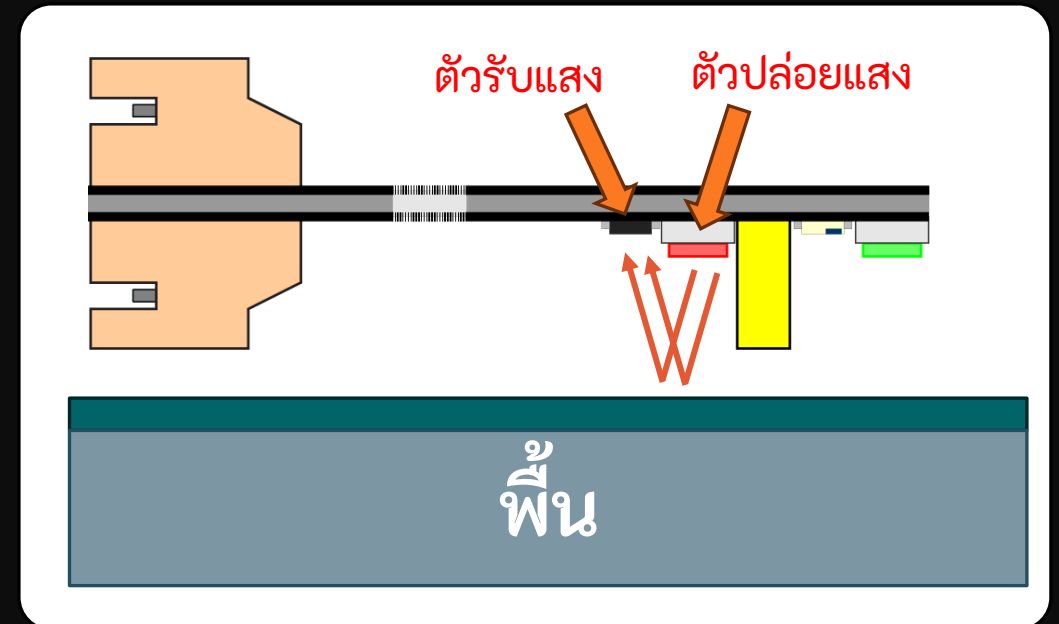


ทดสอบ หลบหลีกสิ่งกีดขวางที่ระยะต่ำกว่า 30 เซนติเมตร

```
#include <POP32.h>
void setup() {
    oled.text(0, 0, "Press SW_OK");
    oled.show();
    waitSW_OK();
}
void loop() {
    int dist = analog(A8) / 41;
    fd(30);
    if (dist < 30) {
        bk(30);
        delay(700);
        sl(30);
        delay(500);
    }
}
```

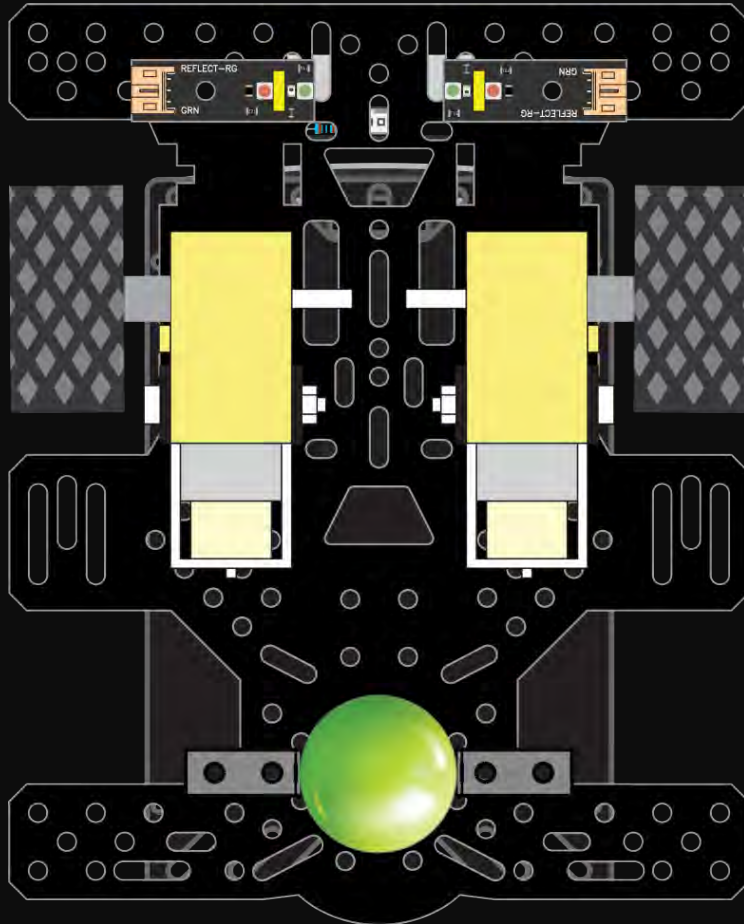
แผงวงจรตรวจจับสนแสง: REFLECT-RG

(เซ็นเซอร์ตรวจจับสน)

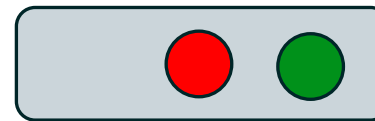


ตำแหน่งการติดตั้งเซนเซอร์ ทั้ง 2 ตัวเข้ากับหุ่นยนต์

เซนเซอร์เมื่อมองจากด้านล่าง



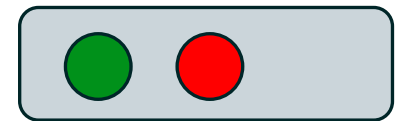
เซนเซอร์ด้านขวา



A3

A2

เซนเซอร์ด้านซ้าย



A1

A0

โปรแกรมอ่านค่าจาก REFLECT-RG แสดงที่ OLED เพื่อหาค่าเฉลี่ยอ้างอิง

```
#include <POP32.h>
void setup(){}
void loop()
{
    oled.text(0,0,"L=%d    ",analog(1));
    oled.text(1,0,"R=%d    ",analog(2));
    oled.show();
}
```

สรุปการทดสอบอ่านสภาวะแวดล้อมสนาม

พื้นสนามสีขาว

เซนเซอร์ด้านซ้าย 3800

เซนเซอร์ด้านขวา 3800

เส้นสนามสีดำ

เซนเซอร์ด้านซ้ายสูงสุด 1200

เซนเซอร์ด้านขวาสูงสุด 1400

สรุปค่าเฉลี่ยอ้างอิง

สำหรับด้านซ้าย

$$(3800+1200)/2 = 2500$$

สำหรับด้านขวา

$$(3800+1400)/2 = 2600$$

สรุปการทดสอบอ่านสถานะแวดล้อมสนาม

สรุปค่าอ้างอิงกึ่งกลาง

เซนเซอร์ด้านซ้าย 2500

เซนเซอร์ด้านขวา 2600

กำหนดตัวแปรการอ้างอิงเซนเซอร์ด้านซ้ายและขวา ดังนี้

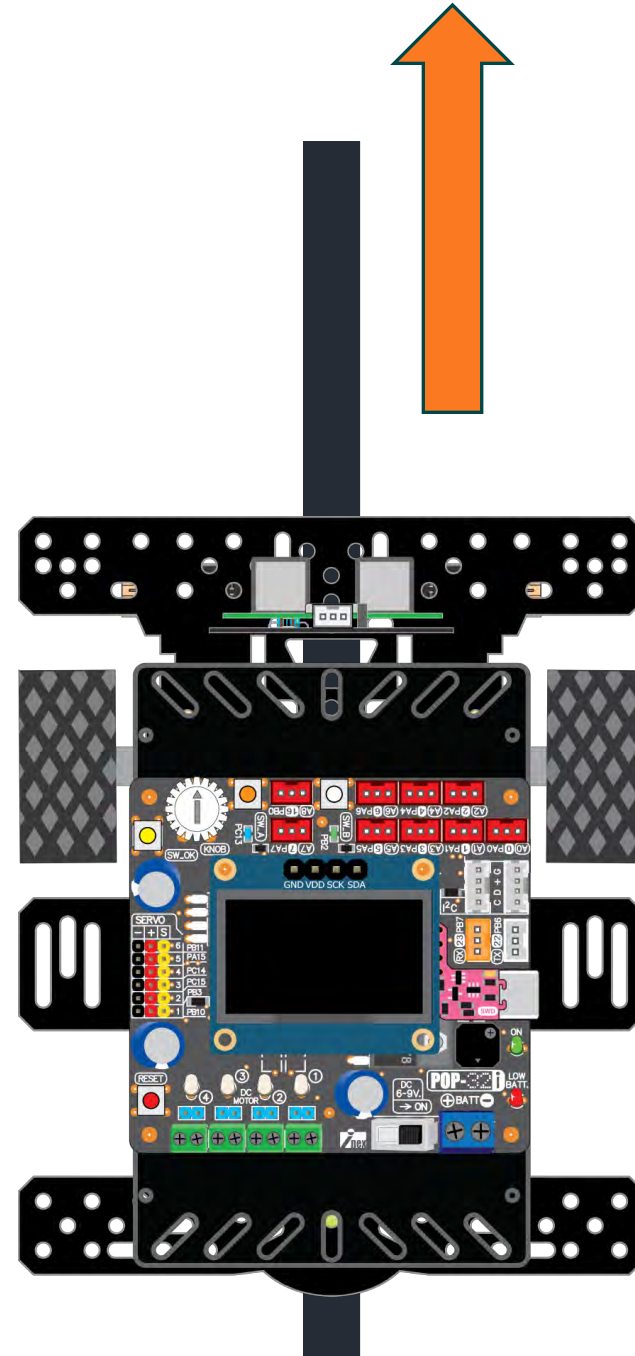
```
int mid_l = 2500;
```

```
int mid_r = 2600;
```

หลักการขับเคลื่อนหุ่นยนต์เคลื่อนที่ตามเส้น

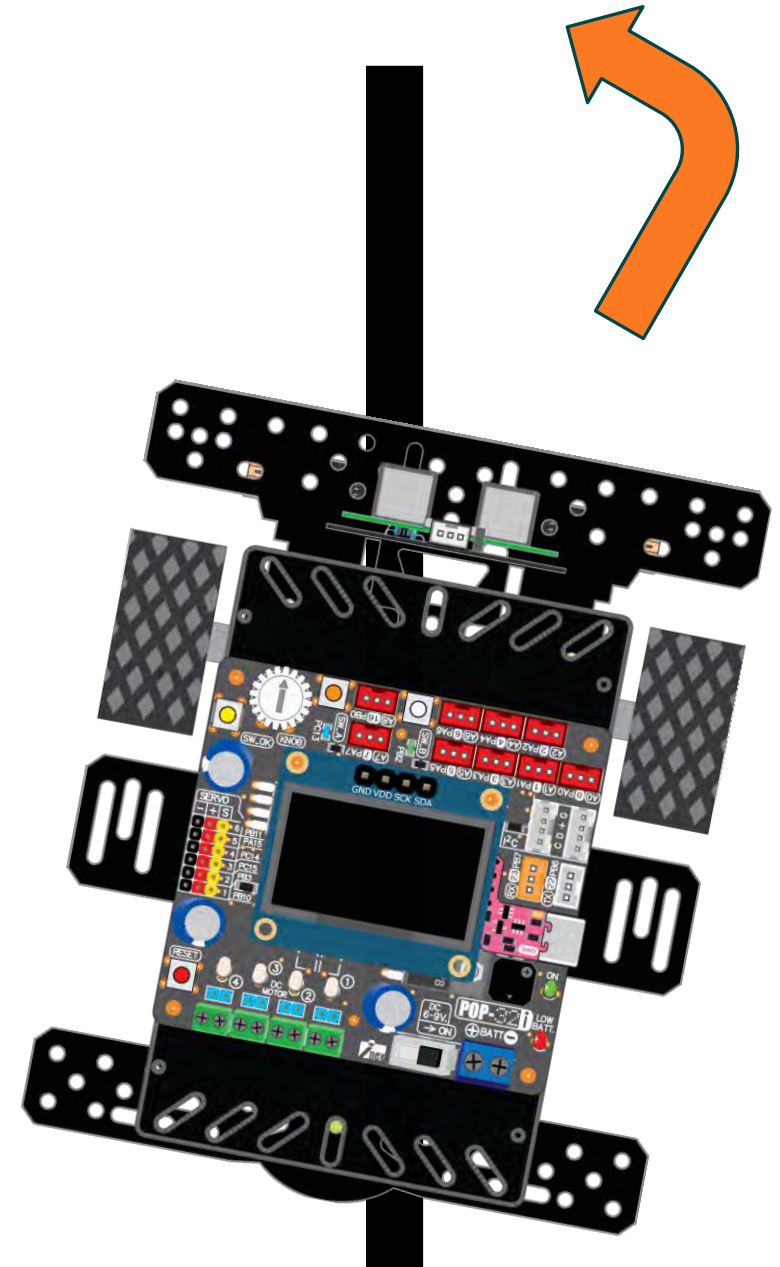
1. ด้านซ้ายและขวาพบพื้นสีขาว

```
if (L>mid_l&&R>mid_r)
{
    fd(40);
}
```



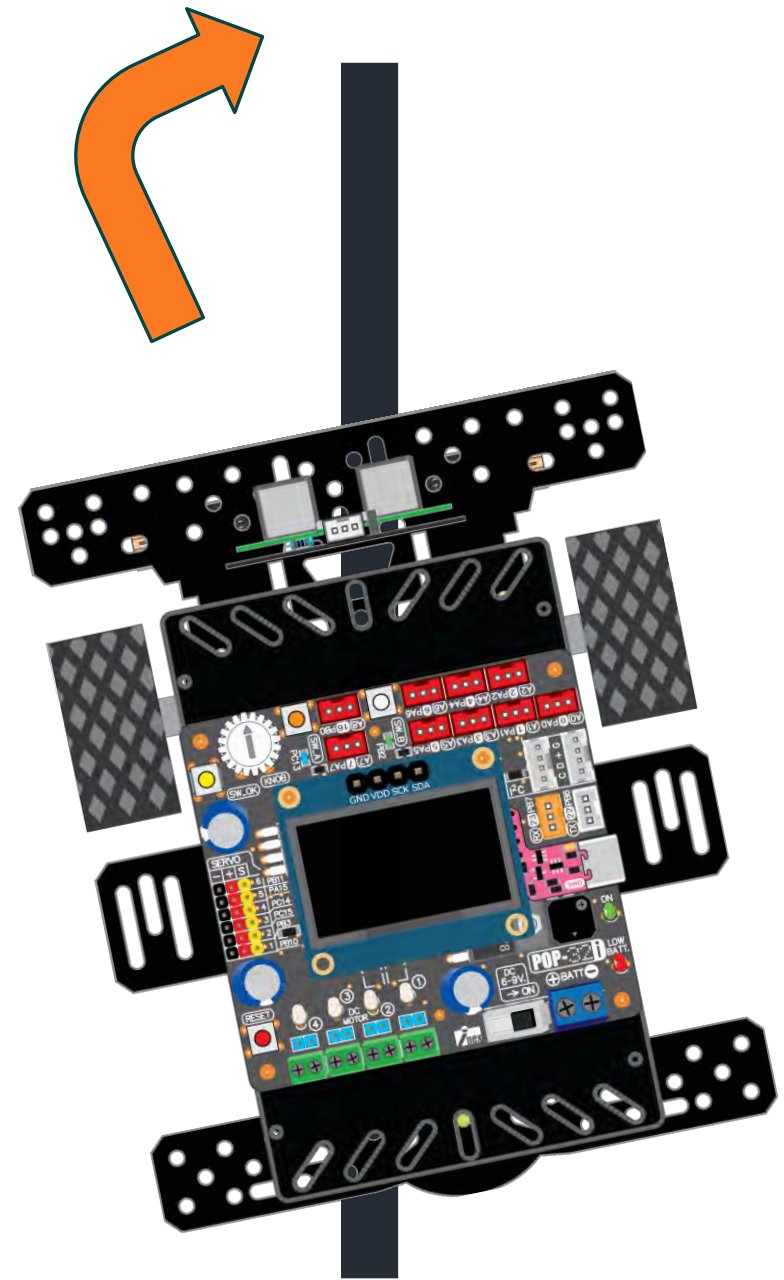
2. ด้านซ้ายพบเส้นสีดำ

```
if (L<mid_l&&R>mid_r)
{
    sl(40);
}
```



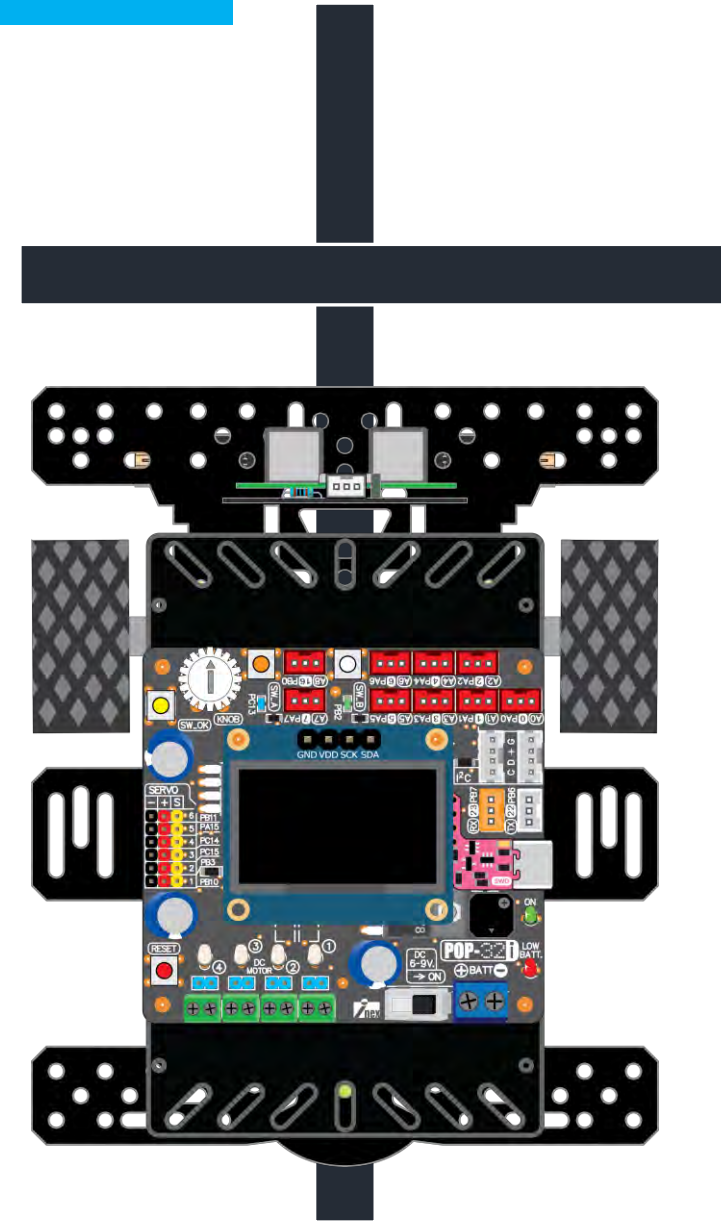
3. ด้านขวาพบเส้นสีดำ

```
if (L>mid_l&&R<mid_r)
{
    sr(40);
}
```

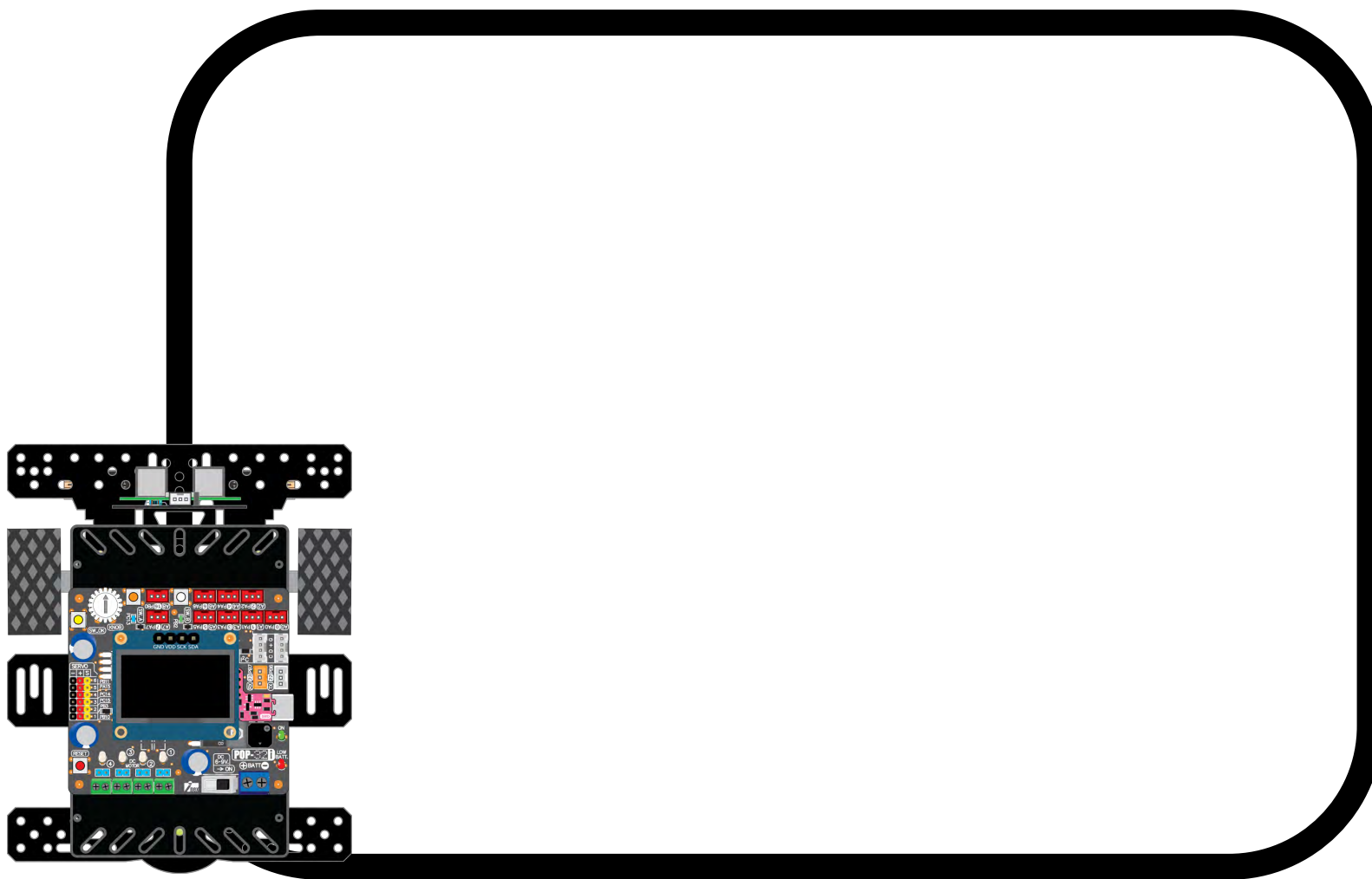


4. ด้านซ้ายและขวาพบเส้นสีดำ(เส้นตัด)

```
if (L<mid_l&&R<mid_r)
{
    // กำหนดได้ตามอิสระได้หลายรูปแบบเช่น
    // 1. อาจสั่งการให้หุ่นยนต์หยุด
    // 2. อาจสั่งการให้หุ่นยนต์ข้ามเส้นตัด
    // 3. อาจสั่งการให้หุ่นยนต์เลี้ยวไปทางซ้าย
    // 4. อาจสั่งการให้หุ่นยนต์เลี้ยวไปทางขวา
}
```



รูปแบบที่ 1 เคลื่อนที่ตามเส้นอย่างง่าย



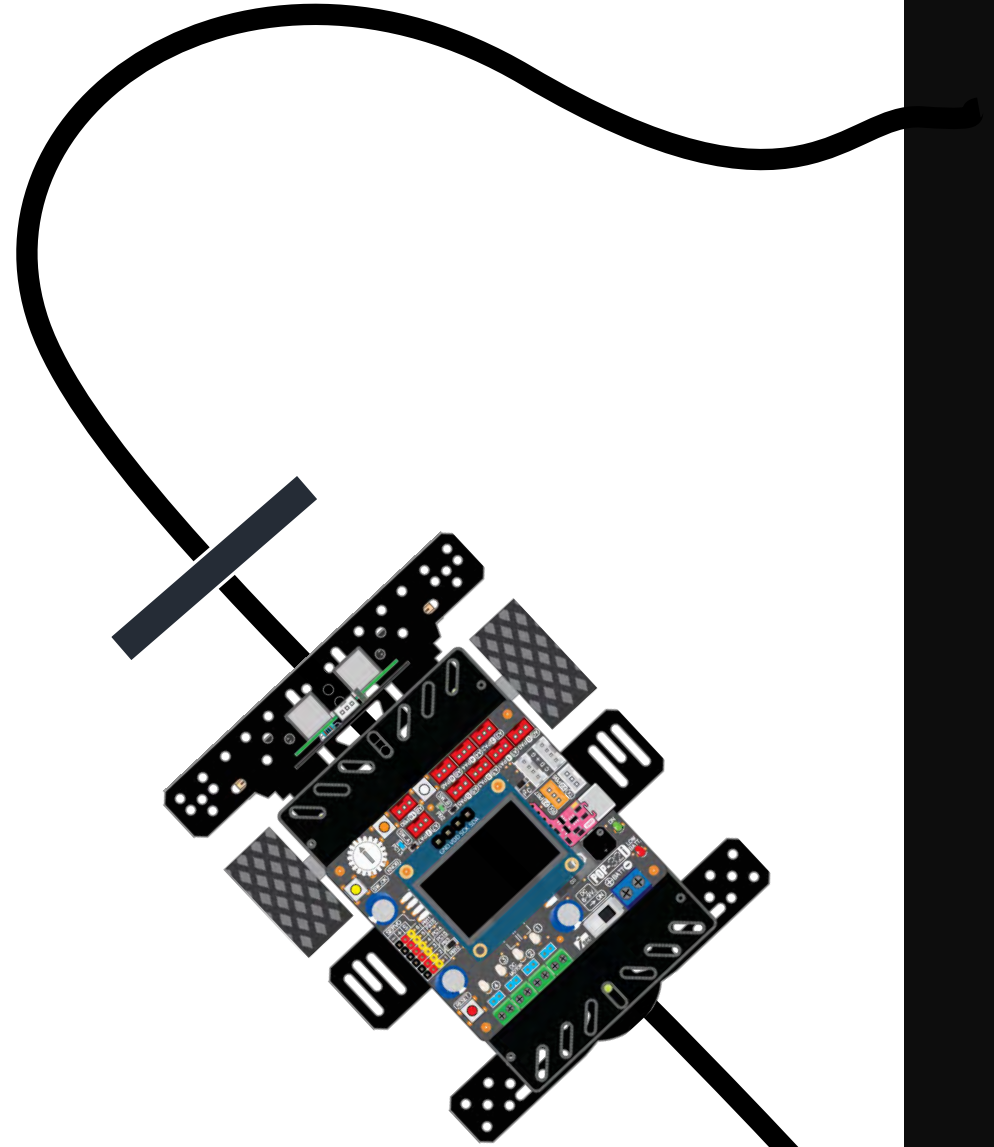
ทดสอบ หุ่นยนต์เคลื่อนที่ตามเส้นอย่างง่าย

```
int mid_l = 2500;
int mid_r = 2600;
void setup() {
    oled.text(0, 0, "Press SW_OK");
    oled.show();
    waitSW_OK();
}
```

```
void loop() {
    int L = analog(1);
    int R = analog(2);
    if (L > mid_l && R > mid_r) {
        fd(30);
    } else if (L < mid_l && R > mid_r) {
        sl(100);
    } else if (L > mid_l && R < mid_r) {
        sr(100);
    }
}
```

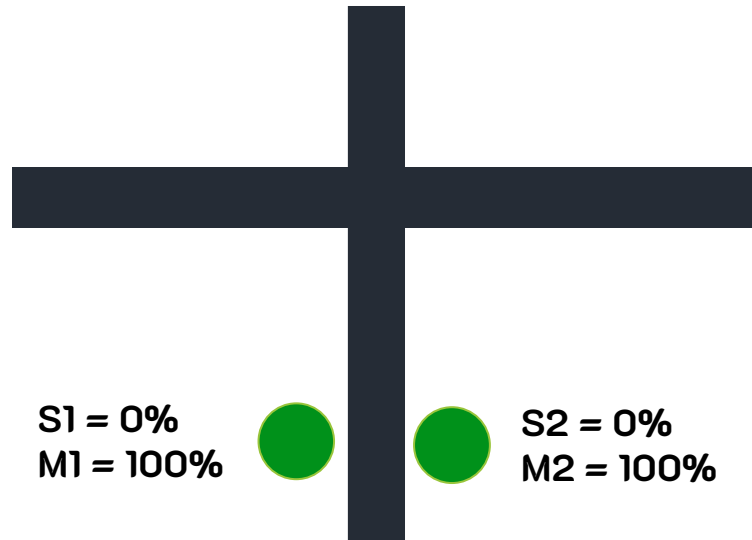
ทดสอบ หุ่นยนต์เมื่อพบเส้นตัดแจ้งเตือนด้วยเสียงและหยุดรอ 2 วินาที

```
#include <POP32.h>
int mid_l = 2500;
int mid_r = 2600;
void setup() {
    waitSW_OK_bmp();
}
void loop() {
    int L = analog(1);
    int R = analog(2);
    if (L > mid_l && R > mid_r) {
        fd(20);
    } else if (L < mid_l && R > mid_r) {
        sl(20);
    } else if (L > mid_l && R < mid_r) {
        sr(20);
    } else if (L < mid_l && R < mid_r) {
        ao();beep();delay(2000);
    }
}
```

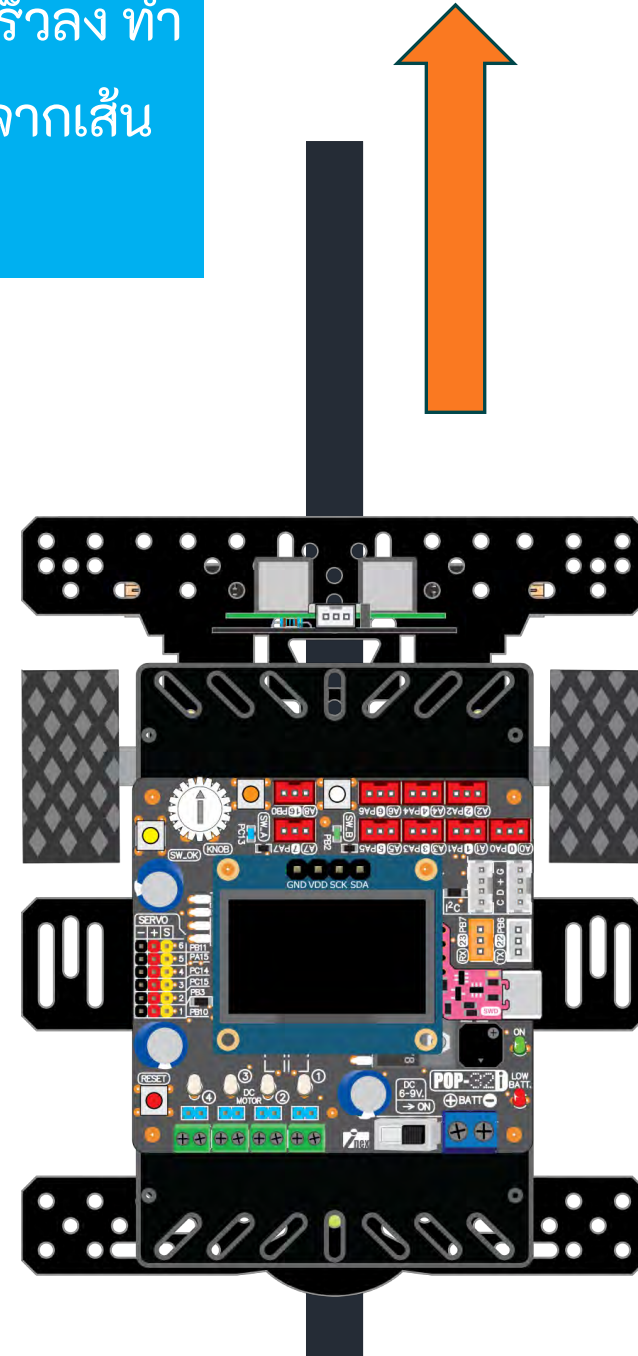


หลักการขับเคลื่อนหุ่นยนต์เคลื่อนที่ตามเส้น
ด้วยการปรับความเร็วของมอเตอร์

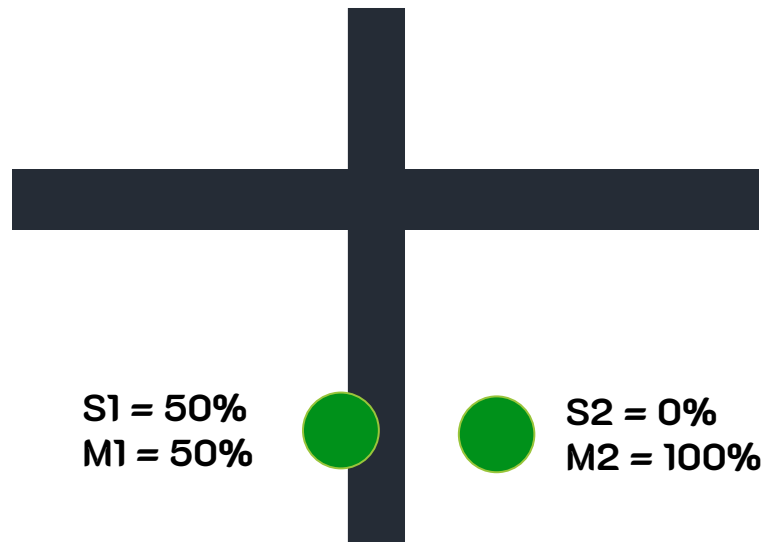
หลักการคิดเมื่อเซนเซอร์เจอเส้นสีดำล้อหุ่นยนต์ด้านนั้นต้องลงความเร็วลง ทำให้หุ่นยนต์เกิดการเคลื่อนที่เอียงไปด้านที่เจอเส้น เซ็นเซอร์จึงอยู่ห่างจากเส้นตลอด ส่งผลให้หุ่นยนต์วิ่งตามเส้นไปเรื่อยๆ



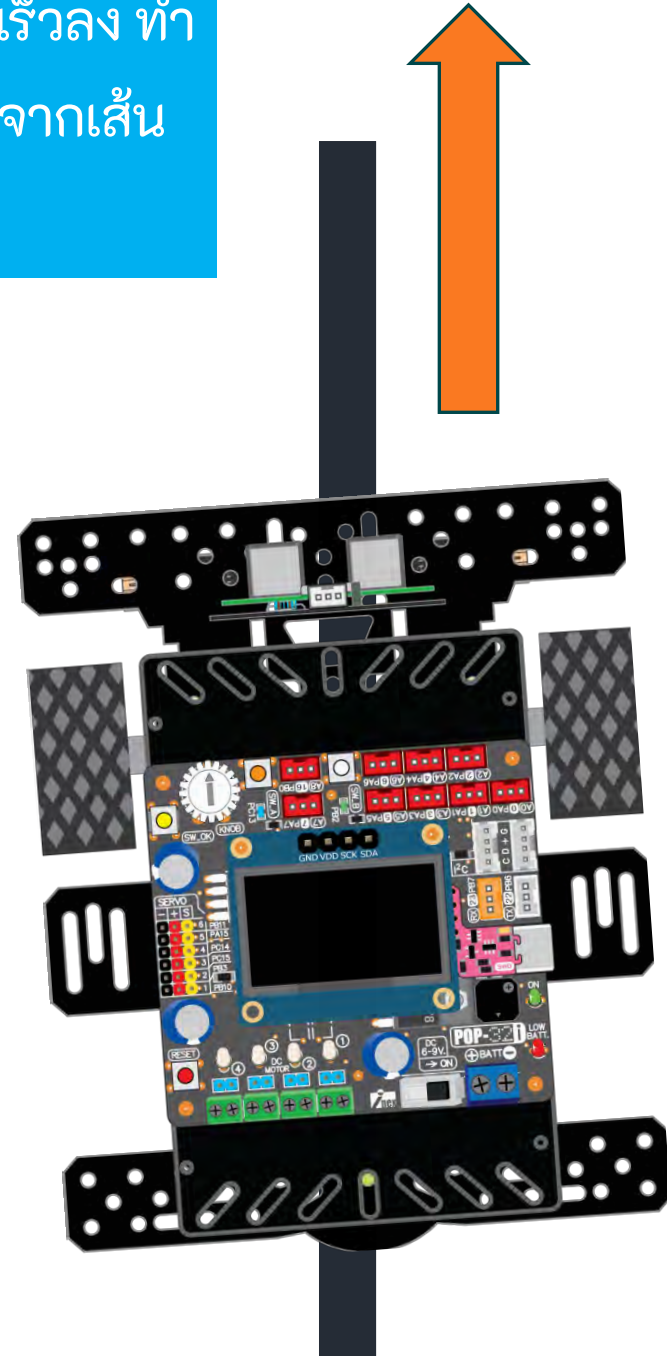
กรณีที่เซนเซอร์ทั้ง 2 ตัว อยู่ระหว่างเส้นมีค่าเท่าๆกันหุ่นยนต์จะเดินตรงไป



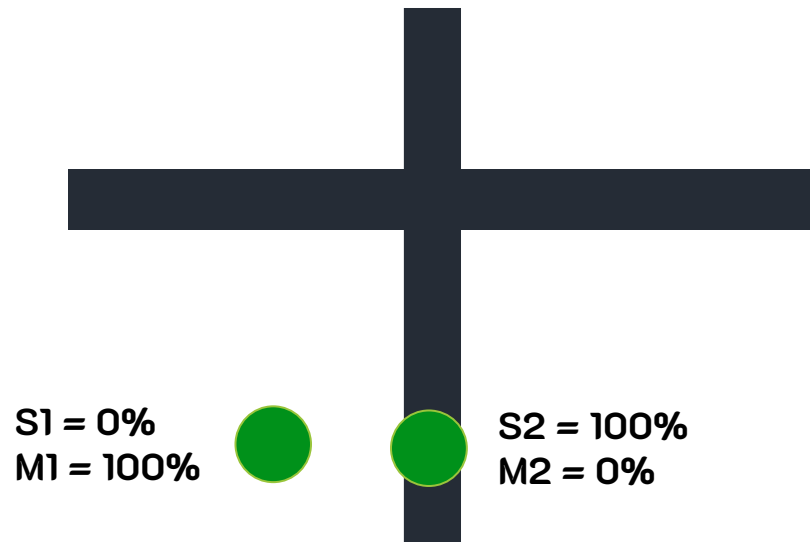
หลักการคิดเมื่อเซนเซอร์เจอเส้นสีดำล้อหุ่นยนต์ด้านนั้นต้องลงความเร็วลง ทำให้หุ่นยนต์เกิดการเคลื่อนที่เอียงไปด้านที่เจอเส้น เซนเซอร์จึงอยู่ห่างจากเส้นตลอด ส่งผลให้หุ่นยนต์วิ่งตามเส้นไปเรื่อยๆ



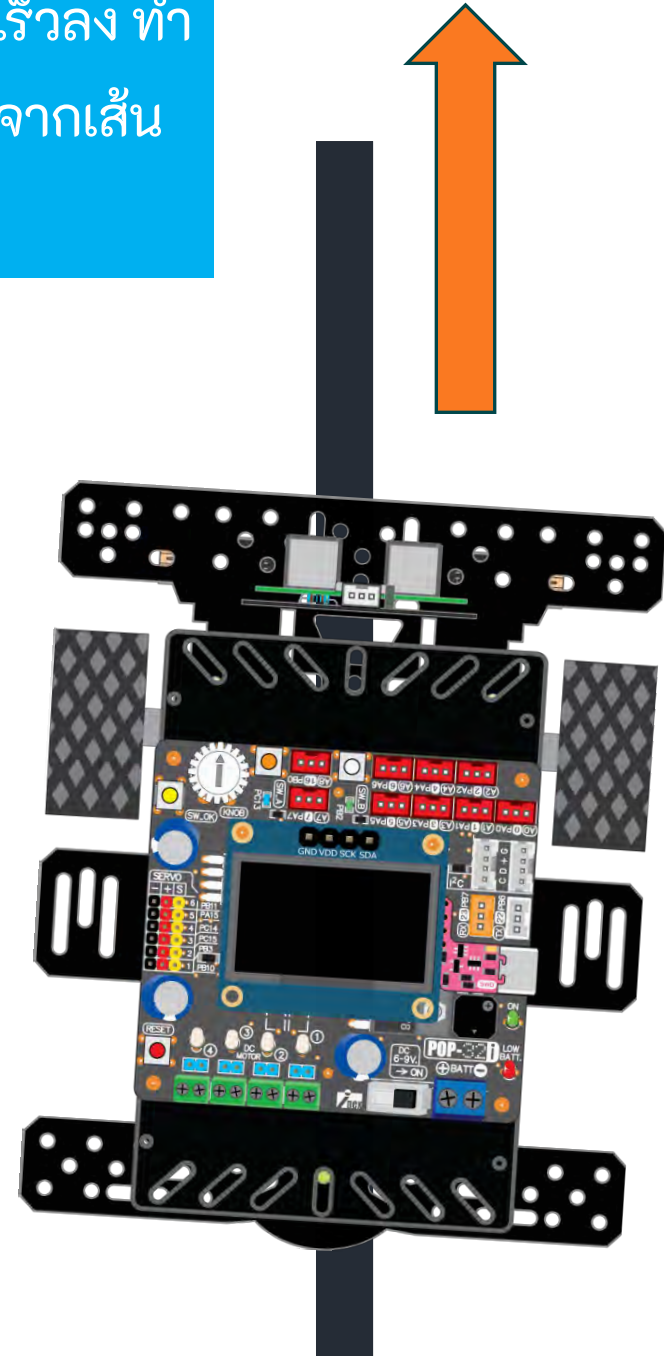
กรณีที่เซนเซอร์ S1 ช่างซ้ายเคลื่อนที่ทับเส้นประมาณ 50 % มอเตอร์ด้านซ้าย M1 ต้องลดความเร็วลงเพื่อให้หุ่นยนต์เอียงกลับมา



หลักการคิดเมื่อเซนเซอร์เจอเส้นสีดำ หุ่นยนต์ด้านนั้นต้องลดความเร็วลง ทำให้หุ่นยนต์เกิดการเคลื่อนที่เอียงไปด้านที่เจอเส้น เซนเซอร์จึงอยู่ห่างจากเส้นตลอด ส่งผลให้หุ่นยนต์วิ่งตามเส้นไปเรื่อยๆ



กรณีที่เซนเซอร์ S2 ข้างขวาเคลื่อนที่ทับเส้นประมาณ 100 % มอเตอร์ด้านขวา M2 ต้องลดความเร็วลง 0 % เพื่อให้หุ่นยนต์เอียงกลับมา

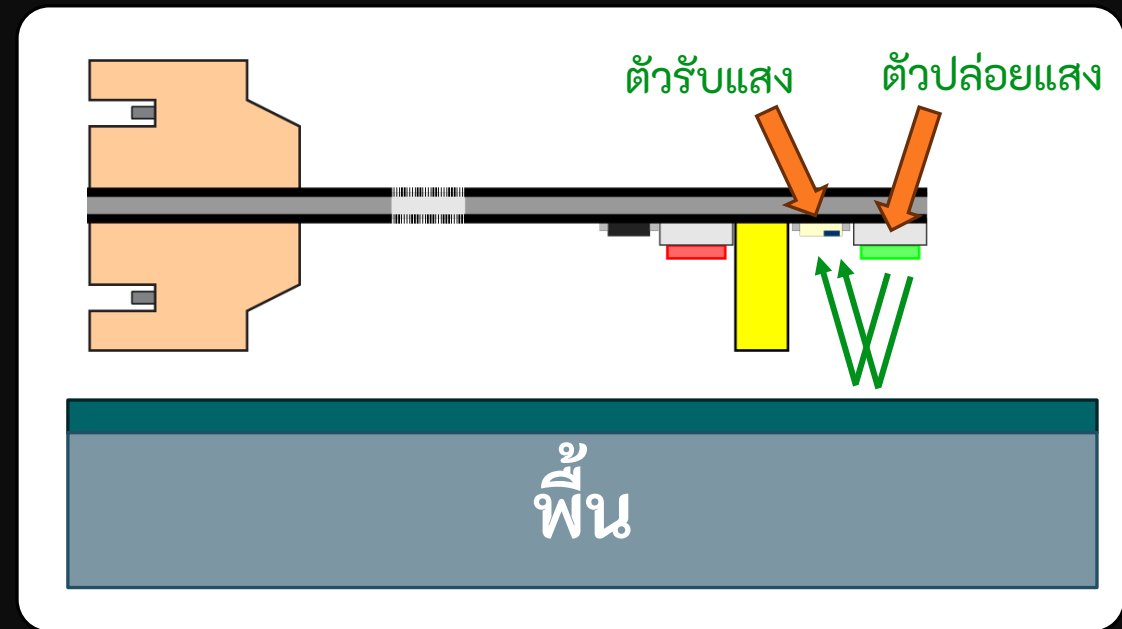


แผงวงจรตรวจจับสนแสง: REFLECT-RG

(เซนเซอร์ตรวจจับสน)



เมื่อเจอพื้นสีดำค่าที่อ่านได้จะน้อย หากเจอพื้นสีขาวค่าที่อ่านได้จะมาก
โดยค่าที่อ่านได้คือ 0 - 4095



คำสั่ง

`map(value, fromLow, fromHigh, toLow, toHigh)`

เป็นคำสั่งที่ใช้ปรับช่วงที่เราต้องการ

ตัวอย่าง

```
int y = map(2500,1200,3800,0,100)
```

y จะมีค่าเท่ากับ 50

คำสั่ง

constrain(value,min,max)

เป็นคำสั่งที่ใช้กำหนดค่าให้อยู่ในช่วงที่เราต้องการ

ตัวอย่าง

```
int y = constrain(300,-180,180)
```

y จะมีค่าเท่ากับ 180

จากหลักแนวคิด เมื่อเซนเซอร์มีค่าไปทิศทางเดียวกันกับความเร็วมอเตอร์ สิ่งที่เราต้องทราบมีดังต่อไปนี้

1. ต้องทราบก่อนว่าสีไหนเป็นสีของเส้นและสีไหนเป็นสีของพื้น
2. จดบันทึกค่าสีของเส้นและค่าสีของพื้น
3. ปรับช่วงของเซนเซอร์ให้อยู่ 0 – 100 โดยให้สีของเส้นเท่ากับ 0 และสีพื้นเท่ากับ 100
4. นำค่าที่ได้จากข้อ 3 ทั้ง 2 ข้างไปคูณกับค่าเกณฑ์ที่ต้องการ
5. นำค่าที่ได้จากข้อ 4 ไปกำหนดความเร็วสูงสุดที่ต้องการ

ตัวอย่างโปรแกรม ข้างเดียว

```
int min = 1200;
int max = 3800;
int maxSP = 40;
float k = 1.0;
int val_S1 = map(analog(1), min, max, 0, 100);
int SP_M1 = constrain(val_S1 * k, 0, maxSP);
motor(1, SP_M1);
```


ตัวอย่างโปรแกรม การเคลื่อนที่โดยการปรับความเร็วมอเตอร์

```
#include <POP32.h>
int min_S1 = 1200;
int max_S1 = 3800;

int min_S2 = 1400;
int max_S2 = 3900;

int maxSP = 50;
float k = 1.0;

void setup() {
    while (!SW_OK()) {
        oled.text(0, 0, "L=%d  ", analog(1));
        oled.text(1, 0, "R=%d  ", analog(2));
        oled.show();
    }
    beep();
}
```

```
void loop() {
    int val_S1 = map(analog(1), min_S1, max_S1, 0, 100);
    int SP_M1 = constrain(val_S1 * k, 0, maxSP);

    int val_S2 = map(analog(2), min_S2, max_S2, 0, 100);
    int SP_M2 = constrain(val_S2 * k, 0, maxSP);

    if (val_S1 < 10 && val_S2 < 10) {
        ao();beep();delay(2000);
    } else {
        motor(1, SP_M1);
        motor(2, SP_M2);
    }
}
```

คำสั่ง

```
#define macro(args)
```

เป็นคำสั่งที่ใช้กำหนดตัวแปร พร้อมคำสั่ง

ตัวอย่าง

```
#define val_S1 map(analog(1), min_S1, max_S1, 0, 100)
```

val_S1 จะมีค่าอยู่ในช่วง 0 – 100 และสามารถเรียกใช้ได้จากทุกส่วนในโปรแกรม

ตัวอย่างโปรแกรม การเคลื่อนที่โดยการปรับความเร็วมอเตอร์ (แบบกำหนด macro)

```
#include <POP32.h>
int min_S1 = 1200;
int max_S1 = 3800;
int min_S2 = 1400;
int max_S2 = 3900;
int maxSP = 50;
float k = 1.0;
```

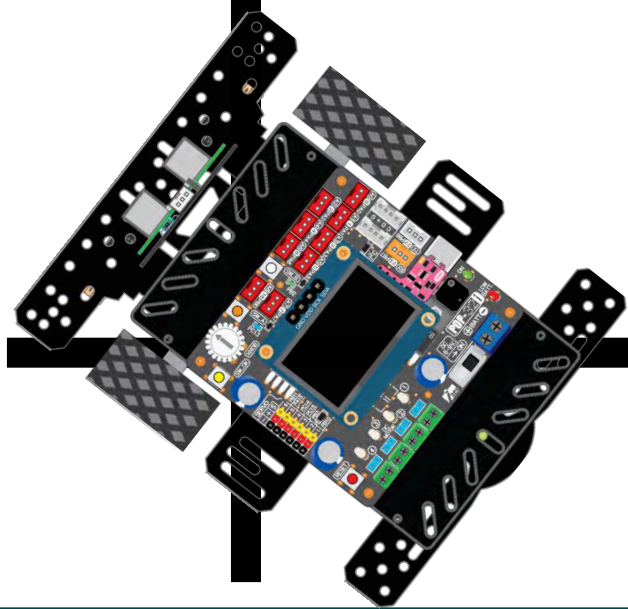
```
#define val_S1 map(analog(1), min_S1, max_S1, 0, 100)
#define val_S2 map(analog(2), min_S2, max_S2, 0, 100)
```

```
void setup() {
  while (!SW_OK()) {
    oled.text(0, 0, "L=%d  ", analog(1));
    oled.text(1, 0, "R=%d  ", analog(2));
    oled.show();
  }
  beep();
}
```

```
void loop() {
  if (val_S1 < 10 && val_S2 < 10) {
    ao();beep();delay(2000);
  } else {
    int SP_M1 = constrain(val_S1 * k, 0, maxSP);
    int SP_M2 = constrain(val_S2 * k, 0, maxSP);
    motor(1, SP_M1);
    motor(2, SP_M2);
  }
}
```

Code -> Track_line.ino

ตัวอย่างการสร้างฟังก์ชันขับเคลื่อนหุ่นยนต์เมื่อพบ
เส้นตัดหรือทางแยก

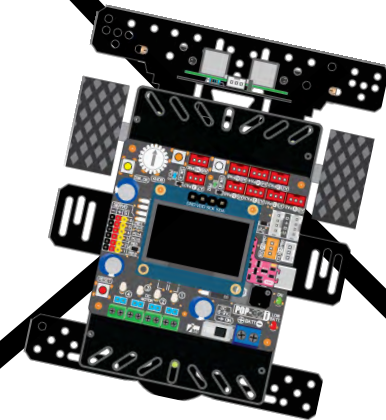


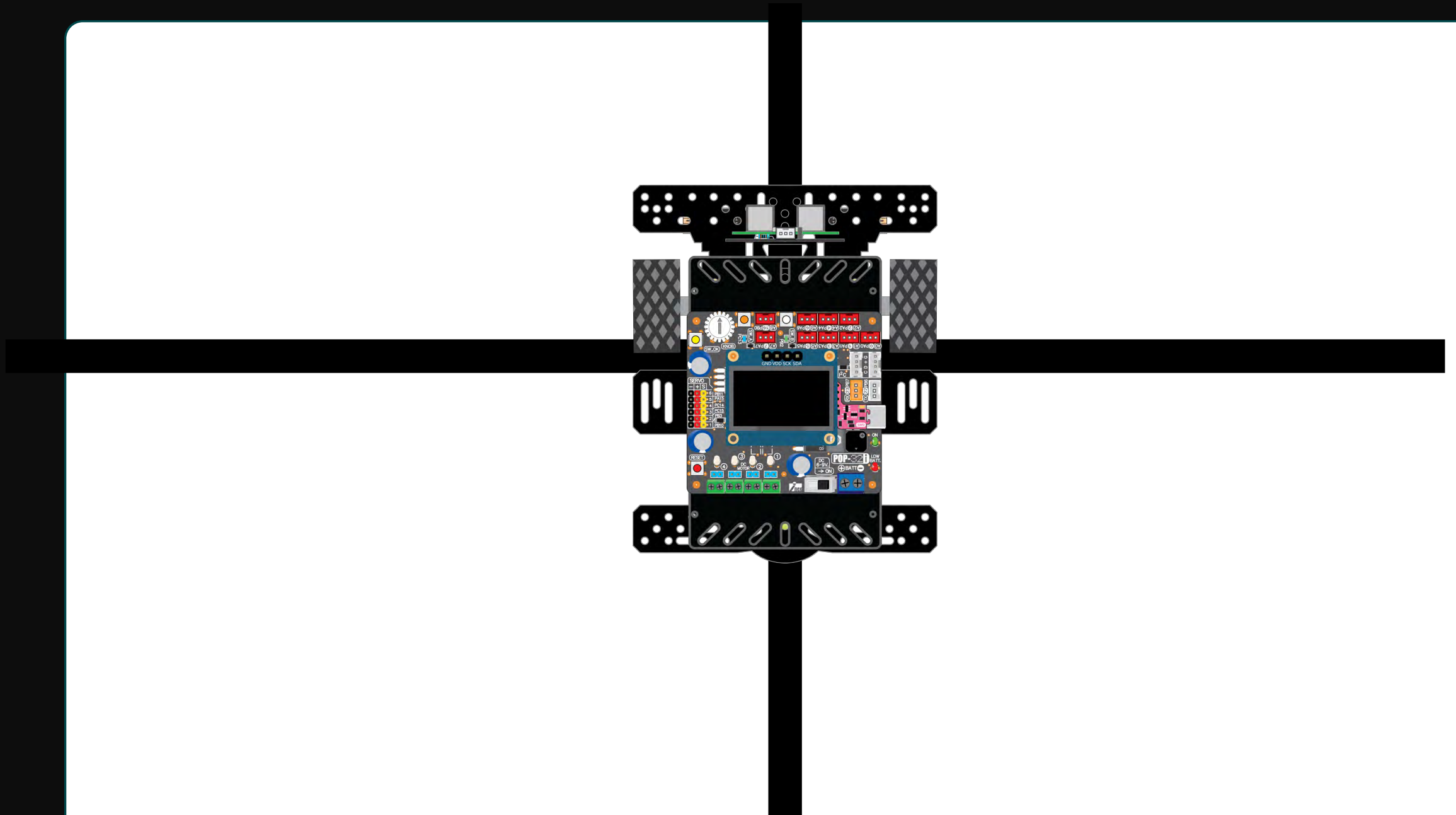
โปรแกรมทดสอบเจอเส้นตัด เลี้ยวขวา

Code -> Track_line_SR.ino

```
void loop() {  
  if (val_S1 < 10 && val_S2 < 10) {  
    fd(20); sound(500, 100);  
    sr(20); delay(1000);  
  } else {  
    int SP_M1 = constrain(val_S1 * k, 0, maxSP);  
    int SP_M2 = constrain(val_S2 * k, 0, maxSP);  
    motor(1, SP_M1);  
    motor(2, SP_M2);  
  }  
}
```

fd(20); sound(500, 100);
sr(20); delay(1000);





คำสั่งตรวจสอบเงื่อนไข **while**

รูปแบบ

```
while(เงื่อนไข) {  
    ทำคำสั่งจนเงื่อนไขเป็นเท็จ;  
}
```

ออกจาก **while** แบบทันทีได้ด้วยคำสั่ง

break;

```
#include <POP32.h>  
void setup() {}  
void loop() {  
    while(SW_A()){  
        beep();  
    }  
}
```

กดสวิตช์แล้วต้องรอปปล่อยถึงจะหยุดเสียงดัง

ทบทวนรูปแบบการสร้างฟังก์ชัน

ชื่อฟังก์ชัน ตัวแปรที่ส่งไปยังฟังก์ชัน

```
void R90(int x){  
    ชุดคำสั่ง  
    y=x+2;  
    ...  
}
```

ชุดคำสั่งในฟังก์ชัน

การใช้งานฟังก์ชัน **R90(200);**

ฟังก์ชันเจอเส้นตัดเลี้ยวขวา/ซ้าย

```
void R90(){  
    fd(30); sound(1500,150);  
    while(val_S2<10){sr(30);}   
    while(val_S2>10){sr(30);}   
    while(val_S1>20){sr(30);}   
}
```

```
void L90(){  
    fd(30); sound(1500,150);  
    while(val_S1<10){sl(30);}   
    while(val_S1>10){sl(30);}   
    while(val_S2>20){sl(30);}   
}
```

```
void UTurn(){  
    bk(30); sound(1500,450);  
    while(val_S2<10){sr(30);}   
    while(val_S2>10){sr(30);}   
    while(val_S1>20){sr(30);}   
}
```

โปรแกรมทดสอบจอเส้นตัด เลี้ยวขวา (ปรับปรุง)

```
#include <POP32.h>
int min_S1 = 1900; //อ่านค่าเส้นสีดำ ด้านซ้าย
int max_S1 = 3700; //อ่านค่าพื้นสีขาว ด้านซ้าย
int min_S2 = 1900; //อ่านค่าเส้นสีดำ ด้านขวา
int max_S2 = 3700; //อ่านค่าพื้นสีขาว ด้านขวา
int maxSP = 50; //ความเร็วสูงสุดที่ต้องการ
float k = 1.0; //ค่าอัตราการขยายของความไวเซนเซอร์

#define val_S1 map(analog(1), min_S1, max_S1, 0, 100)
#define val_S2 map(analog(2), min_S2, max_S2, 0, 100)

void setup() {
    while (!SW_OK()) { //กดปุ่ม OK เพื่อเริ่มต้นทำงาน
        oled.text(0, 0, "L=%d  ", analog(1));
        oled.text(2, 0, "R=%d  ", analog(2));
        oled.text(4, 0, "VS1=%d  ", val_S1);
        oled.text(6, 0, "VS2=%d  ", val_S2);
        oled.show();
    }
    beep();
}
```

Code -> Track_line_R90_L90_UTurn.ino

```
void loop() {
    if (val_S1 < 10 && val_S2 < 10) {
        R90();
    } else {
        int SP_M1 = constrain(val_S1 * k, 0, maxSP);
        int SP_M2 = constrain(val_S2 * k, 0, maxSP);
        motor(1, SP_M1);
        motor(2, SP_M2);
    }
}

void R90(){
    fd(30);sound(1500,150);
    while(val_S2<10){sr(30);}
    while(val_S2>10){sr(30);}
    while(val_S1>20){sr(30);}
}
```

โปรแกรมทดสอบเจอเส้นตัด เลี้ยวซ้าย (ปรับปรุง)

```
#include <POP32.h>
int min_S1 = 1900; //อ่านค่าเส้นสีดำ ด้านซ้าย
int max_S1 = 3700; //อ่านค่าพื้นสีขาว ด้านซ้าย
int min_S2 = 1900; //อ่านค่าเส้นสีดำ ด้านขวา
int max_S2 = 3700; //อ่านค่าพื้นสีขาว ด้านขวา
int maxSP = 50; //ความเร็วสูงสุดที่ต้องการ
float k = 1.0; //ค่าอัตราการขยายของความไวเซนเซอร์

#define val_S1 map(analog(1), min_S1, max_S1, 0, 100)
#define val_S2 map(analog(2), min_S2, max_S2, 0, 100)

void setup() {
    while (!SW_OK()) { //กดปุ่ม OK เพื่อเริ่มต้นทำงาน
        oled.text(0, 0, "L=%d  ", analog(1));
        oled.text(2, 0, "R=%d  ", analog(2));
        oled.text(4, 0, "VS1=%d  ", val_S1);
        oled.text(6, 0, "VS2=%d  ", val_S2);
        oled.show();
    }
    beep();
}
```

Code -> Track_line_R90_L90_UTurn.ino

```
void loop() {
    if (val_S1 < 10 && val_S2 < 10) {
        L90();
    } else {
        int SP_M1 = constrain(val_S1 * k, 0, maxSP);
        int SP_M2 = constrain(val_S2 * k, 0, maxSP);
        motor(1, SP_M1);
        motor(2, SP_M2);
    }
}

void L90(){
    fd(30); sound(1500,150);
    while(val_S1<10){s1(30);}
    while(val_S1>10){s1(30);}
    while(val_S2>20){s1(30);}
}
```

โปรแกรมทดสอบเจอเส้นตัด กลับตัว (ปรับปรุง)

```
#include <POP32.h>
int min_S1 = 1900; //อ่านค่าเส้นสีดำ ด้านซ้าย
int max_S1 = 3700; //อ่านค่าพื้นสีขาว ด้านซ้าย
int min_S2 = 1900; //อ่านค่าเส้นสีดำ ด้านขวา
int max_S2 = 3700; //อ่านค่าพื้นสีขาว ด้านขวา
int maxSP = 50; //ความเร็วสูงสุดที่ต้องการ
float k = 1.0; //ค่าอัตราการขยายของความไวเซนเซอร์

#define val_S1 map(analog(1), min_S1, max_S1, 0, 100)
#define val_S2 map(analog(2), min_S2, max_S2, 0, 100)

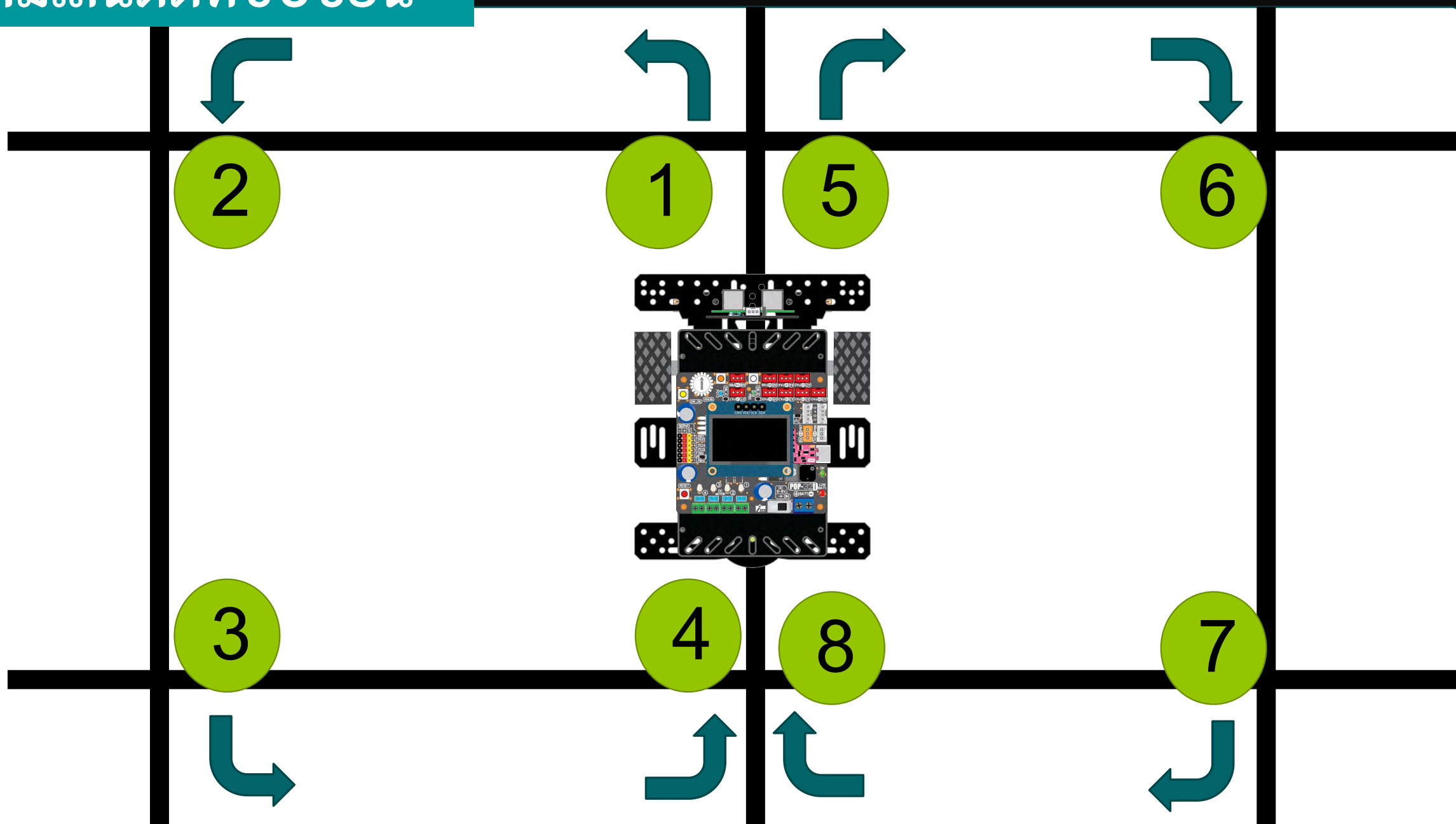
void setup() {
    while (!SW_OK()) { //กดปุ่ม OK เพื่อเริ่มต้นทำงาน
        oled.text(0, 0, "L=%d  ", analog(1));
        oled.text(2, 0, "R=%d  ", analog(2));
        oled.text(4, 0, "VS1=%d  ", val_S1);
        oled.text(6, 0, "VS2=%d  ", val_S2);
        oled.show();
    }
    beep();
}
```

Code -> Track_line_R90_L90_UTurn.ino

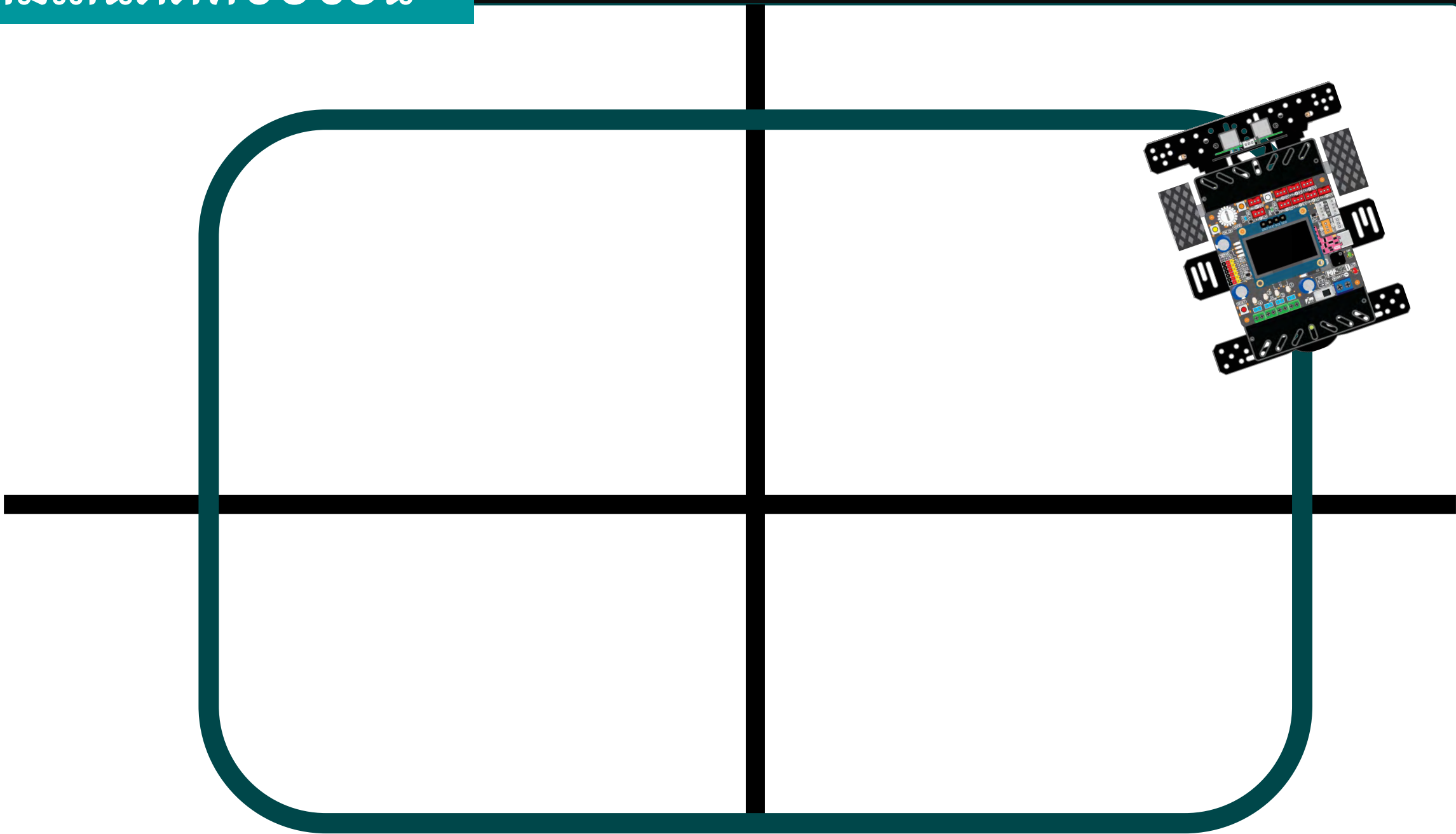
```
void loop() {
    if (val_S1 < 10 && val_S2 < 10) {
        UTurn();
    } else {
        int SP_M1 = constrain(val_S1 * k, 0, maxSP);
        int SP_M2 = constrain(val_S2 * k, 0, maxSP);
        motor(1, SP_M1);
        motor(2, SP_M2);
    }
}

void UTurn(){
    bk(30);sound(1500,450);
    while(val_S2<10){sr(30);}
    while(val_S2>10){sr(30);}
    while(val_S1>20){sr(30);}
}
```

เดินตามเส้นตัดที่ซับซ้อน



เดินตามเส้นตัดที่ซับซ้อน



โปรแกรมทดสอบเดินตามเส้นตัดที่ซับซ้อน

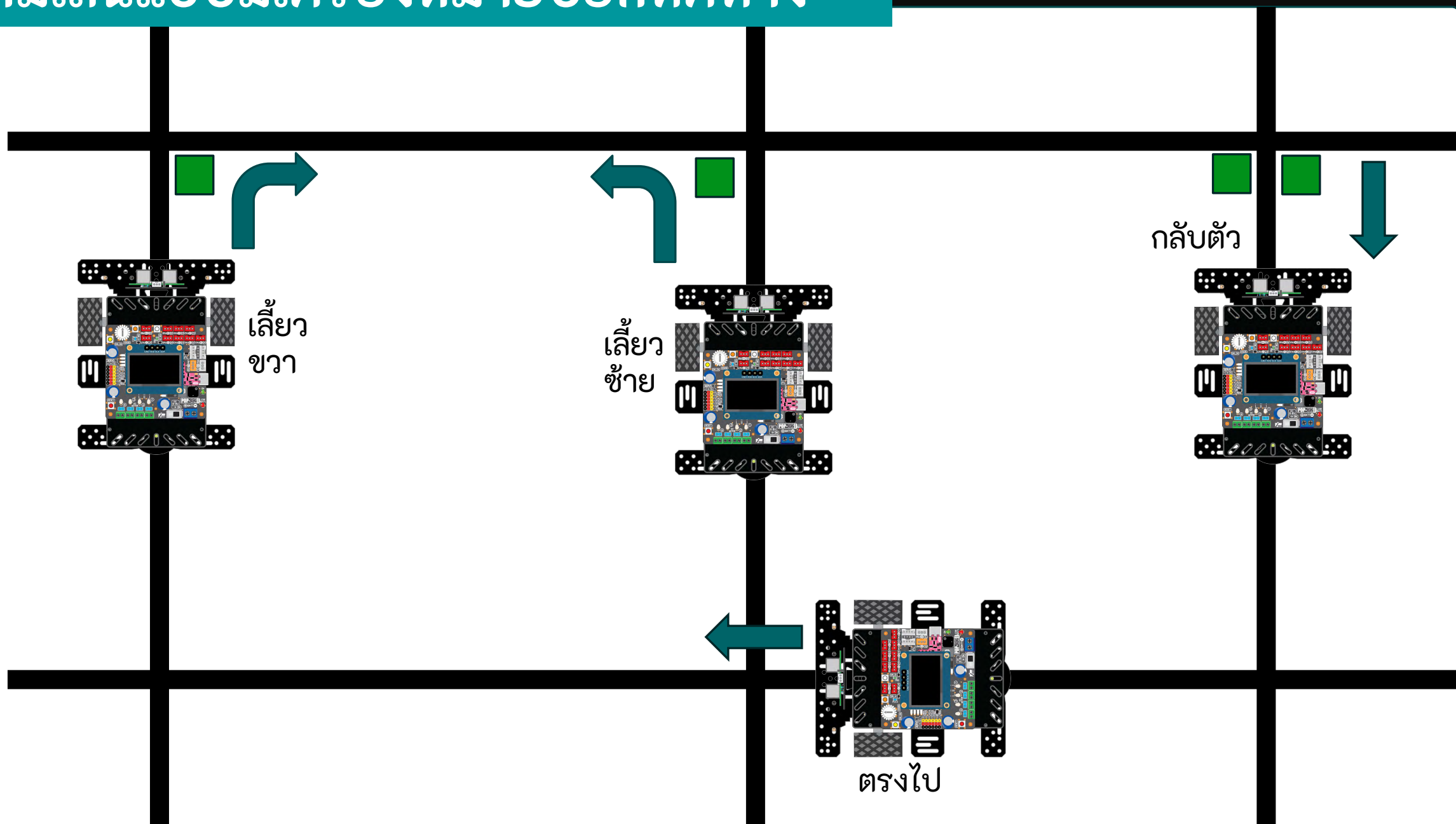
```
#include <POP32.h>
int min_S1 = 1900; //อ่านค่าเส้นสีดำ ด้านซ้าย
int max_S1 = 3700; //อ่านค่าพื้นสีขาว ด้านซ้าย
int min_S2 = 1900; //อ่านค่าเส้นสีดำ ด้านขวา
int max_S2 = 3700; //อ่านค่าพื้นสีขาว ด้านขวา
int maxSP = 50; //ความเร็วสูงสุดที่ต้องการ
float k = 1.0; //ค่าอัตราขยายของความไวเซนเซอร์
#define val_S1 map(analog(1), min_S1, max_S1, 0, 100)
#define val_S2 map(analog(2), min_S2, max_S2, 0, 100)
void setup() {
    while (!SW_OK()) { //กดปุ่ม OK เพื่อเริ่มต้นทำงาน
        oled.text(0, 0, "L=%d ", analog(1));
        oled.text(2, 0, "R=%d ", analog(2));
        oled.text(4, 0, "VS1=%d ", val_S1);
        oled.text(6, 0, "VS2=%d ", val_S2);
        oled.show();
    }
    beep();
}
void loop() {
    Tarck();R90();Tarck();L90();Tarck();ao();while(1);
}
```

Code -> Track_line_route.ino

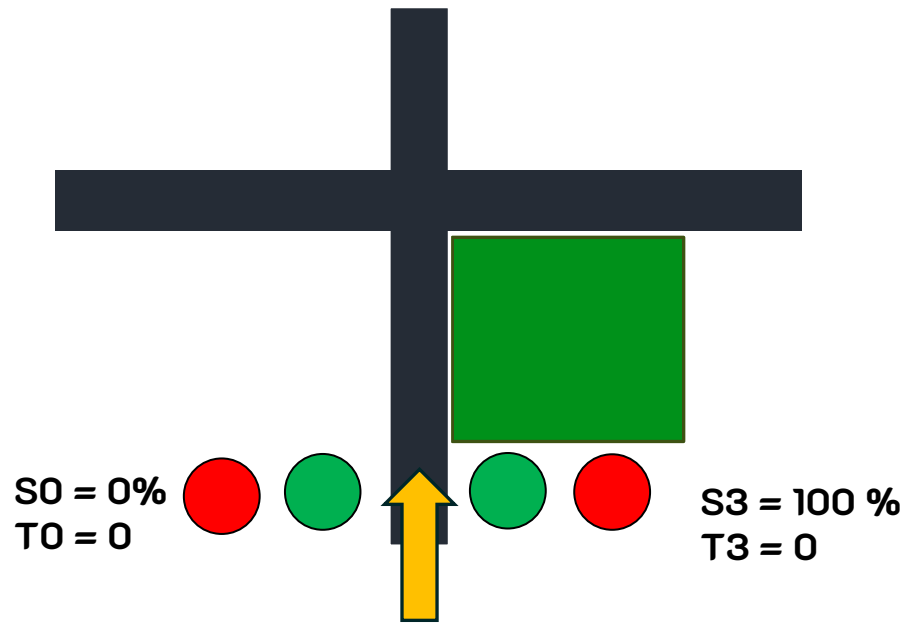
```
void Tarck(){
    while(1){
        if (val_S1 < 10 && val_S2 < 10) {
            break;
        } else {
            int SP_M1 = constrain(val_S1 * k, 0, maxSP);
            int SP_M2 = constrain(val_S2 * k, 0, maxSP);
            motor(1, SP_M1);
            motor(2, SP_M2);
        }
    }
}
void R90(){
    fd(30);sound(1500,150);
    while(val_S2<10){sr(30);}
    while(val_S2>10){sr(30);}
    while(val_S1>20){sr(30);}
}
void L90(){
    fd(30); sound(1500,150);
    while(val_S1<10){sl(30);}
    while(val_S1>10){sl(30);}
    while(val_S2>20){sl(30);}
}
void UTurn(){
    bk(30);sound(1500,450);
    while(val_S2<10){sr(30);}
    while(val_S2>10){sr(30);}
    while(val_S1>20){sr(30);}
}
```

ตัวอย่างการสร้างฟังก์ชันขับเคลื่อนหุ่นยนต์เมื่อพบ
เส้นตัดหรือทางแยก แบบที่ 2

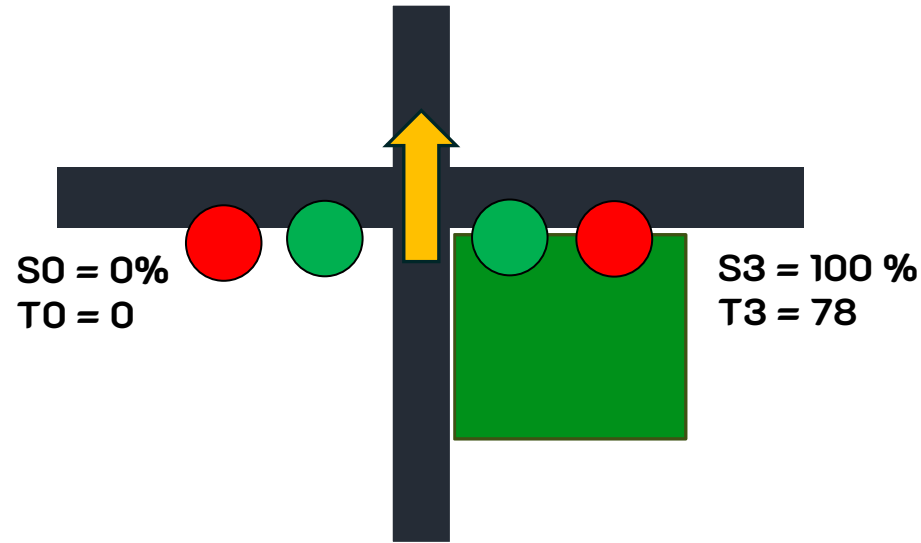
เดินตามเส้นแบบมีเครื่องหมายบอกทิศทาง



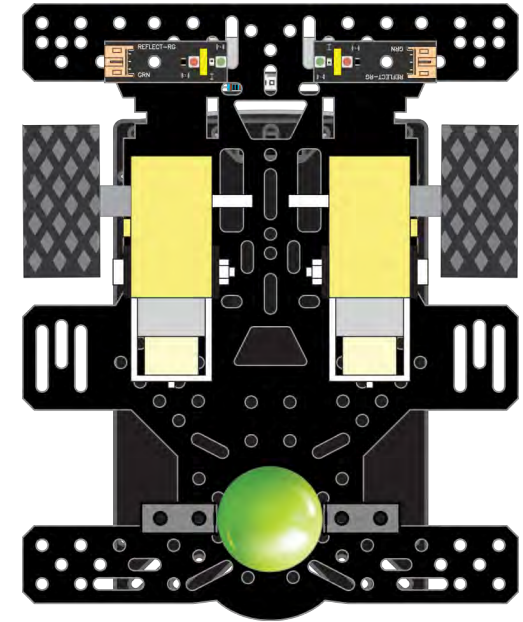
หลักแนวคิด เมื่อเซนเซอร์ตัวแสงสีแดงแต่ละข้าง เจอพื้นสีเขียวจะทำการเพิ่ม
ค่าขึ้นไปเรื่อยๆ เมื่อหุ่นยนต์เจอเส้นก็จะหยุด จากนั้นนำค่าที่ได้มาวิเคราะห์หา
ทิศทางที่จะเคลื่อนที่ทิศทางไหน



กรณีที่เซนเซอร์ S3 ข้างขวาเคลื่อนที่ผ่านพื้นสีเขียว



กรณีที่เซนเซอร์ S3 ข้างขวาเคลื่อนที่ผ่านพื้นสีเขียว
ค่าของ T3 จะเพิ่มขึ้น



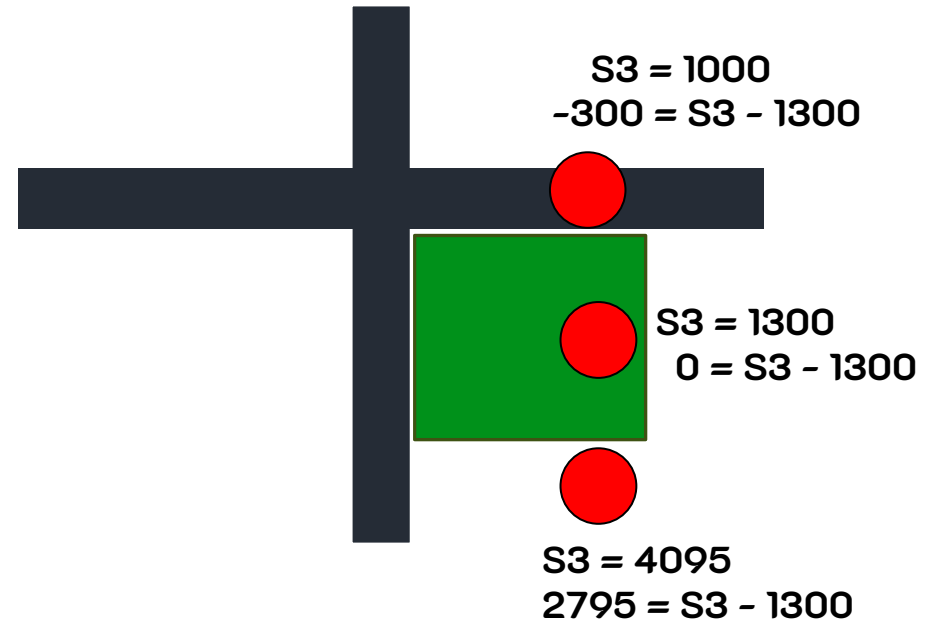
หลักการทดสอบ เราจะใช้เซนเซอร์สีแดงอ่านสีเขียวเทียบกับสีขาวและสีดำ

ตัวอย่าง

สีดำได้ = 1000

สีเขียว = 1300

สีขาว = 4095



หากเอาสีเขียวไปลบออกจากค่าที่อ่านได้ปัจจุบัน เมื่อเจอสีดำค่าจะเป็นลบและ
เมื่อเจอสีขาวค่าจะเป็นบวก โดยใช้สมการดังนี้

ผลลัพธ์ = ค่าอ่านออกปัจจุบัน - ค่าสีเขียว

$-100 = 1200 - 1300$; ใกล้เคียงสีเขียว

$0 = 1300 - 1300$; ใกล้เคียงสีเขียว

$100 = 1400 - 1300$; ใกล้เคียงสีเขียว

$200 = 1500 - 1300$; ไม่ใกล้เคียงสีเขียว

คำสั่ง

abs(value)

เป็นคำสั่งที่ใช้กำหนดค่าให้อยู่ในช่วงบวกเท่านั้น

ตัวอย่าง

```
int y = abs(-50)
```

y จะมีค่าเท่ากับ 50

โปรแกรมทดสอบอ่านค่าเครื่องหมายทางแยก

```
#include <POP32.h>
int tag_S0 = 1300; //อ่านค่าพื้นสีเขียว ด้านซ้าย
int tag_S3 = 1600; //อ่านค่าพื้นสีเขียว ด้านขวา
#define val_S0 abs(analog(0) - tag_S0)
#define val_S3 abs(analog(3) - tag_S3)

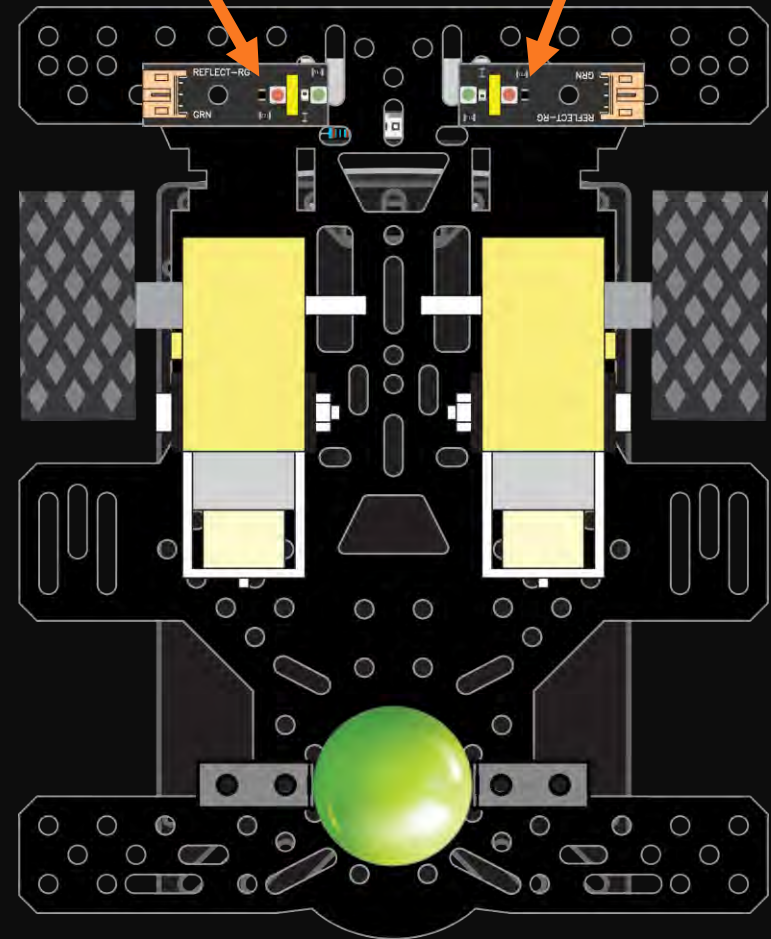
void setup() {
  while (!SW_OK()) { //กดปุ่ม OK เพื่อเริ่มต้นทำงาน
    oled.text(0, 0, "TagL=%d ", analog(0));
    oled.text(2, 0, "TagR=%d ", analog(3));
    oled.text(4, 0, "VS1=%d ", val_S0);
    oled.text(6, 0, "VS3=%d ", val_S3);
    oled.show();
  }
  beep();
}

void loop() {
}
```

Code -> Read_tag_green.ino

ต่อช่อง A3

ต่อช่อง A0



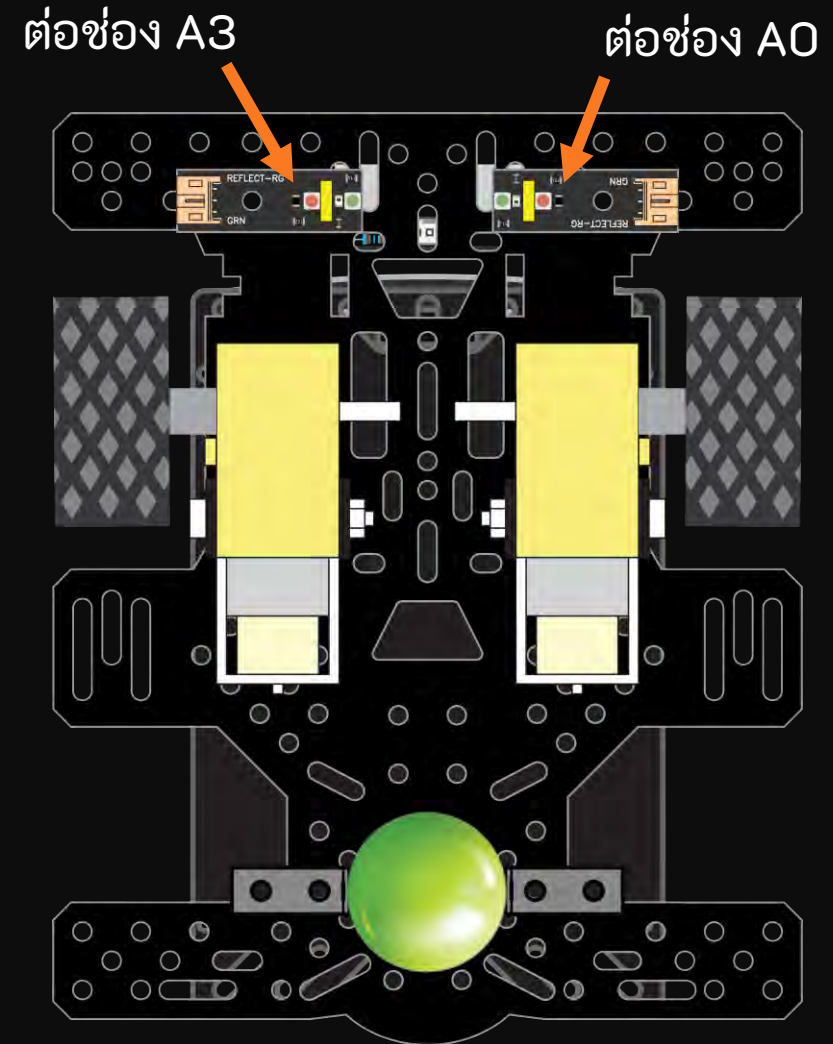
ทดสอบด้วยการเลื่อนหุ่นยนต์ไปยังเครื่องหมายทางแยก

สร้างฟังก์ชัน readTag()

```
int tag_S0 = 1300; //อ่านค่าพื้นสีเขียว ด้านซ้าย  
int tag_S3 = 1600; //อ่านค่าพื้นสีเขียว ด้านขวา  
int T0=0;  
int T3=0;  
#define val_S0 abs(analog(0) - tag_S0)  
#define val_S3 abs(analog(3) - tag_S3)
```

```
if(val_S0 <100) T0++;  
if(val_S3 <100) T3++;
```

จะแทรกเงื่อนไขตรวจสอบสีเขียวตลอดเวลาเมื่อหุ่นยนต์เคลื่อนที่



ทดสอบด้วยการเลื่อนหุ่นยนต์ไปยังเครื่องหมายทางแยก

โปรแกรมทดสอบอ่านค่าเครื่องหมายทางแยกด้วยการวิ่งตามเส้น

```
#include <POP32.h>
int min_S1 = 1900; //อ่านค่าเส้นสีดำ ด้านซ้าย
int max_S1 = 3700; //อ่านค่าพื้นสีขาว ด้านซ้าย
int min_S2 = 1900; //อ่านค่าเส้นสีดำ ด้านขวา
int max_S2 = 3700; //อ่านค่าพื้นสีขาว ด้านขวา
int maxSP = 50; //ความเร็วสูงสุดที่ต้องการ
float k = 1.0; //ค่าอัตราการขยายของความไวเซนเซอร์
#define val_S1 map(analog(1), min_S1, max_S1, 0, 100)
#define val_S2 map(analog(2), min_S2, max_S2, 0, 100)

int tag_S0 = 1300; //อ่านค่าพื้นสีเขียว ด้านซ้าย
int tag_S3 = 1600; //อ่านค่าพื้นสีเขียว ด้านขวา
int T0=0;
int T3=0;
#define val_S0 abs(analog(0) - tag_S0)
#define val_S3 abs(analog(3) - tag_S3)

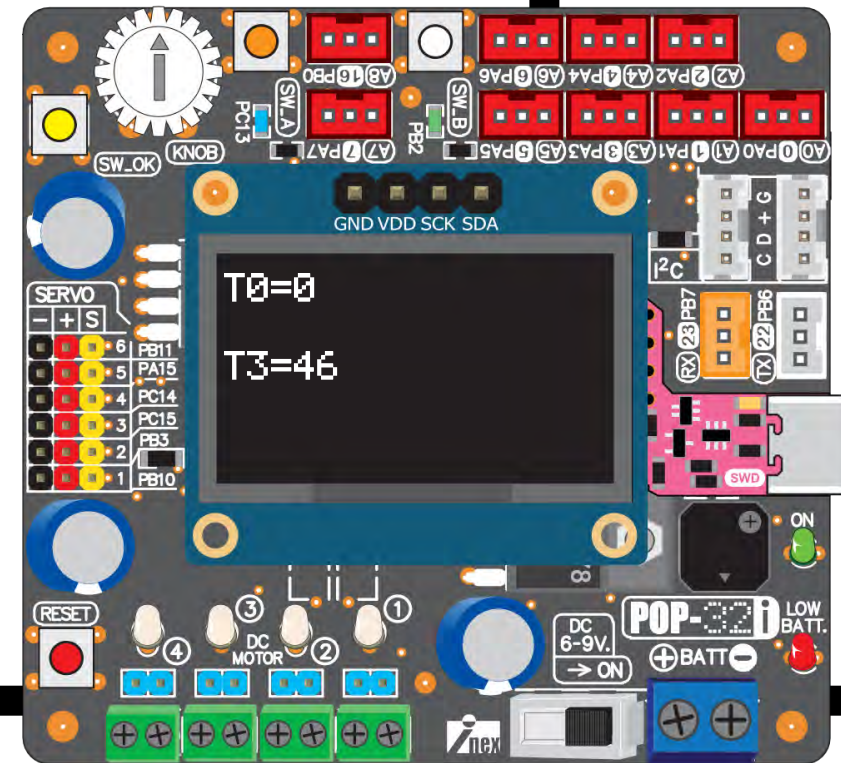
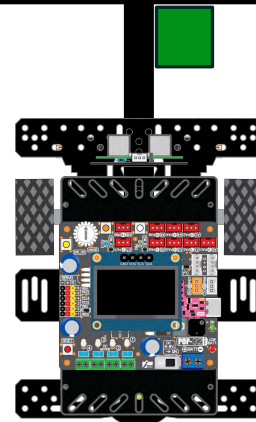
void setup() {
  while (!SW_OK()) { //กดปุ่ม OK เพื่อเริ่มต้นทำงาน
    oled.text(0, 0, "TagL=%d ", analog(0));
    oled.text(2, 0, "TagR=%d ", analog(3));
    oled.text(4, 0, "VS0=%d ", val_S0);
    oled.text(6, 0, "VS3=%d ", val_S3);
    oled.show();
  }
  beep();
}
```

Code -> Track_line_read_tag.ino

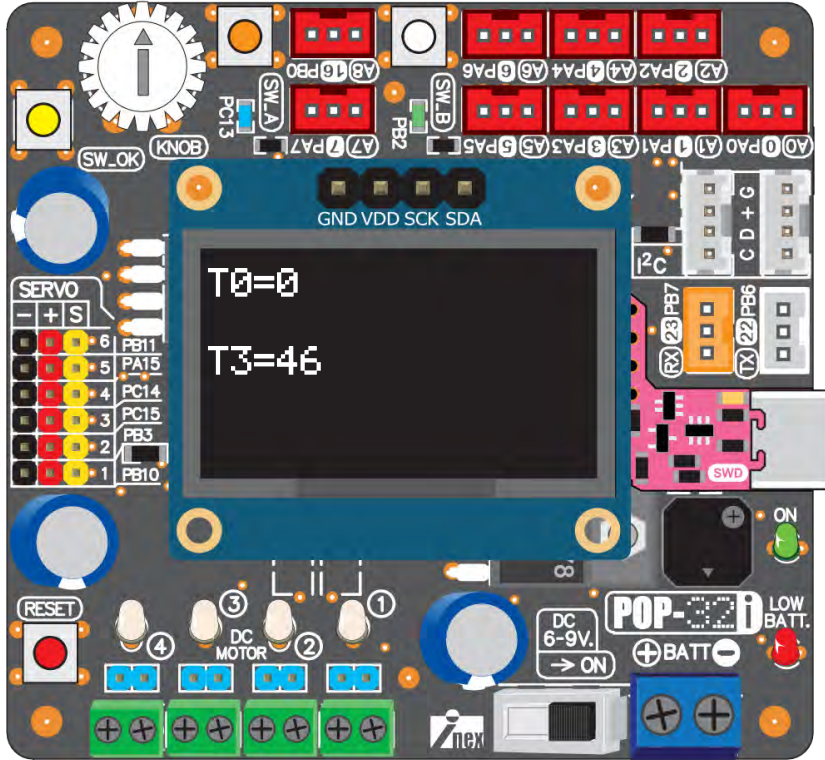
```
void loop() {
  Tarck();ao();oled.clear();
  while(!SW_OK()){
    oled.text(0, 0, "T0=%d ", T0);
    oled.text(2, 0, "T3=%d ", T3);
    oled.show();
  }
  beep();
  T0=0;
  T3=0;
}
void Tarck(){
  while(1){
    if (val_S1 < 10 && val_S2 < 10) {
      break;
    } else {
      if(val_S0 <100) T0++;
      if(val_S3 <100) T3++;
      int SP_M1 = constrain(val_S1 * k, 0, maxSP);
      int SP_M2 = constrain(val_S2 * k, 0, maxSP);
      motor(1, SP_M1);
      motor(2, SP_M2);
    }
  }
}
```

ทดสอบการเคลื่อนที่

เมื่อกดปุ่ม SW_OK หุ่นยนต์จะเดินหน้า
เจอแยกแล้วหยุด จากนั้นจะแสดงค่าที่
ตรวจสอบ Tag



สร้างฟังก์ชันกำหนดการเคลื่อนที่เมื่อเจอทางแยก



```
void getTag(){  
    int Gain = 20;  
    if(T0 < Gain && T3 < Gain){  
        fd(30);sound(1500,150);  
    }else if (T0 > Gain && T3 < Gain){  
        L90();  
    }else if (T0 < Gain && T3 > Gain){  
        R90();  
    }else if (T0 > Gain && T3 > Gain){  
        UTurn();  
    }  
}
```

จากการทดสอบพบว่า ในบางครั้งค่า T0 และ T3 ก็จะมีค่าเพิ่มขึ้นมาไม่เท่ากับ 0 ถึงแม้จะไม่พบสี่เหลี่ยมก็ตาม ดังนั้นตัวแปร Gain จะเป็นเงื่อนไขเอาไว้สำหรับระบุ Tag สี่เหลี่ยม

โปรแกรมทดสอบเคลื่อนที่เลี้ยวเมื่อเจอทางแยก

```
#include <POP32.h>
int min_S1 = 1900; //อ่านค่าเส้นสีดำ ด้านซ้าย
int max_S1 = 3700; //อ่านค่าพื้นสีขาว ด้านซ้าย
int min_S2 = 1900; //อ่านค่าเส้นสีดำ ด้านขวา
int max_S2 = 3700; //อ่านค่าพื้นสีขาว ด้านขวา
int maxSP = 50; //ความเร็วสูงสุดที่ต้องการ
float k = 1.0; //ค่าอัตราการขยายของความไวเซนเซอร์
#define val_S1 map(analog(1), min_S1, max_S1, 0, 100)
#define val_S2 map(analog(2), min_S2, max_S2, 0, 100)

int tag_S0 = 1300; //อ่านค่าพื้นสีเขียว ด้านซ้าย
int tag_S3 = 1600; //อ่านค่าพื้นสีเขียว ด้านขวา
int T0=0;
int T3=0;
#define val_S0 abs(analog(0) - tag_S0)
#define val_S3 abs(analog(3) - tag_S3)

void setup() {
  while (!SW_OK()) { //กดปุ่ม OK เพื่อเริ่มต้นทำงาน
    oled.text(0, 0, "TagL=%d ", analog(0));
    oled.text(2, 0, "TagR=%d ", analog(3));
    oled.text(4, 0, "VS1=%d ", val_S0);
    oled.text(6, 0, "VS3=%d ", val_S3);
    oled.show();
  }
  beep();
}
```

Code -> Track_line_get_tag.ino

```
void loop() {
  Tarck(); getTag();ao();
  oled.clear();
  while(!SW_OK()){
    oled.text(0, 0, "T0=%d ", T0);
    oled.text(2, 0, "T3=%d ", T3);
    oled.show();
  }
  beep();
  T0=0;
  T3=0;
}

void getTag(){
  int Gain = 20;
  if(T0 < Gain && T3 < Gain){
    fd(30);sound(1500,150);
  }else if (T0 > Gain && T3 < Gain){
    L90();
  }else if (T0 < Gain && T3 > Gain){
    R90();
  }else if (T0 > Gain && T3 > Gain){
    UTurn();
  }
}
```


โปรแกรมทดสอบเคลื่อนที่เลี้ยวเมื่อเจอทางแยก (ต่อ)

```
void R90(){
    fd(30);sound(1500,150);
    while(val_S2<10){sr(30);}
    while(val_S2>10){sr(30);}
    while(val_S1>20){sr(30);}
}
void L90(){
    fd(30); sound(1500,150);
    while(val_S1<10){sl(30);}
    while(val_S1>10){sl(30);}
    while(val_S2>20){sl(30);}
}
void UTurn(){
    bk(30);sound(1500,450);
    while(val_S2<10){sr(30);}
    while(val_S2>10){sr(30);}
    while(val_S1>20){sr(30);}
}
```

```
void Tarck(){
    while(1){
        if (val_S1 < 10 && val_S2 < 10) {
            break;
        } else {
            if(val_S0 <100) T0++;
            if(val_S3 <100) T3++;
            int SP_M1 = constrain(val_S1 * k, 0, maxSP);
            int SP_M2 = constrain(val_S2 * k, 0, maxSP);
            motor(1, SP_M1);
            motor(2, SP_M2);
        }
    }
}
```

การทำงาน กดปุ่ม SW_OK หุ่นยนต์จะเคลื่อนที่เมื่อเจอแยกจะเลี้ยวหรือตรงไปตาม Tag ที่ตรวจพบ จากนั้นหุ่นยนต์จะหยุดพร้อมแสดงค่า T0 และ T3 หากต้องการไปต่อให้ กดปุ่ม SW_OK อีกครั้ง หากเลี้ยวผิดให้สังเกตที่ T0 และ T3 ว่าค่า Gain ที่กำหนดไว้สำหรับเลี้ยวอาจจะค่าน้อยไปหรือมากไป

โปรแกรมเคลื่อนที่เลี้ยวเมื่อเจอทางแยก

```
#include <POP32.h>
int min_S1 = 1700; //อ่านค่าเส้นสีดำ ด้านซ้าย
int max_S1 = 4095; //อ่านค่าพื้นสีขาว ด้านซ้าย
int min_S2 = 1900; //อ่านค่าเส้นสีดำ ด้านขวา
int max_S2 = 3700; //อ่านค่าพื้นสีขาว ด้านขวา
int maxSP = 50; //ความเร็วสูงสุดที่ต้องการ
float k = 1.0; //ค่าอัตราการขยายของความไวเซนเซอร์
#define val_S1 map(analog(1), min_S1, max_S1, 0, 100)
#define val_S2 map(analog(2), min_S2, max_S2, 0, 100)

int tag_S0 = 1300; //อ่านค่าพื้นสีเขียวก ด้านซ้าย
int tag_S3 = 1700; //อ่านค่าพื้นสีเขียวก ด้านขวา
int T0=0;
int T3=0;
#define val_S0 abs(analog(0) - tag_S0)
#define val_S3 abs(analog(3) - tag_S3)

void setup() {
    while (!SW_OK()) { //กดปุ่ม OK เพื่อเริ่มต้นทำงาน
        oled.text(0, 0, "TagL=%d ", analog(0));
        oled.text(2, 0, "TagR=%d ", analog(3));
        oled.text(4, 0, "VS1=%d ", val_S0);
        oled.text(6, 0, "VS3=%d ", val_S3);
        oled.show();
    }
    beep();
}
```

Code -> Track_line_tag_route.ino

```
void loop() {
    Tarck();
}
void Tarck(){
    while(1){
        if (val_S1 < 10 && val_S2 < 10) {
            getTag();
            T0=0;
            T3=0;
        } else {
            if(val_S0 <100) T0++;
            if(val_S3 <100) T3++;
            int SP_M1 = constrain(val_S1 * k, 0, maxSP);
            int SP_M2 = constrain(val_S2 * k, 0, maxSP);
            motor(1, SP_M1);
            motor(2, SP_M2);
        }
    }
}
```

โปรแกรมเคลื่อนที่เลี้ยวเมื่อเจอทางแยก (ต่อ)

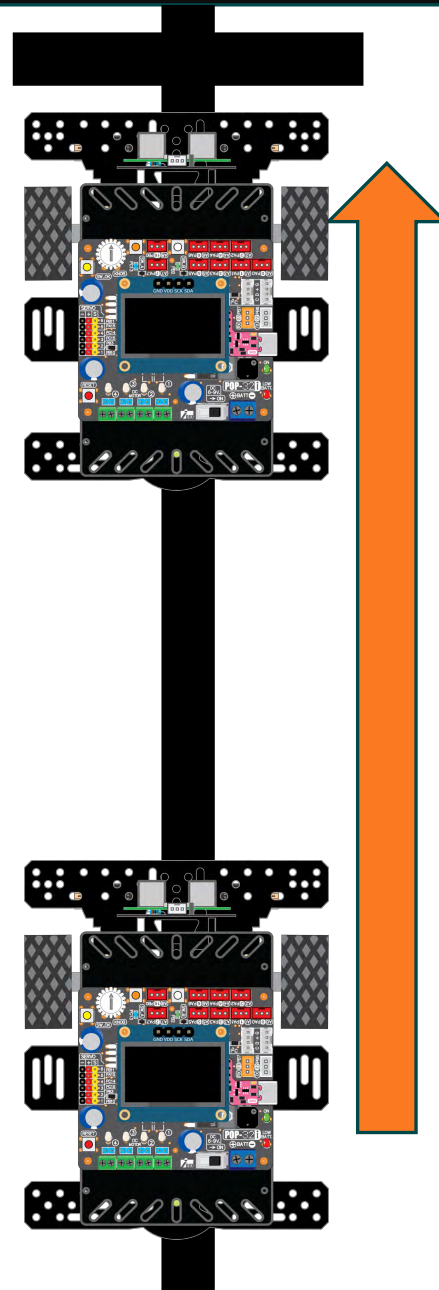
```
void R90(){
    fd(30);sound(1500,150);
    while(val_S2<10){sr(30);}
    while(val_S2>10){sr(30);}
    while(val_S1>20){sr(30);}
}
void L90(){
    fd(30); sound(1500,150);
    while(val_S1<10){sl(30);}
    while(val_S1>10){sl(30);}
    while(val_S2>20){sl(30);}
}
void UTurn(){
    bk(30);sound(1500,450);
    while(val_S2<10){sr(30);}
    while(val_S2>10){sr(30);}
    while(val_S1>20){sr(30);}
}
```

```
void getTag(){
    int Gain = 20;
    if(T0 < Gain && T3 < Gain){
        fd(30);sound(1500,150);
    }else if (T0 > Gain && T3 < Gain){
        L90();
    }else if (T0 < Gain && T3 > Gain){
        R90();
    }else if (T0 > Gain && T3 > Gain){
        UTurn();
    }
}
```

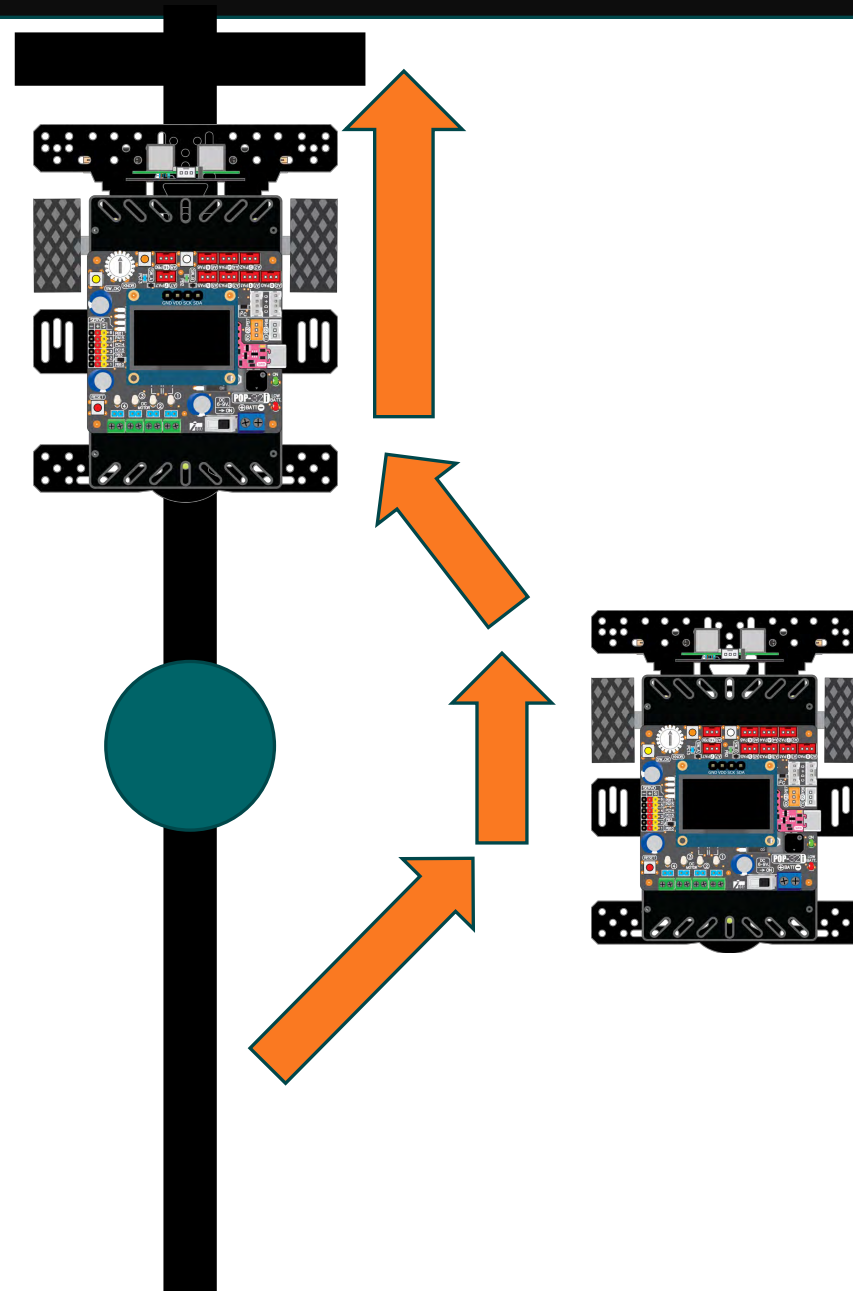
การทำงาน กดปุ่ม SW_OK หุ่นยนต์จะเคลื่อนที่เมื่อเจอแยกจะเลี้ยวหรือตรงไปตาม Tag ที่ตรวจพบ

ตัวอย่างการสร้างฟังก์ชันหลบหลีกสิ่งกีดขวางในขณะ
หุ่นยนต์เคลื่อนที่ตามเส้นอย่างง่าย

เมื่อไม่พบเดินตรงไปตาม
จอเส้นตัดทำตามเงื่อนไข

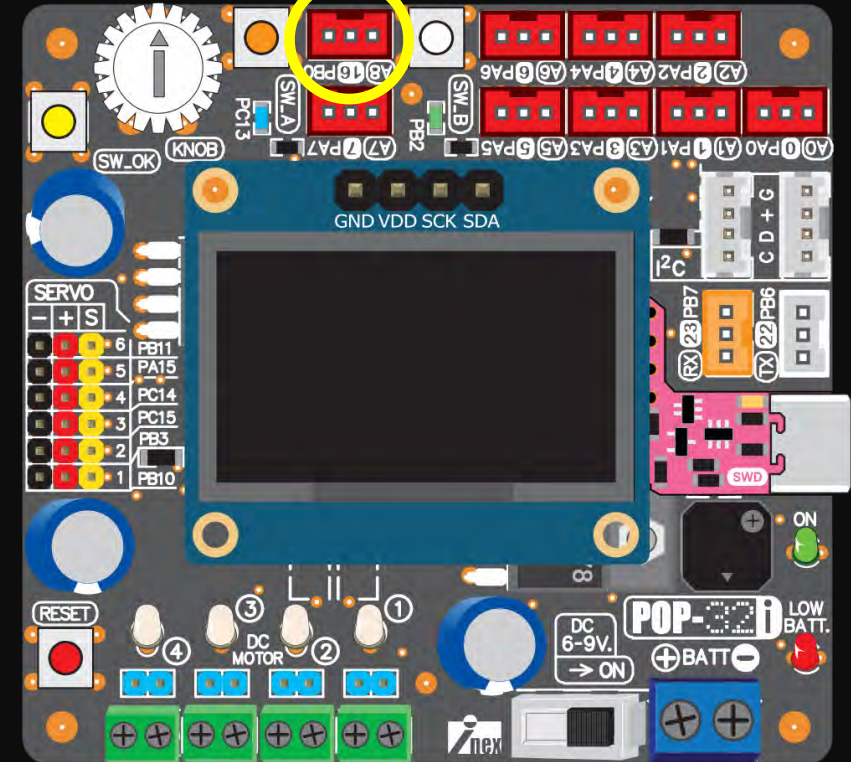
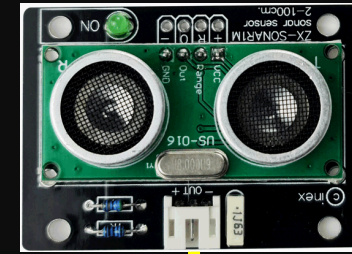


เมื่อพบเส้นตัดทำตามเงื่อนไข แต่
ในระหว่างทางพบสิ่งกีดขวางให้
หลบสิ่งกีดขวางด้วย



ทดสอบ อ่านค่าระยะทางจาก ZX-Sonar 1M

```
#include <POP32.h>
void setup(){}
void loop()
{
    int dist = analog(A8)/41;
    oled.text(0,0,"dist: %d cm ",dist);
    oled.show();
}
```



ฟังก์ชันเจอสิ่งกีดขวาง แล้วกลับเข้าเส้น

```
void fd_Check_dis() {  
    int Dist = analog(A8) / 41;  
    if (Dist < 20) {  
        sr(30);delay(400);  
        fd(20);delay(1500);  
        sl(30);delay(300);  
        fd(20);delay(1500);  
        while (val_S2 > 10) { fd2(10, 30); }  
        while (val_S2 < 10) { fd2(10, 30); }  
        R90();  
    }  
}
```

เมื่อระยะห่างที่ตั้งไว้น้อยกว่า 20 จะเข้าเงื่อนไข
หลบสิ่งกีดขวาง

โปรแกรมเดินตามเส้นตัดหลบสิ่งกีดขวาง

```
#include <POP32.h>
int min_S1 = 1900; //อ่านค่าเส้นสีดำ ด้านซ้าย
int max_S1 = 3700; //อ่านค่าพื้นสีขาว ด้านซ้าย
int min_S2 = 1900; //อ่านค่าเส้นสีดำ ด้านขวา
int max_S2 = 3700; //อ่านค่าพื้นสีขาว ด้านขวา
int maxSP = 50; //ความเร็วสูงสุดที่ต้องการ
float k = 1.0; //ค่าอัตราการขยายของความไวเซนเซอร์

#define val_S1 map(analog(1), min_S1, max_S1, 0, 100)
#define val_S2 map(analog(2), min_S2, max_S2, 0, 100)
#define Dist (analog(A8) / 41)

void setup() {
    while (!SW_OK()) { //กดปุ่ม OK เพื่อเริ่มต้นทำงาน
        oled.text(1, 0, "L=%d ", analog(1));
        oled.text(2, 0, "R=%d ", analog(2));
        oled.text(3, 0, "VS1=%d ", val_S1);
        oled.text(4, 0, "VS2=%d ", val_S2);
        oled.text(6, 0, "Dis=%d ", Dist);
        oled.show();
    }
    beep();
}
```

Code -> Track_line_object.ino

```
void loop() {
    if (val_S1 < 10 && val_S2 < 10) {
        R90();
    } else {
        fd_Check_dis();
        int SP_M1 = constrain(val_S1 * k, 0, maxSP);
        int SP_M2 = constrain(val_S2 * k, 0, maxSP);
        motor(1, SP_M1);
        motor(2, SP_M2);
    }
}

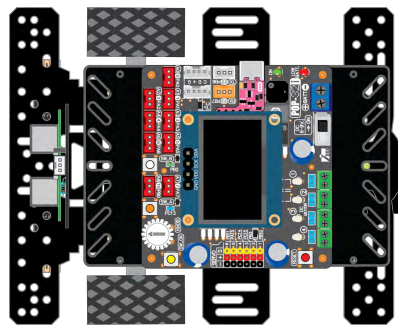
void fd_Check_dis() {
    if (Dist < 20) {
        sr(30);delay(400);fd(20);delay(1500);
        sl(30);delay(300);fd(20);delay(1500);
        while (val_S2 > 10) { fd2(10, 30); }
        while (val_S2 < 10) { fd2(10, 30); }
        R90();
    }
}
```

โปรแกรมเดินตามเส้นตัดหลบสิ่งกีดขวาง (ต่อ)

```
void R90(){  
    fd(30);sound(1500,150);  
    while(val_S2<10){sr(30);}   
    while(val_S2>10){sr(30);}   
    while(val_S1>20){sr(30);}   
}
```

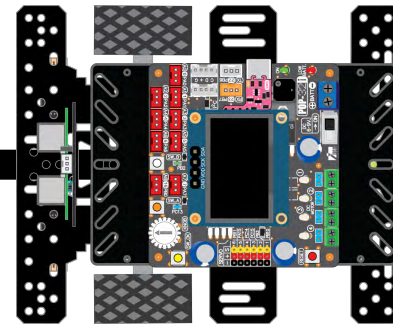
ตัวอย่างโปรแกรมเดินตามเส้นตัดหลบสิ่งกีดขวาง
Track_line_tag_object.ino

การกำหนดเวลาในงานควบคุม



หยุดเวลา

เวลา วินาที



เริ่มจับเวลา

คำสั่ง

millis()

เป็นคำสั่งที่ใช้อ่านค่าเวลาที่กำลังเดิมอยู่มีหน่วยเป็น มิลลิวินาที

ตัวอย่าง

```
unsigned long Counter = millis()
```

Counter จะมีค่าเป็นเวลานับเพิ่มขึ้นไปเรื่อยๆ

สร้างโปรแกรมออกจาก while โดยใช้เวลาในการหยุด

```
StartTime = millis();  
EndTime   = StartTime + TotalTime;  
while (millis() <= EndTime) {  
    command1;  
    command2;  
    คำสั่งต่างๆที่เกี่ยวข้องในโปรแกรม  
    command3;  
    command4;  
}
```

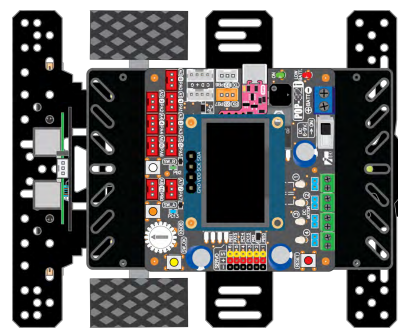

โปรแกรมจับเวลาการทำงาน

```
#include <POP32.h>
unsigned long TotalTime = 5000;
unsigned long StartTime = 0;
unsigned long EndTime = 0;
void setup() {
  StartTime = millis();
  EndTime = StartTime + TotalTime;
  while (millis() <= EndTime) {
    oled.text(0, 0, "S_Time = %d  ", StartTime);
    oled.text(2, 0, "E_Time = %d  ", EndTime);
    oled.text(4, 0, "Countdown = %d  ", EndTime-millis());
    oled.show();
    if (SW_OK()) {
      StartTime = millis();
      EndTime = StartTime + TotalTime;
      beep();
    }
  }
  oled.clear();
  oled.text(0, 0, "Finish");
  oled.show();
}
void loop() {
}
```

Code -> Counter_Timer.ino

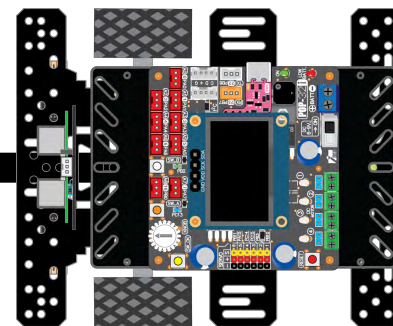
หน้าจอจะแสดงค่าเริ่มต้นเวลาสิ้นสุด
และเวลาที่เหลืออยู่ หากต้องการต่อ
เวลาให้กดปุ่ม SW_OK()

ส่งหยุดเคลื่อนที่เมื่อไม่เจอเส้นในเวลาที่กำหนด



หยุดเวลา

เวลา วินาที



เริ่มจับเวลา

โปรแกรมจับเวลาการทำงาน

```
int TotalTime = 1000; //ระยะเวลาที่หุ่นยนต์ไม่พบเส้น
unsigned long EndTime = 0;

void Tarck(){
  EndTime = millis() + TotalTime;
  while(millis() <= EndTime){
    if (val_S1 < 10 && val_S2 < 10) {
      getTag();
      T0=0;
      T3=0;
      EndTime = millis() + TotalTime;
    } else {
      fd_Check_dis();
      if(val_S0 <100) T0++;
      if(val_S3 <100) T3++;
      int SP_M1 = constrain(val_S1 * k, 0, maxSP);
      int SP_M2 = constrain(val_S2 * k, 0, maxSP);
      motor(1, SP_M1);
      motor(2, SP_M2);
    }
  }
}
```

Code -> Track_line_tag_object_timer.ino

ในฟังก์ชัน Tarck จากแต่ก่อนคำสั่ง while(1) จะวนทำซ้ำไปตลอด เปลี่ยนมาเป็น while(millis() <= EndTime) เมื่อหมดเวลาจะหยุดการทำงานทันที และจะมีชุดคำสั่ง EndTime = millis() + TotalTime; ค่อยต่อเวลาไปเรื่อยๆที่เจอเส้นตัด เมื่อไม่เจอเส้นตัดในเวลาที่กำหนด จะออกจากฟังก์ชัน Tarck ทันที และจะต่อเวลาอีกครั้งเมื่อหลบสิ่งกีดขวางฟังก์ชัน fd_Check_dis() เสร็จแล้ว

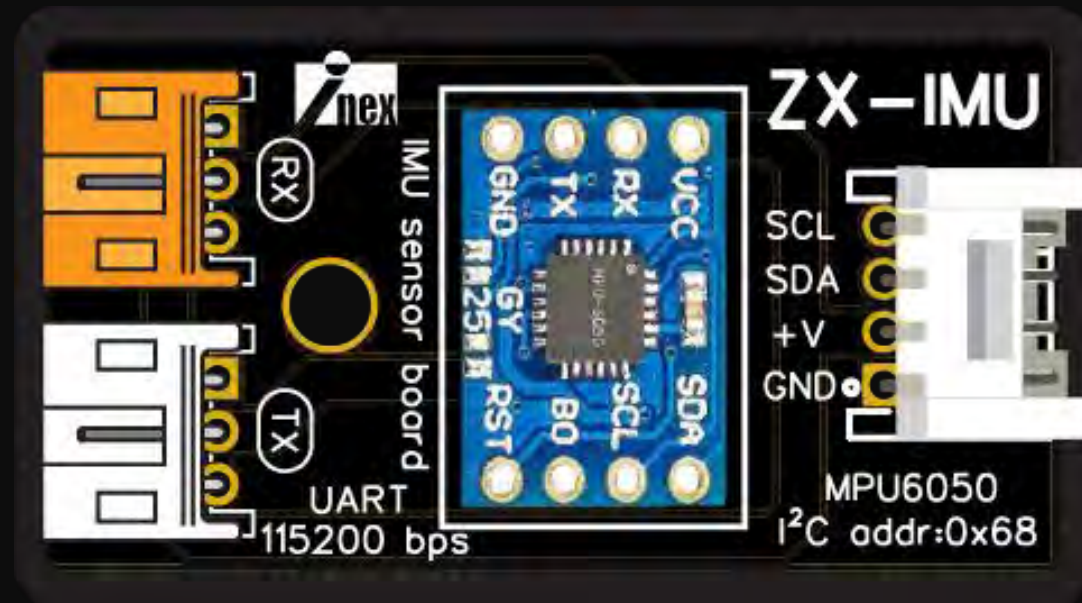
โปรแกรมจับเวลาการทำงานในฟังก์ชัน Tarck (ต่อ)

```
void loop() {  
  Tarck();  
  ao();  
  oled.clear();  
  oled.text(0, 0, "Finish");  
  oled.show();  
  while(1);  
}
```

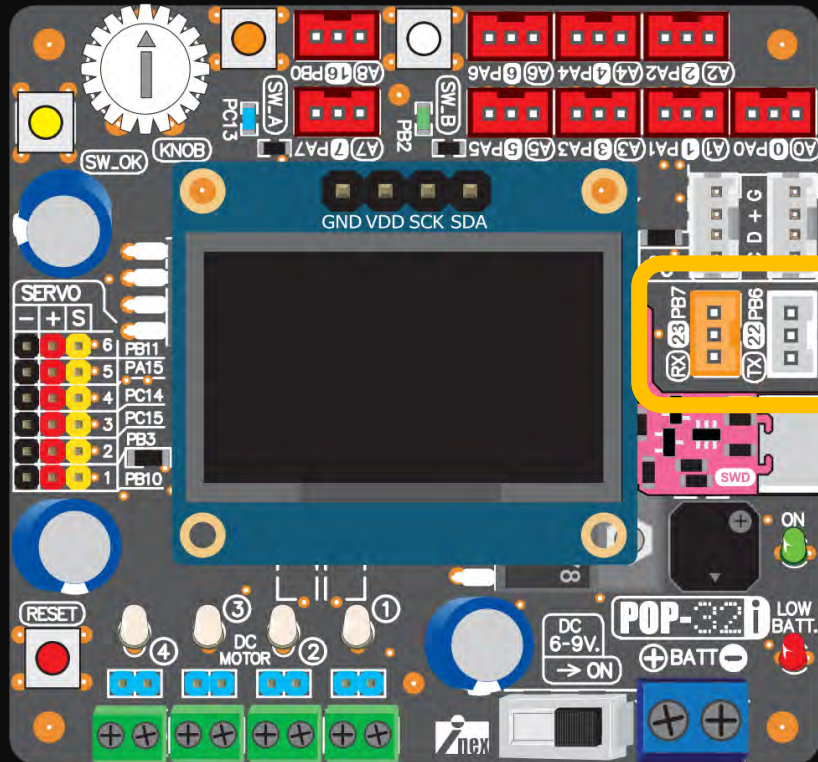
ในฟังก์ชัน Loop จะเรียกใช้ Tarck จนกว่าจะหมดเวลา
จากนั้นหุ่นยนต์ก็จะหยุดและหน้าจอจะแสดง Finish

ZX-IMU

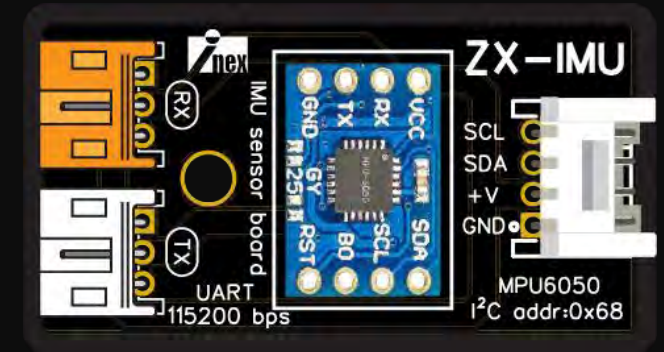
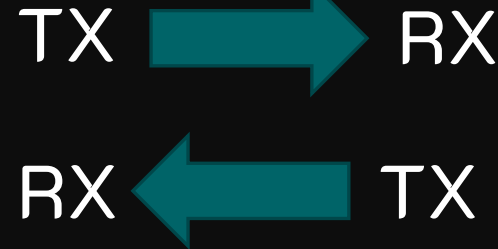
สามารถต่อใช้งานได้ทั้ง I2C(SDA,SCL) และ Serial(Rx,Tx) โมดูลประกอบไปด้วย MPU6050 เป็นเซนเซอร์ Gyro และ Accelerometer โดยมีไมโครคอนโทรลเลอร์ ใช้เป็นตัวกลางเพื่ออ่านค่าจาก MPU6050 และส่งค่าออก Serial(Rx,Tx)

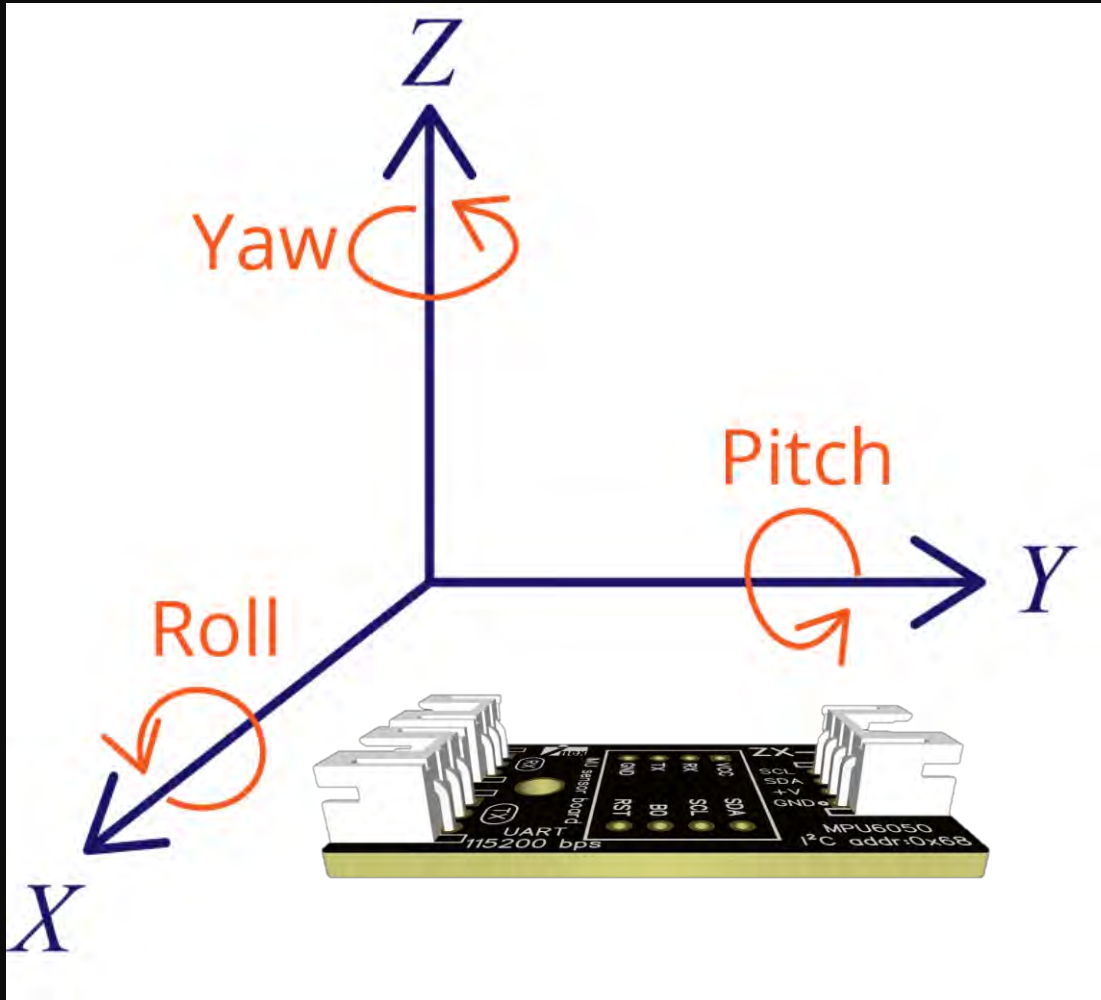


การต่อใช้งาน



ใช้แบบ UART





Yaw

นำมาใช้เป็นเข็มทิศอ้างอิงให้หุ่นยนต์

คำสั่ง

Serial.begin(baud rate)

กำหนดอัตราการสื่อสารข้อมูลเช่น 115200 bs

Serial.write(data)

ส่งข้อมูลออกไป ได้เพียงแค่ว่าละ 1 byte

Serial.available()

จำนวนไบต์ที่สามารถอ่านได้

Serial.read()

อ่านข้อมูลอนุกรมที่เข้ามา

ฟังก์ชันรีเซ็ตมุม Yaw

ขณะรีเซ็ตหุ่นยนต์ต้องอยู่นิ่งๆ อย่างน้อย
3 วินาที ก่อนใช้งาน

เริ่มต้นใช้งาน Serial ช่องหมายเลข 1

ปรับปรุงค่ามุม pitch และ roll ให้เป็น 0

```
void zeroYaw() {  
  Serial1.begin(115200);delay(100);  
  Serial1.write(0XA5);Serial1.write(0X54);delay(100);  
  Serial1.write(0XA5);Serial1.write(0X55);delay(100);  
  Serial1.write(0XA5);Serial1.write(0X52);delay(100);  
}
```

เลือกโหมดอัตโนมัติ

ปรับปรุงค่ามุม Yaw ให้เป็น 0

ฟังก์ชันอ่านค่า Yaw

```
float pvYaw;  
uint8_t rxCnt = 0, rxBuf[8];  
bool getIMU() {  
    while (Serial1.available()) {  
        rxBuf[rxCnt] = Serial1.read();  
        if (rxCnt == 0 && rxBuf[0] != 0xAA) return false;  
        rxCnt++;  
        if (rxCnt == 8) {  
            rxCnt = 0;  
            if (rxBuf[0] == 0xAA && rxBuf[7] == 0x55) {  
                pvYaw = (int16_t)(rxBuf[1] << 8 | rxBuf[2]) / 100.f;  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

เมื่อมีข้อมูลส่งเข้ามา

อ่านค่าเก็บไว้ในตัวแปร

ตรวจสอบความถูกต้อง

ตรวจสอบความถูกต้อง

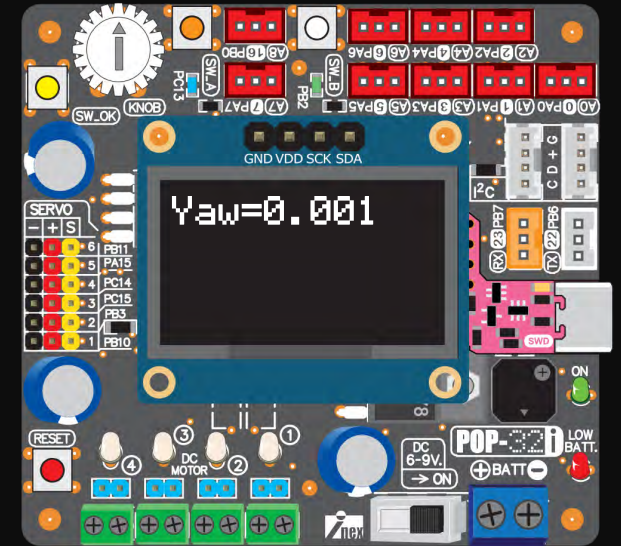
อ่านและคำนวณค่ามุม Yaw

สร้างโปรแกรมอ่านค่าYaw

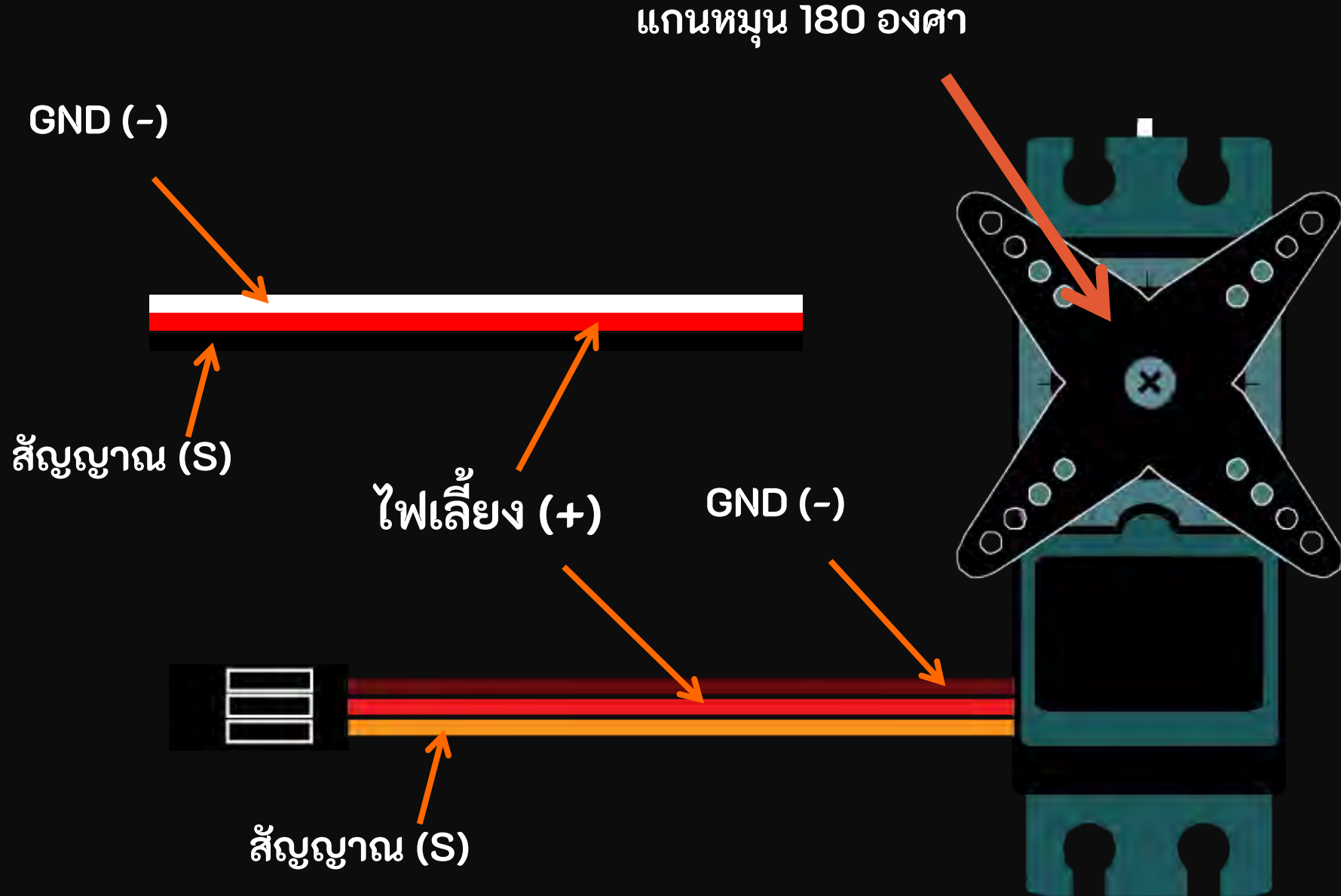
Code -> Test_ZX-IMU.ino

```
#include <POP32.h>
void setup() {
  zeroYaw();
}
void loop() {
  if (SW_A()) {
    zeroYaw();
    beep();
  }
  if (getIMU()) {
    oled.text(0, 0, "Yaw=%f", pvYaw);
    oled.show();
  }
}
```

กดปุ่ม A เพื่อรีเซ็ตเข็มทิศ
จากนั้นหมุนหุ่นยนต์เพื่อทดสอบ



การใช้งานเซอร์โวมอเตอร์



ฟังก์ชันขับเซอร์โวมอเตอร์

servo (CH, POS) ;

CH ช่องที่ใช้ขับ = 1 ถึง 6

POS ตำแหน่งองศาเซอร์โว = 0-180 ,-1

ค่า -1 หมายถึงหยุดจ่ายสัญญาณให้เซอร์โว
เซอร์โวจะไม่ล็อกแกน

ทดสอบ ควบคุมตำแหน่งเซอร์โวอย่างง่าย

```
#include <POP32.h>
void setup() {
  servo(1,90);
  oled.text(0,0,"Servo: 90 ");
  oled.show();
}
void loop() {
  if(SW_A()){
    servo(1,20);
    oled.text(0,0,"Servo: 20 ");
    oled.show();
    delay(500);
  }
  if(SW_B()){
    servo(1,160);
    oled.text(0,0,"Servo: 160");
    oled.show();
    delay(500);
  }
}
```


คำสั่งวนทำซ้ำแบบมีเงื่อนไข **for**

รูปแบบ

```
for (ค่าเริ่มต้น; เงื่อนไข; เพิ่ม/ลดค่า)
{
    //statement(s);
}
```

ออกจาก for แบบทันทีได้ด้วยคำสั่ง

break;

```
#include <POP32.h>
void setup() {
    waitSW_OK_bmp();
}
void loop() {
    for (int i = 0; i < 5; i++) {
        oled.text(0, 0, "%d ", i);
        oled.show();
        delay(100);
    }
}
```

ทดสอบ ควบคุมตำแหน่งเซอร์โวมอเตอร์ปรับความเร็ว

```
#include <POP32.h>
void setup() {
    servo(1,10);
    waitSW_OK_bmp();
}
void loop() {
    if(SW_A()){
        for(int i = 10;i<=176;i+=2){
            servo(1,i);
            oled.text(0, 0, "%d  ", i);
            oled.show();
        }
    }
    if(SW_B()){
        for(int i = 176;i >= 10;i-=2){
            servo(1,i);
            oled.text(0, 0, "%d  ", i);
            oled.show();
        }
    }
}
```